Client internals and version profiles, part 1 of 4

Client internals and version profiles, part 1 of 4

This part contains 50 notes, from `client-internals/bare-level-names.md` through `client-internals/decomp-findings-2026-10-25-e/04-name-index-case-link.md`.

Notes under `client-internals/`, then `version-profiles/`, each in path order.

`client-internals/bare-level-names.md`

Bare level names and the `.zelda` extension

Which client tries `.zelda` for a level name with no extension, and whether a missing `.zelda` falls back to the `.graal` of the same stem. The answer is that none does, in any client read.

Read in 26 Windows client builds: 1st Graal to 15th Graal (with the 4th, 5th and 9th alphas), 1.1, 1.1.9, 1.21, 1.28 beta 2, 1.3 beta, 1.31, 1.32, and 2.05.

Findings

- **No client adds `.zelda`.** No build carries `.zelda` as a bare extension string. Its only `.zelda` strings are full built-in level names, such as `level1.zelda`, `cave2.zelda`, `mod26.zelda` and `house12.zelda`. Those are compared against the current level's name or used as fixed warp targets, and are present in 1st Graal to 9th Graal.
- **The one extension rule adds `.graal`, and only in the editor.** Every build has the same rule in the level editor's open and save buttons, and in 1.3 and later also the snapshot save: when the chosen file name has no extension, `.graal` is added.
- **No fallback.** No build has code that swaps one extension for the other. A link or warp to `name.zelda` loads `name.zelda` as written; a missing file is just missing.
- **Contents still decide the format.** The loader decides the format from the file's magic, not its extension (`decomp-findings-2026-10-25-j/03-extensions-and-magic.md`), so a `.graal` name holding a `Z3-V1` level still loads.

For a reimplementation

- Keep a level name exactly as the link, warp or server wrote it.
- Add `.graal` only when the editor saves a name typed without one.
- Pick the reader from the magic.

Confirmed, inferred, not tested

- Confirmed from the programs: there is no bare `.zelda` string, the editor adds `.graal`, and no extension-swap code exists.
- Inferred: that the level loader uses the name unchanged. That follows from there being no other extension string near the load path; the loader's file-open call was not traced to its end.
- Not tested: the Java Zelda Online client, and builds between 1.32 and 2.05 other than those listed.

`client-internals/board-respawn.md`

Bush, pot and stone respawn: who restores the board, when, and who drops items

Date: 2026-10-30

How cut bushes and grass, lifted bushes, signs, pots (vases) and stones, and other board changes come back, and where the rupees and hearts they drop come from. Covers the 1999 GServer13 (shipped with 1.3 test), the 1.3 test client, the 2.05 rev 2 client, and the GServer-v2 lab server.

Tags: **[bin]** read from the binary · **[live]** measured with the real program under Wine (`tools/respawn-lab/`) · **[cap]** earlier lab capture (`protocol/packet-layouts.md`) · **[src]** GServer-v2 source · **[doc]** the official `newfeatures*.txt` / `readme.txt` · **[inf]** inferred.

Summary

	1.3 era (GServer13 + 1.3 client)	2.00 to 2.05 (client)	GServer-v2 lab server
Who restores a cut or lifted object	Server only	Server only	Server only
Clients regrow anything on their own	No	No, offline nothing ever comes back **[live]**	n/a
Which changes respawn	Any change that is exactly **2 × 2** **[bin][live]**	n/a	A change whose top-left tile was one of 11 listed ids **[src]**
Timer	`trunc(R·rand·0.5 + 0.75·R)` one-second ticks, R = `respawntime`, default **10**, so 6 to 12 s **[bin][live]**	n/a	Exactly `respawntime`, default **15** **[src][cap]**
Restore packet	`PLO_BOARDMODIFY` (7) with the old tiles, to everyone in the level	handled as any board modify	same
Sender gets its own change echoed	No **[bin][live]**	n/a	Yes, to the whole level **[cap]**
Item drops from bushes and pots	**Client**, local random roll, sent peer to peer, and from 1.38 also to the server (`PLI_ITEMADD`) **[bin] [cap]**	**Server** ("bush rupees and pot hearts are server-side", NewMain 1.42 **[doc]**). The 2.05 cut and lift code spawns no item **[bin]**	Server, 50 % of 6 items for bushes, a heart for a vase **[src]**

The objects

The tables are `arrays.dat` lists. They are the same in 1.21, 1.3 test and the 2.22 install (2.05 runs from that folder) **[bin]** (`tools/respawn-lab/arrays_dump.py`). In 1.21 they sit one index lower because 1.21 has 24 arrays, not 25.

Cut with the sword:

#	Object	Tiles (2 × 2, top row then bottom row)	Leaves
0	bush	2 3 / 18 19	677 678 / 693 694
1	tall grass	420 421 / 436 437	679 680 / 695 696

Lift with A, in glove order (name table in the 2.05 exe: `bush sign vase stone blackstone`):

#	Object	Tiles	Leaves
0	bush	2 3 / 18 19	677 678 / 693 694
1	sign	512 513 / 528 529	1802 1803 / 1818 1819
2	pot (`vase`)	684 685 / 700 701	1770 1771 / 1786 1787
3	stone	34 35 / 50 51	1834 1835 / 1850 1851

#	Object	Tiles	Leaves
4	black stone	990 991 / 1006 1007	1834 1835 / 1850 1851

Glove power g allows the first g + 2 lift entries (`decomp-findings-2026-10-25-l/03-cut-lift-chest.md`). Players start at glove 1, so the pot lifts with no glove item.

A thrown object does not touch the board when it lands. It breaks into leaves (`TAdventureLeap`, `crush.wav`, or `water.wav` for entry 5). The board change happens once, at the lift. Respawn therefore puts the object back where it was lifted, not where it landed.

"Swamp" is not a destructible object. It is tile type 6 (and 7, lava swamp) in the tile-type list (`newfeatures2001.txt`, `setshape2`). GServer-v2's comment "Bushes, grass, swamp" refers to its own tile list, not to a third cut type.

Push and pull blocks are different again. The 2.05 client sends the 4 × 4 area around the old spot (`ledges-push-pull.md`). GServer13 never respawns that (see below).

Clients

No client-side respawn

2.05 rev 2, offline, live. Level `resp1.nw` (made with `tools/respawn-lab/mklevel.py`) was played with the real client under Wine, and both 64 × 64 boards in its memory were polled every 10 ms for 10.5 minutes (`boardwatch.py`).

- Sword on a bush: tiles 2 3 18 19 became 677 678 693 694 at once.
- A on a pot: 684 685 700 701 became 1770 1771 1786 1787.
- A second bush was cut later.
- No tile changed back in the whole run, 633 s after the pot was lifted.

The client has a `TBoardModify` class, but it is only a record built to send one change (`x, y, w, h`, tile array). It has no timer **[bin]**. The 1.3 client is the same: its `TBoardModify` users are the cut and lift sender, the push and pull code, and one other board-write path, and none of them keeps a timer **[bin]**.

So a cut bush stays cut until the level is loaded again. Whether 2.05 offline reloads the level from disk on re-entry was not tested here.

Sending the change

- **2.05:** cut and lift both call one routine. It writes the leftover tiles to the board, then, only when online, sends `PLI_BOARDMODIFY` (1) with `c x, c y, c w, c h` + w × h GSHORT tiles. It sends nothing if the tiles were already equal. Cut then adds the leaves effect **[bin]**.
- **2.04+ (2.05 checked):** the whole 2 × 2 has to match a table entry before anything happens. 1.3 matched the single tile under the sword tip and wrote the leftover block at the derived corner **[bin]**. This is the 2.04 change "check for correct tiles when lifting/eating/slaying bushes" **[doc]**.
- **2.05:** "bombs don't destroy half bushes" **[doc]**.

Item drops in 1.3 (client-side)

Drops are rolled by the client that cut or lifted the object **[bin]**:

When	Drop
Any cut entry (bush and tall grass), a bush lifted, or a bush destroyed by the bomb blast (cut entry 0) (corrected in `item-drops.md`)	`Random(100)`: 0–24 green rupee (25 %), 25–26 blue rupee (2 %), 27–31 heart (5 %), 32–36 bombs (5 %), 37–99 nothing (63 %)
Pot lifted, or destroyed by that blast routine (lift entry 2)	A heart, every time

- **Timing:** the drop happens at the lift, not when the thrown pot breaks.
- **Item flags:** items are created with flag value 164 and the "announce" flag set.
- **When online:** the client sends the new item straight to the other players over UDP (`sendto` each peer), as id 19, with `chr(x·2+32) chr(y·2+32) chr(item+32)`. Only item kinds marked in a per-item table are sent. The GServer does not take part. The board change itself still goes to the server over TCP (id 1).

Item drops from 1.42 on (server-side)

- **1.42:** "bush rupees and pot hearts are server-side" (`newfeatures142.txt`; the roll is gone from NewMain 24 and 26, and 1.41 still has it) **[doc][bin]**. This note said 2.00 before; see `item-drops.md`.
- **2.05 binary:**
  - The cut routine creates only `TAdventureLeap`.
  - The lift routine creates only the carried object.
  - The only `TAdventureExtra` constructor is reached from the chest code, a weapon-drop path, one network handler (most likely `PLO_ITEMADD` **[inf]**) and the GS1 executor. None of these is the cut or lift code.
  - So **an offline 2.05 bush or pot gives nothing**, and online every drop is a `PLO_ITEMADD` (22) from the server **[bin]**. The capture in `packet-layouts.md` shows the same thing: no client sent `PLI_ITEMADD` and the server sent the drops **[cap]**.
- **Not covered:** a 2.x-era official GServer binary is not in the archive. The real 2.x drop rates are unknown, and only GServer-v2's rates are written down.

GServer13 (November 1999, GSERV014)

Options

`serverconfig.txt` keys and the start dialog (screenshot-checked, defaults filled in):

Key	Dialog label	Default
`respawntime`	Respawn time (bushes etc.) in sec	10
`horselifetime`	Horse lifetime in sec	10
`apincrementtime`	Alignment increment time in sec	1200
`baddyrespawntime`	Baddies respawn time in sec	60

GServer 1.28 beta 2 (`GServerv.exe`) has the same four keys.

`PLI_BOARDMODIFY` (1) handling [bin]

1. Needs at least 4 bytes and a current level, and `len ≥ 4 + 2·w·h`.
2. Writes the new tiles into the server's 64 × 64 board and builds the old-tile string from what was there.
3. If any tile changed:
   - Relays `PLO_BOARDMODIFY` (7) with the client's bytes to every other player in the level. **The sender is skipped.**
   - Makes a change record: new tiles, old tiles, a countdown and a reference count.

    - **If w = 2 and h = 2:** countdown = `Trunc(Random · R/2 + 3R/4)` with R = `respawntime`, and the record joins the level's respawn list.
    - **Any other size:** countdown −1, so it never respawns. This covers 1 × 1, 2 × 1, 3 × 3 and the 4 × 4 push/pull block.
    - The record is also added to the "changed since you left" list of every player who has left this level since loading it. Any earlier record with the same `x, y, w, h` header is replaced.
4. **No tile check.** Any 2 × 2 change respawns, whatever the tiles. A client can blank a 2 × 2 of floor and it comes back.

Respawn tick [bin]

- `Timer1` (TTimer default interval, 1000 ms) walks every loaded level, empty or not, and decrements each countdown.
- At < 1, the server:
    - removes the record;
    - sends `PLO_BOARDMODIFY` (7) with the **old** tiles to everyone in the level, the original sender included;
    - writes the old tiles back into its board;
    - updates every "changed since you left" list. If the list still holds the original change, it is dropped, because that player never saw it. If not, the restore is queued instead.
- **Timing:** the countdown starts at an integer n in 7..12 for R = 10, and the first tick comes 0 to 1 s after the change. The restore therefore lands n − 1 to n seconds later.
- **Weights:** n = 7 and n = 12 are half as likely as 8 to 11, because of the truncation of [7.5, 12.5).

Measured [live]

GServer13.exe ran under Wine in a loopback-only namespace with one load-only account. Two scripted clients (`g2cli.py`) were used: A sends changes and B watches.

Test	Result
2 × 2, 60 positions, one every 0.37 s	All 60 respawned. Delays 6.21 to 11.98 s, mean 9.21 s. ceil(delay) 7/8/9/10/11/12: 4, 9, 16, 11, 12, 8 (expected 6, 12, 12, 12, 12, 6).
1 × 1, 2 × 1, 1 × 2, 3 × 3, 4 × 4	Relayed to B at once, never respawned (watched 20 s past the 2 × 2 respawns)
Echo	A never received its own change. A and B both received every respawn.
Restored tiles	The level's original tiles (632 for `house1.graal` floor), not a fixed "bush" id

Raw rows: `version-profiles/lab-gs13-respawn.tsv`.

Level re-entry [bin]

- **On leave:** the server starts a per-player change list for that level.
- **On re-entry:**
    - If there is no list, the client gets the level as on a first visit.
    - Otherwise only the queued changes go out (as id 0), and the list is dropped. **[bin]**, the branch is hard to read, so treat the exact packet as **[inf]**.
- **Effect:** a player who leaves while bushes are cut and comes back after they regrew is sent nothing for them.
- **Disconnect:** the player's lists are freed.

GServer-v2 (lab server) [src]

This is the open reimplementation used for the 2.05 and 2.22 captures. It is not an original server.

- **Respawn condition:**
  - The top-left tile *before* the change must be one of `0x1ff 0x3ff 0x2ac 0x002 0x200 0x022 0x3de 0x1a4 0x14a 0x674 0x72a`. Those are bush (2), tall grass (420), pot (684), sign (512), stone (34), black stone (990), and five others.
  - Then the change respawns after exactly `respawntime` (default 15).
  - Otherwise it is kept with no timeout.
- **Size:** any size can respawn.
- **Overlap:** a new change deletes stored changes it fully covers.
- **Echo:** relays `PLO_BOARDMODIFY` to the whole level, sender included.
- **Restore:** swaps old and new tiles. It keeps the record, so late joiners get the restored state.
- **Drops:** only for clients at the 2.1 version slot or above.
  - **Bush, grass or swamp tiles** (2, 0x1a4, 0x1ff, 0x3ff) with `bushitems`: `tiledroprate` % (default 50) of the time, item `rand() % 6`.
  - **Pot** (0x2ac) with `vasesdrop`: a heart.
  - **Why 2.05 got drops in the lab:** the lab maps `GNW03081` (2.05) into the 2.1 slot (`tools/lab-setup.md`), which is why the earlier capture shows server drops for 2.05.
- **Push-pull blocks:** a 4 × 4 change made of a red or blue push-pull stone is refused and the board is sent back when `clientsidepushpull` is on.

What a Seihai version profile should do

Profile	Respawn	Drops
1.x online (GServer13-style)	Server, any 2 × 2, `trunc(rand·R/2 + 3R/4)` s on a 1 s tick, R = 10	Client rolls the 1.3 table (above) on any cut and on a lifted or blasted bush, a heart for every pot. Announce to peers; from 1.38 also `PLI_ITEMADD` to the server (`item-drops.md`).
1.x offline	None	Same client table
2.00–2.05 online	Server pushes `PLO_BOARDMODIFY`. Client never regrows.	Only `PLO_ITEMADD` from the server
2.00–2.05 offline	None	None
Seihai's own server	Pick per profile: GServer13 (random 0.75–1.25 × R, 2 × 2 only, no echo) or GServer-v2 (fixed R, tile whitelist, echo)	

Open

- The real 2.x GServer's timer, tile whitelist and drop rates. No binary of a 2000–2002 official server has been found.
- Whether 2.05 offline reloads a level from disk on re-entry, which would make a cut bush come back.
- Drop tables in 1.21: same routine shape expected, not read. 1.33 to 1.41 were read in `item-drops.md`: the same table.
- Later clients (2.2+ `updateboard2` / `PLO_BOARDMODIFY2` on gmaps, v4+). Not covered here.

Lab notes

- **2.05 offline:** `tools/respawn-lab/mklevel.py` writes a level with every object in a row. `boardscan.py` finds the board copies in client memory. `boardwatch.py` logs every tile change with a timestamp. Keys:

`xdotool keydown Up`, `s` for sword, `a` for lift. The Wine screen often stops repainting tiles after a few actions, so trust the memory log.

- **GServer13:**
  1. Extract `graal13b.7z` and start `GServer13.exe` under Wine inside the namespace.
  2. Press OK on the options dialog.
  3. Edit accounts, then Guest Accounts, then Add: name `guest`, password `guest`, load only, level `house1.graal`.
  4. Run `gs13_respawn.py out.jsonl 60` inside the namespace.

`client-internals/body-colours-and-attributes.md`

Body colours, looks commands and GANI attributes (2.05)

How 2.05 colours a player's body, what the looks commands change, and how a GANI's attribute sprites (the hat) pick their picture. Derived from the 2.05 client unless marked otherwise.

Body colours

Fact	Status
A body picture is a palette image. Before it is drawn, palette entry 5 takes the skin colour, 6 the coat, 4 the sleeves, 3 the shoes and 1 the belt. Entry 0 always takes colour 18 (black, the outline)	derived
The five colours are prop 13 (COLORS), in the order skin, coat, sleeves, shoes, belt. This settles the order `protocol/player-props.md` marks as inferred	derived
The colour list, by index: 0 white 255,255,255; 1 yellow 255,255,0; 2 orange 255,173,107; 3 pink 255,132,132; 4 red 255,0,0; 5 darkred 206,24,41; 6 lightgreen 132,255,132; 7 green 0,255,0; 8 darkgreen 0,198,0; 9 lightblue 132,132,255; 10 blue 0,0,255; 11 darkblue 0,0,198; 12 brown 132,0,0; 13 cynober 0,255,255; 14 purple 255,0,255; 15 darkpurple 132,0,132; 16 lightgray 206,206,206; 17 gray 132,132,132; 18 black 0,0,8; 19 transparent 0,0,0	derived
A new player is orange, darkred, black, darkred, black (2, 5, 18, 5, 18), with `body.png` and `head0.gif`	derived
The stock bodies follow that palette layout, and entry 2 is the clear colour	checked on the 2.22 `body.png` and `body13.png`
Graal Reborn's GServer gives a new account 2, 0, 10, 4, 18 instead	read from GServer-v2 beta4 source

Looks commands

Fact	Status
`sethead`, `setbody`, `setskincolor`, `setcoatcolor`, `setsleevecolor`, `setshoecolor` and `setbeltcolor` change the local player, whichever NPC or weapon runs them. The NPC is not changed; an NPC's own look is `setcharprop`	derived
A colour command takes a colour name from the list above or a number, rounded, into its slot	derived
`sethead`, `setbody`, `setskin`, `setcoat`, `setsleeves`, `setshoes` and `setbelt` typed in chat are run by the GServer, which sends the new props back. They are not client commands	read from GServer-v2 beta4 source

GANI attributes

Fact	Status
A GANI sprite whose picture is `ATTR1` … `ATTR5` draws the player's attribute 1 to 5	derived
Attributes 1 to 5 are props 37 to 41 (GATTRIB1 to GATTRIB5). 2.05 reads them for other players as well as its own	derived
A player with no attribute of their own draws the GANI's `DEFAULTATTR1` … `DEFAULTATTR5`. The stock `idle.gani` names `hat0.png` as attribute 1, the hat	checked on the 2.22 GANIs
`setplayerprop #P1,<image>` … `#P5` set attributes 1 to 5; `#3`, `#8` and `#C0` … `#C5` set head, body and colours the same way (`npc-library/gs1-command-reference-205.md`). What a sixth colour slot `#C5` is for is not known	derived

Not checked

- Which folder 2.05 looks in for an attribute picture. The stock hats are in `hats/`, so the hat is inferred to be looked for there.
- What 2.05 draws for an attribute set to an empty name.
- None of this has been checked against a running 2.05 client.

`client-internals/build-catalogue.md`

Client build catalogue (1998–2001)

Date: 2026-10-21

This covers every client in the uploaded archive set: 39 top-level archives plus 16 from the second upload, 55 in total. There are 52 Delphi client executables (53 with GraalX, see below), the Graal3D side project, and the 1998 Java precursor. Everything here comes from static reads of the binaries (PE headers, resources, strings, and immediates in the code) and from the release notes shipped beside each exe. Nothing in this file comes from a decompiler.

How each column was read

Column	Source
MD5 / size	The exe as shipped in the archive.
Compiled	The resource-directory timestamp, a DOS date/time that Delphi sets at link time, in local time. The COFF timestamp is useless for these builds because every Delphi build carries the fixed 1992-06-19 value.
FileVersion	The VERSIONINFO resource. It only exists from 1.37 on and is sometimes stale (2.05 still says 2.0.4.0).
Token	The login version string. Up to 1.37 it is a plain string literal. From 1.38 on it is built one character at a time, so `strings` misses it, and it has to be read from the immediates passed to the char-to-string helper.
Gen	The encryption generation, using the GServer-v2 numbering. 1 means no compression. 2 means zlib with no encryption (a `nodecompressionerror` option exists). 3 means zlib plus the one-byte LCG insertion (see `protocol/` codec notes). Gen 3 is detected by the connection-object initialiser storing seed `0x04A80B38` and then the `Random & 0x7F` key. Two other uses of the same seed (web-level cache file encryption, from 1.3 on) are excluded.

Column	Source
Server / list	1.39 and earlier: the default host:port the connect dialog is pre-filled with (no listserver support). 1.4x and later: the listserver host:port and the list name sent after connecting.
Levels	Which level and map magics the loader compares against. `Z3-V` is `.zelda`, `GR-V` is `.graal`, `GLEVNW01` is `.nw`, and `GRMAP001` is `.gmap`. `Graalsav` is the save-game magic.

Catalogue

Sorted by compile date. "Archive" is the archive file name without `.7z`.

Archive	Release name	Compiled	FileVersion	MD5	Size	Token	Gen	Server / list	Levels and saves	Notable
zeldaonline2	Zelda Online (Java applet)	1997–98	–	–	–	none	text lines	host:14801	`Z3-V1.02`/`1.03`	Java predecessor, see `zelda-online-1998.md`
1stgraal-1	first Delphi beta	1998-09-06 23:54	–	07452f8941fdfcdce878b035156e701c	654336	none	1	the loopback address:14801	Z3-V1.03/1.04 only; Graalsav1	Two-player local game, no `.graal` loader yet, no DirectSound
2ndgraal	beta	1998-09-16 01:33	–	f74b35fc7975a4873087f7fd2aa1d79b	777216	none	1	the loopback address:14801	+ GR-V1.00–1.01; Graalsav1	First `.graal` loader
3rdgraal	beta	1998-09-30 01:47	–	fc78fbc96bdbc6ca22ae56d3ed8635ab	665600	none	1	the loopback address:14801, dialog 14802	GR-V1.00–1.01; Graalsav1	Connect dialog gains a second port field (14802)
4thgraal	beta	1998-10-05 01:33	–	e040af2e19912419ab80aa6dd202b4ea	666112	none	1	as above	GR-V1.00–1.01; Graalsav1	
4thgraalalpha	beta (alpha)	1998-10-12 22:54	–	8cae23a58e1b1b10f3149bdbe53d0431	711680	none	1	the loopback address:14801	GR-V1.00–1.01; Graalsav2	Built after 4thgraal despite the name; DirectSound appears
5thgraal	beta	1998-10-19 02:27	–	0e7c3573f0a916f8b2376060e768fe96	723968	none	1	the loopback address:14801, dialog 14802	GR-V1.00–1.02; Graalsav2	GR-V1.02

Archive	Release name	Compiled	FileVersion	MD5	Size	Token	Gen	Server / list	Levels and saves	Notable
5thgraalalpha	beta (alpha)	1998-10-29 02:21	–	e54397e7d4f59d30b77d5cf0475335a1	739840	none	1	the loopback address:14801	GR-V1.00–1.02; Graalsav2	Built after 5thgraal
6thgraal	beta5	1998-11-09 02:08	–	53c3caf3d930534db3f44f346f5d651a	750080	none	1	the loopback address:14801, dialog 14802	GR-V1.00–1.02; Graalsav3	Published by Cyberjoueur; web levels from <cyberjoueur-host>
7thgraal	beta7	1998-12-04 00:28	–	deb40c31268e499fc4436b8d0687f05d	766976	none	1	as above	GR-V1.00–1.03; Graalsav4	First GR-V1.03 (the last `.graal` revision any build here reads)
8thgraal	beta8	1998-12-08 01:13	–	a49aa1b773e41741f211867fd5ea55d7	775680	none	1	as above	GR-V1.00–1.03; Graalsav4	
9thgraalalpha	beta9 (alpha)	1998-12-16 02:36	–	e226e1b83fd437b0aede968fb395b911	788992	none	1	as above	GR-V1.00–1.03; Graalsav5	Built before 9thgraal
9thgraal	beta9	1998-12-18 01:53	–	f0185ff8525a13f2f4e5997f649bc250	793088	none	1	as above	GR-V1.00–1.03; Graalsav5	
10thgraal	V1.0	1998-12-22 03:24	–	f1f34b4479a8ad7614bd8f7ce56112c6	795648	none	1	as above	GR-V1.00–1.03; Graalsav5	1.0 final: map heads, Alt+1/2 screenshots, Alt+3 minimap
11thgraal	V1.0	1998-12-23 01:00	–	a25521262190 7affc94a4483c6dfc8e3	795648	none	1	as above	GR-V1.00–1.03; Graalsav5	Adds the altern.org RealWorld web level
12thgraal	V1.0	1999-01-05 01:04	–	788ae59e9419806024d14d3e5b611971	794624	none	1	as above	GR-V1.00–1.03; Graalsav5	

Archive	Release name	Compiled	FileVersion	MD5	Size	Token	Gen	Server / list	Levels and saves	Notable
13thgraal	V1.1 beta1	1999-01-12 01:15	–	a211a0ecf3b58daa2f46759444e18bd3	797184	none	1	as above	GR-V1.00–1.03; Graalsav6	`this.` vars, canbepushed/pulled, sleeping/sitting, `.gsave`
14thgraal	V1.1	1999-01-15 02:20	–	d0999d44fcd9c226f350fcf204618885	858112	none	1	as above	GR-V1.00–1.03; Graalsav7	
15thgraal	V1.1	1999-01-25 00:12	–	9a7d66dd5c8c506ad4d3e63ee4a9a1f1	878592	none	1	as above	GR-V1.00–1.03; Graalsav7	Faster script interpreter, IP caching, `puthorse` boats
graal1.1.9	V1.1.9 Gold Edition	1999-02-11 23:50	–	0a6eef88acb4c0fce6e84eee3e76c04a	882176	none	1	as above	GR-V1.00–1.03; Graalsav7/8	Ships `Gserver.exe` (1999-01-22, port 14802, no token); home moves to <graalonline-site>
graal1.1	V1.1 (readme)	1999-02-14 23:03	–	1a9580655cce686c602c01daa251c453	882176	none	1	as above	GR-V1.00–1.03; Graalsav7/8	Same size as 1.1.9 but linked 3 days later
graal121	V1.2.1	1999-05-10 18:29	–	8da2e8284dd8b315a28e2d4fccae79ac	973824	`GSERV007`	2	<list-host-classic>:14804	GR-V1.00–1.03; reads Graalsav9/Graalsv10/Graalsv11, writes Graalsv11	InstallShield 3 package (the exe is inside `_SETUP.1`); "packet compression" (zlib 1.0.4); NPC weapons; GServer and RC on the same token
graal128b2	v1.28 beta2	1999-11-23 01:09	–	8e3c1a7233864da994a0db26f1811212	1196544	`GSERV013`	2	<list-host-classic>:14804	GR-V1.00–1.03; reads Graalsav9/Graalsv10/Graalsv11, writes Graalsv11	"Direct data packages" (positions sent straight to room players); ships GServer (accepts GSERV007–013) and RC 1.28 (GSERV011)
graal13b	v1.3 test	1999-11-29 02:03	–	b62aaa4d606a519e7f4689b3398342 9b	1311744	`GSERV014`	2	<list-host-classic>:14804	GR-V1.00–1.03; reads Graalsav9/Graalsv10/Graalsv11, writes Graalsv11	Alignment system; levels cached on disk (web-level cache seed first appears); ships GServer13 (accepts 007–014)
graal131	V1.31	1999-12-07 02:56	–	bff87a95a34399b2d35b243f281169a7	1318400	`GSERV016`	2	<list-host-classic>:14900	GR-V1.00–1.03; reads Graalsav9/Graalsv10/Graalsv11, writes Graalsv11	Default port moves to 14900; UDP options (`noudp`, fixed port 14899)

Archive	Release name	Compiled	FileVersion	MD5	Size	Token	Gen	Server / list	Levels and saves	Notable
graal132	V1.32	2000-01-12 01:13	–	42b2596af4b75bcf82db03e0fc274f7d	1328128	`GSERV018`	2	<list-host-classic>:14900	GR-V1.00–1.03; reads Graalsav9/ Graalsv10/ Graalsv11, writes Graalsv11	Double-click fullscreen
graal3d	Graal3D	2000-02-05 01:57	–	305c985a020cec6ab8d81ac95d447343	605184	none	own	the loopback address:14901 (UDP 14898)	none	Separate 3D client plus its own GServer.exe (14901); zlib linked, no Graal level loaders
graal133	V1.33	2000-02-21 00:36	–	a3667ce55a5c67cf025882014b9540b0	1350144	`GNW20020`	2	<list-host-classic>:14900	GR-V1.00–1.03; carries Graalsav9/ Graalsv10/ Graalsv11 (string scan)	First GNW token; PNG support; 4-section player list
graal134	V1.34	2000-03-23 00:38	–	9e403616e73a6079ac37d65d475c81cd	1349632	`GNW22030`	2	as above	GR-V1.00–1.03; carries Graalsav9/ Graalsv10/ Graalsv11 (string scan)	Script debugger (F6); MNG appears
graal135	V1.35	2000-04-13 23:17	–	fe9d99c4bf6e6bc9b1b127823f39646f	1389568	`GNW13040`	2	as above	GR-V1.00–1.03; carries Graalsav9/ Graalsv10/ Graalsv11 (string scan)	
graal136	V1.36	2000-05-12 17:34	–	018f51662c7c8862cafb9bfbd14dea41	1399296	`GNW10050`	2	as above	GR-V1.00–1.03; carries Graalsav9/ Graalsv10/ Graalsv11 (string scan)	Server player cap 220 → 28000; nickname limit
graal137	V1.37	2000-05-23 02:40	1.3.7.0	5e966a9d1a079af6d682b2618674edb1	1401344	`GNW21050`	2	as above	GR-V1.00–1.03; carries Graalsav9/ Graalsv10/ Graalsv11 (string scan)	First VERSIONINFO; new sound-effects system
graal138	V1.38	2000-06-23 22:11	1.3.8.0	4401f7d02966af709c2df662cebe34a1	1426432	`GNW22060`	2	as above	GR-V1.00–1.03; carries Graalsav9/ Graalsv10/ Graalsv11 (string scan)	Token now built char by char; `setnick`, `warpto`
graal139	V1.39	2000-08-07 02:49	1.3.8.0	62513ab884f1802a0bcd8114abe349b9	1448960	`GNW07080`	2	as above	GR-V1.00–1.03; carries Graalsav9/ Graalsv10/ Graalsv11 (string scan)	New head-image handling; profiles. FileVersion not bumped

Archive	Release name	Compiled	FileVersion	MD5	Size	Token	Gen	Server / list	Levels and saves	Notable
graal_gr1411	V1.41 rev1 (<graal in-site>-patched copy)	– (resource stamp missing)	1.4.1.1	a97740e711ffe9b698ec03712d4c06af	1506816	`GNW13110`	**3**	list `<list-host-reborn>`:14923	GR-V1.00–1.03, GLEVNW01; carries Graalsav9/ Graalsv10/ Graalsv11 (string scan)	First server-list window (from 1.4) and first gen 3 build. See the note below on the patching
graalnm24	NewMain client (1.42 notes)	2001-01-10 19:55	1.4.1.1	229f05de7a43b98e223d8e1b58bffc63	1783296	`GNW08011`	3	list <list-host-classic>:14921, "newmain" (plain)	GR-V1.00–1.03, GLEVNW01	First `.gani` animation system; ships GraalShop and RC-NewMain (GSERV021)
graalnm26	NewMain client (1.42 notes)	2001-01-12 21:19	1.4.1.1	22a08ed154f67948e01b969b68cdb05e	1784320	`GNW08011`	3	as above	GR-V1.00–1.03, GLEVNW01	Adds GraalLevelGen 1.0
Graal-v2.0	v2.00	2001-02-01 14:54	2.0.0.0	c93e659e2cda36379aaea4706db20b38	1812480	`GNW24011`	3	list <list-host-classic>:14921, "newmain" (host DES-hidden)	GR-V1.00–1.03 (no `.nw`)	No 2/3/4-player mode; tutorial levels; server-side bush rupees. First DES-hidden list host
Graal-v2.0-rev1	v2.00 rev1	2001-02-02 19:19	2.0.0.1	30bd230e578d3a347866a87060efec20	1812992	`GNW24011`	3	as above	as above	
Graal-v2.0-rev2	v2.00 rev2	2001-02-05 02:32	2.0.0.2	fc4bb7e3b7ca9cd8ea6693a033635534	1815040	`GNW24011`	3	as above	as above	
Graal-v2.0-rev3	v2.00 rev3	2001-02-07 01:05	2.0.0.3	1ce5c1662b65246ae1c6559af5ff71f0	1815040	`GNW24011`	3	as above	as above	
GraalX (`.rar`, separate upload)	2.01-era side build	2001-03-07 21:40	2.0.0.3	d3ee52ef9d22a0d99b154b47c5f56f60	1821696	`GNW23021`	3	as above	GR-V1.00–1.03, **GLEVNW01**	2.01 plus the `.nw` reader and writer. See the note below
Graal-v2.01	v2.01	2001-03-08 14:59	2.0.1.0	08340a8868f1d11e0285cc70e51200fe	1815552	`GNW23021`	3	as above	as above	Level preloading on the map; showimg fixes

Archive	Release name	Compiled	FileVersion	MD5	Size	Token	Gen	Server / list	Levels and saves	Notable
Graal-v2.01-rev1	v2.01 rev1	2001-03-15 21:10	2.0.1.0	010a32afce66729b83825880 9a97c0d9	1815552	`GNW23021`	3	as above	as above	
Graal-v2.02	v2.02	2001-04-15 20:48	2.0.2.0	60018a19ef671c5067438267 28c49718	1808384	`GNW15041`	3	as above	as above	Hats; HTML window navigation
Graal-v2.02-rev1	v2.02 rev1	2001-04-15 23:16	2.0.2.0	e06ff79aded885a223b933617 d456577	1808896	`GNW15041`	3	as above	as above	Speed-cheat protection; client language sent to server
Graal-v2.02-rev2	v2.02 rev2	2001-04-20 18:48	2.0.2.2	993a1829dbb6acaa2b46063f 5efe636c	1807872	`GNW20041`	3	as above	as above	
Graal-v2.03	v2.03	2001-05-07 15:31	2.0.3.0	36972c74baad09ceb33dad94 b0eb1e33	1836032	`GNW06051`	3	as above	as above	`triggeraction`, `move`; NPC-server support
Graal-v2.04	v2.04	2001-06-25 07:42	2.0.4.0	a40608308d4f890cb5e4efdd 45b38ba4	1872384	`GNW24061`	3	as above	as above	`replacegani`, `addtiledef`/`addtiledef2`
Graal-v2.05	v2.05	2001-08-03 19:12	2.0.4.0 (stale)	9abce477945a1307bb2be9ed 47d2f170	1891328	`GNW03081`	3	as above	GR-V1.00–1.03, **GLEVNW01, GRMAP001**	`.nw` and `.gmap` return; script-editor button; `setshape2` arrays
Graal-v2.05-rev1	v2.05 rev1	2001-08-04 03:26	2.0.5.0	2780f2f370396c580e1648696698c871	1895936	`GNW03081`	3	as above	as above	
graalnwb1	NewWorld beta 1	2001-08-06 04:13	2.0.5.0	88d1f84c96da1dcc3942b139 fbd741c9	1820672	`G2N15041`	3	list <list-host-classic>:14921, "newworld" (DES)	as 2.05	Terrain heights, backtile areas, NW time variables (string arrays are nwb2 only; see `newworld-betas.md`); ships NPCControl, LevelFiller, TerrainGenerator, Greaditor and other tools

Archive	Release name	Compiled	FileVersion	MD5	Size	Token	Gen	Server / list	Levels and saves	Notable
Graal-v2.05-rev2	v2.05 rev2	2001-08-16 17:24	2.0.5.2	f3fdf54d943fa91c89acba5bb695db99	1891840	`GNW03081`	3	list <list-host-classic>:14921, "newmain" (DES)	as 2.05	The decompile target; same bytes as `Graal_v2.05_rev2.exe`
graal21test2	v2.1 test 2	2001-09-21 17:54	2.1.0.0	d15d877822ad4145578107857c90fb3c	1748480	`GNW03081`	3	list <list-host-classic>:**14922**, "newmain" (DES)	as 2.05	New tiles, new starting window; the token is still 2.05's
graal21test3	v2.1 test 3	2001-09-27 21:11	2.1.0.0	2116a7ba03648dcc00403553 4ec94de8	1753600	`GNW03081`	3	as above	as 2.05	
graalnwb2	NewWorld beta 2	2001-10-03 20:46	2.1.0.0	af2d21066659bed6af6c77e52b2cc0e1	1613824	`G2N15041`	3	list <list-host-classic>:14922, "newworld" (DES)	as 2.05	String arrays (`addstring` etc.), terrain heights with per-level `HEIGHTS` grids from the gmap; see `newworld-betas.md`

GraalX

GraalX came as a separate upload, `GraalX.rar`, holding a single `GraalX.exe`. Its VERSIONINFO says 2.0.0.3, the same stale value as 2.0 rev3. It was linked one day before 2.01 and carries 2.01's login token. Its classes and instance sizes, its GS1 command table (144 commands) and every packet and prop id it handles are identical to 2.01. Matched function by function, it shares 6350 identical functions with 2.01 and differs in only 7. The one real difference is `.nw` support. GraalX keeps the GLEVNW01 reader, writer and chest-line parser that NewMain 1.42 had and that 2.0 and 2.01 dropped. Several of those functions are identical to NewMain's. It also adds the GLEVNW01 magic checks and the `.nw` open-dialog filter that reappear in 2.05. So it looks like a 2.01 build compiled with the `.nw` code left in, for example a level-format test. Nothing in the binary says why it was built or who it went to.

Patterns worth knowing

The GNW token is a build date. It is `GNW` followed by DDMMY, with Y being the last digit of the year. Every build from 1.33 to 2.05 fits, and the token date is on or a few days before the link date: GNW20020 is 20 Feb 2000 (1.33, linked 21 Feb), GNW07080 is 7 Aug 2000 (1.39, linked 7 Aug), and GNW03081 is 3 Aug 2001 (2.05, linked 3 Aug). A rev that does not touch the protocol keeps the previous token (2.0 rev1–3, 2.01 rev1, 2.02 rev1, 2.05 rev1–2), and so does the 2.1 test line. So an unmapped token like GNW30123 can be read as 30 Dec 200x, which probably points to a 2.1-era build, since 2.1.0.x is known to use GNW31101.

Before GNW, the token was a protocol revision. `GSERVnnn` went 007 (1.21), 011 (RC 1.28), 013 (1.28b2), 014 (1.3 test), 016 (1.31), 018 (1.32), and 021 (RC-NewMain, Jan 2001). The bundled GServers accept a range rather than one token: GServer 1.28 accepts GSERV007–013 and GServer13 accepts 007–014.

Encryption generations by build.

Gen	Builds	How it was determined
1	1st Graal through 1.1.9	No zlib linked; no decompression-error option
2	1.21 through 1.39	zlib 1.0.4 linked; the 1.2 notes say "packet compression"; `nodecompressionerror` from 1.28b2
3	1.41 rev1, NewMain 1.42, all 2.0x, 2.1 tests, NW betas	Connection initialiser seeds the insertion LCG with `0x04A80B38` and a 7-bit random key. 1.39 lacks it

Server discovery. 1.39 and earlier have no listserver: the connect dialog is pre-filled with a host:port and the player edits it. The defaults were the loopback address on 14801 for the 1998 betas through 1.1.9, <list-host-classic>:14804 for 1.21–1.3 test, and <list-host-classic>:14900 for 1.31–1.39. The server-list window arrives with 1.4. NewMain 1.42 writes host, port and list name as plain char-by-char strings. From 2.0 the host is DES-hidden with the bit-reversed key described in the 2.05 notes. The key and ciphertext are the same in every 2.0x build and decrypt to `<list-host-classic>`. The list name chooses the main list ("newmain") or the NewWorld list ("newworld"). The port moves from 14921 to 14922 with the 2.1 test builds.

The 1.41 rev1 copy is patched. It contains both "<graalonline-site> Server List" and "<graalin-site> Server List", `<www-graalin-site>` register and homepage links, and `<list-host-reborn>:14923`. Its resource-directory timestamp is zeroed, which is what a resource editor leaves behind. This is the only 1.41 copy available, so the listserver host and port it shows may not be the original 1.41 values. The token (GNW13110 = 13 Nov 2000) and the code-level facts look untouched.

`.nw` support is not monotonic. NewMain 1.42 (Jan 2001) and the 1.41 rev1 copy contain the GLEVNW01 loader, but 2.00–2.04 contain no GLEVNW01 or GRMAP001 string, even as char-built strings. Both reappear in 2.05 (Aug 2001) together with gmap support. This suggests 2.0 branched from a pre-NewMain tree and the NewWorld code was merged back for 2.05. That conclusion is an inference from strings only. A live check (loading a `.nw` level in 2.04) would confirm it.

Save magics after 1.1.9. 1.21 to 1.32 write `Graalsv11` (with `sav` shortened to `sv` so the magic stays 9 characters) and read `Graalsav9` and `Graalsv10` too. A search for `Graalsav` misses the last two. 1.33 to 1.39 and the 1.41 rev1 copy carry the same three magic strings in the same arrangement as 1.21 to 1.32: `Graalsv11` is referenced from two places (writer and reader) and the other two from one (reader only). So they write `Graalsv11` too. The writer itself was traced only in 1.21 and 1.32. See `formats/graalsav.md`.

Compiler. Every build is Borland Delphi with linker 2.25, DelphiX for DirectDraw, and an RTL copyright reading "1983,97 Borland". Two compilers are in use, and the class metadata tells them apart. In the older layout the VMT self pointer sits 64 bytes before the VMT and the class name 32 bytes before it, which is the Delphi 2 layout. In the newer layout those offsets are 76 and 44, which is the layout from Delphi 3 on. Only 2nd Graal and 1.28 beta 2 onwards use the newer layout. They also link `TMultiReadExclusiveWriteSynchronizer`, which arrived in SysUtils with Delphi 4, so they are Delphi 4 builds. 1st Graal, 3rd Graal to 1.1.9, and 1.21 use the older layout and lack that class. Their exact compiler is unresolved: the layout says Delphi 2, but the RTL string "1983,97" is a year late for Delphi 2. A Delphi-2-level compiler shipped in 1997, such as C++Builder 1's, would fit both, but that hasn't been checked. Either way the switch to Delphi 4 happened between 1.21 (May 1999) and 1.28 beta 2 (Nov 1999), and 2nd Graal is an outlier built with the newer compiler. (An earlier version of this note said every build from 2nd Graal on has the class. A per-build check shows otherwise.)

Archive naming vs build order. The "Nth graal" archive names are not strictly chronological. 4thgraalalpha and 5thgraalalpha were linked after their non-alpha namesakes, and 9thgraalalpha before 9thgraal. `graal1.1` and `graal1.1.9` have identical sizes, but graal1.1 was linked three days after the Gold Edition.

Other executables in the archives

Archive	File	Compiled	MD5	Token	Port
graal1.1.9	Gserver.exe	1999-01-22	7882dba6ebb2ab1026de260c3043fa47	none	14802
graal121	GServer.exe	1999-05-09	bfcdaae0217d953124a9b21ff7858967	GSERV007	14804
graal121	remotecontrol.exe	1999-05-10	5932d6baa08289a65515342dfc4b9f5a	GSERV007	14804
graal128b2	GServerv.exe	1999-11-22	8d6bff85bda6e6c9558503afa6b7259d	accepts GSERV007–013	14804
graal128b2	RemoteControl1.28.exe	1999-10-20	d7150d1385ea11b812d10d30212f9757	GSERV011	14804
graal13b	GServer13.exe	1999-11-29	d1d0470e53a60b8f000b9ea5da9d37b6	accepts GSERV007–014	14804
graal13b	RemoteControl13.exe	1999-11-29	1f2c19ab385c8b8af6236197f98ad735	GSERV011	14804
graal3d	GServer.exe	2000-02-05	aab1a34a1f196d4f4fddf12112ccbf52	none	14901
graalnm24/26	RemoteControl-NewMain.exe	2001-01-08	bc2b8a2e80acc09a6bb418970cff5a64	GSERV021	14804
graalnwb1	RemoteControl-NW.exe	2001-05-22	6fe24bdd4e135900fc40d7fb70bdcdd2	–	14804
graalnwb2	RemoteControl-NW.exe	2001-08-27	ce454437af8880f61ac2173b0bd11047	–	14804

Also present: gif2mng (2000-06-12, identical in every archive that has it), GraalShop (gani editor, 2000-12 and 2001-04), GraalLevelGen and GraalLevelGenNW, NPCControl (2001-07-13), LevelFiller with levelviewer, TerrainGenerator, PNGView, YoshiChess, and Greaditor (UPX-packed).

See also `drive-clients-inventory.md` (the Drive clients set: which of these builds exist as exes, file versions, and Wine start-up notes) and `../version-profiles/drive-clients-exes.tsv`.

`client-internals/build-diff-204-205.md`

2.04 → 2.05 → 2.05 rev2: function-level diff

Date: 2026-10-21

This compares the three builds closest to the decompile target. All three were run through the same Ghidra pipeline: import with the Borland Delphi compiler spec, apply RTTI-recovered class and method names, recover the switch tables, then export per-function features. The functions were then matched across builds. The function counts below come from this pipeline (Ghidra 11.4.2 with the tools in `tools/build-diff/`). Other Ghidra passes over the same binaries can report different totals, depending on how many switch tables and RTTI methods they recover. This note is cleaned: it has no addresses and no decompiler output. Builds are identified by MD5 in `build-catalogue.md`.

> **Live debugging:** 2.05 and every later build start an anti-debug component that looks for debuggers, loaders and monitors and stops the client when it finds one. 2.04 and earlier do not have it. See Anti-debug component.

How the matching works

Each function is reduced to an address-free instruction stream (mnemonics and operands, with in-image addresses replaced by a placeholder). The matcher then runs these passes in order:

1. Unique RTTI names (`Class.Method`, `Class.virt_N`, `Class.dyn_N`). Virtual-slot names are skipped for classes whose VMT shifted (see below).
2. Unique exact streams of 6 or more instructions.
3. Unique sets of referenced string literals.
4. Unique sets of large immediates.
5. Propagation through ordered callee lists and single unmatched callers, repeated until stable.

A matched pair is **identical** if the streams are equal and **changed** otherwise. Unmatched functions are counted as only-old or only-new. Small look-alike helpers (many 15-instruction VCL thunks, for example) can fail to pair, so the only-old and only-new counts are an upper bound on real churn.

Pair	Functions (old / new)	Identical	Changed	Only old	Only new
2.05 rev1 → rev2	6522 / 6529	6345	94	83	90
2.04 → 2.05 rev2	6471 / 6529	5799	558	114	172

2.05 rev1 → rev2

The game code barely moves. About 20 game functions differ. Class layouts are identical (same instance sizes, same VMT lengths) and the GS1 command table is the same. What changed matches the rev2 changelog:

- **Player counting** (changelog: "the script variable playerscount is fixed"). Four helpers count players in the current level and its gmap neighbourhood. In rev1 they read the level's player list through the caller's object for one test and through the global level for the other. rev2 reads both through the same object.
- **Tutorial / map start-up.** rev1 created a map object while setting up the tutorial map files. rev2 leaves that slot empty at start-up. When a gmap is active, rev2 runs an extra step after the map object is created.
- **Level reset.** One level-reset path no longer sets a flag that depended on game mode 5. A per-level loop takes its bound from a field on the main form instead of a hard-coded 632. These may belong to the changelog's "switching to other servers" fix, but that link is not proven.
- **NPC script text.** Of the three helpers that translate the `§` line marker into CR/LF, one was dropped and the other two were reworked.
- **Small changes** in the server-message window, the server-list form constructor, the main form constructor and the mouse-move handler (a few instructions each).

The library code was rebuilt. The VCL and RTL units in rev2 (Forms, ComCtrls, Controls, docking, ImgList and others) differ from rev1 in instruction shape. rev1's library code, by contrast, matches 2.04's. So rev2 was linked against a rebuilt or updated VCL, not just recompiled game units. For carrying names, this means RTTI-named VCL methods still match by name, but unnamed library helpers between rev1 and rev2 need the shape or callee passes.

2.04 → 2.05

This is where the NewWorld code lands in the mainline.

New classes: `TGraalMap` (gmap support) and `TADP` (see the anti-debug section below). **Removed classes:** `TEditorBaddy` and `TEditorLevel`.

The level-object VMT shifted by one slot. `TDrawableObject` gained a virtual method (2 → 3 user slots). Every descendant's virtual numbering therefore moves by one from 2.04 to 2.05: `TDrawableSurface`, `TSurfaceOwner`, `TLevelObject`, `TServerPlayer → TPlayer` / `TEditorNPC`, `TComputer`, `TDisplayedNPCImg`, `TEditorChest`, and the ten level-object classes (horse, bomb, ball, carry, extra, flying, leap, explosion…). So 2.04 `X.virt_N` corresponds to 2.05 `X.virt_(N+1)`. Any name or type carried between 2.0–2.04 and 2.05 by slot number has to apply this offset.

Instance-size changes: `TAdventureLevel` grows from 96 to 184 bytes (gmap and `.nw` state), `TAdventureCommand` from 68 to 76, and `TServerPlayer` / `TPlayer` / `TEditorNPC` each shrink by 4.

`.nw` support (GLEVNW01) is new code. The text-level reader parses `BOARD`, `LINK`, `BADDY` … `BADDYEND` and `CHEST` sections and decodes tiles with the standard base64 alphabet. It sits next to the binary `GR-V1.00–1.03` reader, which stays. A matching writer emits `GLEVNW01`, then one `BOARD 0 <y> 64 0 <tiles>` line per row (the `BOARD x y width layer tiles` form in `formats/glevnw01.txt`). A new chest-line parser carries the error "Wrong chest data!". Three other existing functions gained a GLEVNW01 magic check: one in the level class, one in the player/script class, and one in the main form unit. `.nw` is added to the level open dialog.

gmap support is new code. It covers the `TGraalMap` class, a loader that builds the map image (`map.png`) and is reachable from the GS1 executor, and a `bigmap.txt` / `minimap.txt` setup routine reachable from the reconnect window. A main-form routine that set up `bigmap.txt`, `minimap.txt` and `map.png` in 2.04 no longer references them.

GS1 commands new in 2.05: `loadmap`, `setz`, `updateterrain`, `setbacktile2`, and a `setbacktile` form that takes an argument. The 2.05 table holds 161 commands (163 records counting the `up` / `right` direction-table entries that follow it), identical in rev1 and rev2. Only `loadmap` does anything: `setz`, `updateterrain` and `setbacktile2` have no executor case in 2.05, so they parse and are ignored (see the parsed-but-ignored table from #4).

Smaller additions: - PMs are written to `pms\messages_<name>.txt`. - One drawing routine gains a `drawaslight` check. The `drawaslight` script command itself predates 2.0 (it is already in the NewMain 1.42 command table). - The script editor gains a 400-instruction routine that handles `/* */` comments and tabs. It is reached from a new button handler, matching the changelog's "new button in the script editor". - A new level-editor routine, reached from a toolbar button, handles `playerx` / `playery` with values clamped to 0–61. - 2.04's routine that installed `tempsitc.ttf` into the Windows fonts folder was not matched to anything called in 2.05. The string and the code body are still in rev2, so it looks uncalled rather than removed. The `Courier New` default in the main form constructor is gone.

Anti-debug component (ADP), new in 2.05

2.05 links a third-party Delphi component `TADP`. The main form constructor starts it, and the form's DFM sets all four switches: `Kill_Debugers = True`, `Stop_Execution = True`, `Anti_Procdump = True` and `Kill_Monitors = True`. It uses the Toolhelp32 process, module and heap walkers. It looks for loader and monitor tools by name (the binary contains `FPLOADER.EXE` and `FROGSI`, likely a fragment of a FrogsICE-style name) and stops execution when it finds one. The component appears in 2.05, 2.05 rev1, 2.05 rev2, the 2.1 test builds and both NewWorld betas. It is absent from every build up to 2.04. This matters for live work on 2.05 and later under a debugger or monitor.

What to reuse

- **2.05 rev1 is close enough to share names with rev2** wherever the RTTI or shape match holds. Only the ~20 functions above, plus library helpers, need re-checking.
- **About 90% of 2.04's functions are identical in 2.05.** It is the cleanest reference for what the NewWorld merge added: TGraalMap, GLEVNW01, the VMT slot, the new GS1 commands and ADP.

`client-internals/carry-hide-and-glove.md`

Hiding under a carried object, and the glove (2.05)

Date: 2026-10-29. Read from the 2.05 rev 2 client; tile lists from the 2.22 install's `arrays.dat`. **[2.05]** unless tagged. No addresses or decompiler text. Key slots are graalconfig's: D is the item key, S attack, A action.

Hiding (mode 14)

- **D while carrying** a bush, a light stone, a black stone or an NPC hides the player under it. The carry sprites that allow it are 261 (bush), 263 (light stone) and 239 (black stone). A pot (265) or a sign (267): D does nothing.
- **D again**, or **A**, comes out: the player goes back to carrying (`carrystill`). Out from under a bush (not an NPC) the client starts break effect 0 (the leaves) at `(x + 0.5, y + 1)`.
- **S** throws, from hiding too.
- **Poses.** `hiddenstill` standing, `hidden` walking. The walk routine applies a factor of 0.2 while hidden, the speed: 0.1 tile per tick, seen live. **[live]**
- **Hits.** The hit gate refuses hits on a hidden player unless a player byte that was not traced is set.
- **Wire.** `hiddenstill` is animation step 24, sprite 18 (as `carrystill`); `hidden` is steps 25 to 28, sprites 19, 20, 19, 18. STATUS gets bit 0x02. The carry sprite prop does not change.
- Seen live (2026-10-30, `player-mannerisms-live.md`): stopping while hidden shows `hiddenstill`, and walking hidden is 0.1 tile per tick (the 0.2 factor is the speed).

Glove power and the lift table

- The lift routine reads the tile at the A key's facing point and tries the first **glove + 2** entries of `arrays.dat` array 11 (lift tiles; array 12 holds what each leaves behind). All four tiles must match.
- Array 11 in the 2.22 file: bush 2/3/18/19, sign 512/513/528/529, **pot** 684/685/700/701, light stone 34/35/50/51, black stone 990/991/1006/1007. The carry sprites in the same order are 261, 267, 265, 263, 239.
- Every player starts at **glove 1** (bush, sign, pot). The `glove1` item raises the power to at least 2 (adds the light stone), `glove2` to at least 3 (the black stone). GServer agrees: its default account has `GLOVEP 1` and its glove items set 2 and 3. **[srv]**
- Online the power is player prop 6 (GLOVEPOWER).
- An NPC is lifted only when its script set `canbecarried` (command 41, cleared by 42), at the same point and with no glove check. It leaves the level's NPC list and is carried. No packet goes out for it.

`client-internals/chat-text.md`

Chat text and nicknames over players

How the classic Windows clients draw the chat line (`PLPROP_CURCHAT`, prop 12) over a player, and how long it stays. Read from the client binaries (1st Graal to 2.31) and checked on the real 2.05 rev 2 client under Wine on 2026-10-30.

There is no bubble. Every build draws the text straight onto the game view with the Windows canvas (`TCanvas.TextOut`), in a bold TrueType font, with no box behind it.



real 2.05 rev 2 (top) and Seihai (bottom), "hello world Wg"

Font per build

The font is a global pair, name and `Font.Height`, set before each draw. From 1.31 on, `font=` and `fontsize=` in the client config replace the defaults (see `graalconfig-reference.md`). The style is always bold (`fsBold`).

Builds	Face asked for	Height	Shadow colour	Source
1st Graal, 4th, 9th, 15th, 1.1.9, 1.21	`Dialog`	28	always blue	literals at each draw call
1.31, 1.38	`Dialog`	28	contrast rule (below)	font globals, `font=`/ `fontsize=`
1.39, 1.41	`Helvetica`	24	contrast rule	font globals
NewMain 1.42, 2.00 to 2.05 rev 2, 2.17, 2.22, 2.31	`Tempus Sans ITC`	24	contrast rule	font globals; `tempsitc.ttf` is installed from the client folder with `AddFontResourceA`

Notes: - **Height.** The value is written to `Font.Height`. It is positive, so it is the cell height in pixels (ascent plus descent), not the point size. For Tempus Sans ITC (2048 units per em, Windows ascent 1974, descent 698) a cell of 24 px is an em of about 18.4 px. The live capture matches this size: "hello world Wg" is 130 px wide and 20 px

from the top of the `h` to the bottom of the `g` in both the real client and a rasteriser set to cell height 24. - **No such font.** Windows has no font named `Dialog`. The GDI font mapper then picks a sans serif by charset and pitch, which is Arial on a normal install. `Helvetica` is mapped to Arial by the standard `FontSubstitutes` entry. Neither of these was checked on a real Windows machine. - **Bold.** Tempus Sans ITC ships only a regular face, so Windows makes the bold itself: each glyph is smeared one pixel to the right and each advance grows by one pixel. - **6.x.** The v6 thread reports the same default (Tempus Sans ITC, size 24) and the same colour, shadow, anchor and wrap rules in the Linux 6.03 renderer. See `protocol/v6x-clients.md` when it lands.

Colour and shadow

- **Text colour.** White (`0xFFFFFF`) for chat.
- **Two passes.** The shadow is drawn first at (+1, +1), then the text at (0, 0).
- **1.21 and earlier.** The shadow is always pure blue, RGB (0, 0, 255).
- **1.31 and later.** The shadow colour is worked out from the text colour. If the red or the green of the text is above `0xE0`, the shadow's red and green are 0. Otherwise they are `0xFF`. The shadow's blue is always the text's own blue. For white text that gives (0, 0, 255), the same blue as before. The same routine draws other text over sprites, such as the nickname under a player and number pop-ups, so darker text gets a light yellow shadow.

Position

The routine takes a point in tiles and centres the text on it horizontally.

- **x.** The player's x + 1.5 tiles, the middle of the 48 px player box.
- **y.** The top of the text cell is the player's y − 1 tile, so the text sits one tile above the top of the player box. When the player carries something, it is y − 2 tiles. The carry part comes from a field of the player object that is non-zero while something is held. It was read from the code, not checked live.
- **Nickname.** The name is drawn by the same routine at y + 2.5 tiles (see below). In the capture the chat glyphs were 56 px (3.5 tiles) above the nickname glyphs, which matches.

The view is clipped to the level view rectangle (`TextRect`).

Nickname (1.31 to 2.31)

The nickname is drawn by the same routine as chat, under the player. Wrapping and the shadow rule are the same as for chat.

- **Position.** Centred on x + 1.5 tiles. The top of the text cell is at y + 2.5 tiles, 40 px below the top of the player box.
- **Colour.** The colour depends on the player's AP (alignment, prop 32). Each step below is `floor(n × k / d)`, computed with a 0.0001 bias before truncating.

AP	Colour (R, G, B)
0	(64, 0, 0)
1 to 26	(128 + (ap−1)·127/25, 64 − (ap−1)·64/25, same as G): dark red to red
27 to 49	(255, (ap−26)·255/24, same as G): red to pale pink
50 to 55	white
56 to 60	(255 − (ap−55)·127/5, 255, same as R): white to light green
61 to 70	(128 − (ap−61)·128/10, 255, 128 + (ap−61)·127/10): light green to cyan
71 to 99	(0, 255 − (ap−70)·255/29, 255): cyan to blue

AP	Colour (R, G, B)
100	(224, 196, 0), gold

The same ramp's byte sequence is present in 1.31, 1.38, 1.39, 1.41, NewMain 1.42, 2.00, 2.05, 2.17, 2.22 and 2.31. 1.21 has no AP colours. The v6 thread reports an AP ramp for 6.x too.

Checked live on 2.05 rev 2 by sending prop 32 for another player. The drawn colours for AP 0, 20, 40, 58, 65, 85 and 100 were exactly (64,0,0), (224,16,16), (255,148,148), (179,255,179), (77,255,178), (0,124,255) and (224,196,0). AP 50, which is also the value before any AP is sent, was white.

- **When it shows.**
  - *Other players.* Always.
  - *Your own.* Only for 60 frames (3 s) at a time. A per-player counter is set to 60 when:
    - the player is set up, which is why it shows at login;
    - the player's AP changes, from a packet or a script;
    - you type exactly `showname`. That line is not sent to the server.

    The counter counts down in the own player's tick, and the name is drawn while it is 0 or more. Other players' counters never count down.

  Live: after login the own name went away. A typed `showname` brought it back for 3.0 s: it was up from the first frame, still up at 3.11 s and gone at 3.32 s. Nothing was sent.

- **Length.** It is cut to `nicknamelimit` characters only when `limitnicknames` is on. That option is off by default, and the limit is 50.

Wrapping (1.31 and later)

When the text is wider than the view (640 px in 2.05), it is split at spaces:

- Words are added to the line while it still fits.
- A word that does not fit starts the next line.
- A word that does not fit on a line of its own becomes a line by itself, unbroken.

The line height is `TextHeight(' ')`, which is the cell height. Lines stack upward: the first line is drawn (lines − 1) × line height above the anchor, so the last line stays on the anchor.

Each line is centred on its own. If it would run past the right edge it is moved left until it fits, and then moved right if it would start left of 0.

1.21 and earlier draw a single centred line, clipped to the view, with no wrapping.

The 6.x renderer wraps only text of 11 or more characters. It breaks after any of `.,;+-?!` (v6 thread). The classic builds break at spaces only.

Special texts (2.05)

- **`showap`.** A chat of exactly `showap` shows the player's AP number instead of the text.
- **Other displays.** The same slot above the head is used by other things. While a per-player counter runs, it shows a pair of numbers joined by `/`. In some sprite states it shows an item image or a number from what the player carries. In those cases the chat is not drawn. These branches were read but not named.

How long it stays (live, 2.05 rev 2, 2026-10-30)

One frame is 50 ms. The live timings below confirm the 60-frame = 3 s reading.

- **Your own line.** It stays up as long as you stand still: it was still up after more than 7 minutes. The first direction key starts the countdown, and the line went 3.0 s later (between 2.92 s and 3.11 s in screenshots taken every 0.2 s). At that moment the client sent an empty CURCHAT.
- **Another player's line.** It shows for at least 3 s from when it arrives, and until that player's empty CURCHAT arrives. Three runs against the lab stub server:
  - Empty line sent 20 s or more after the text: the text was gone in the next frame (under 0.15 s).
  - Text, then empty after 2.0 s: gone about 1 s after the empty, 3.0 s after the text.
  - Text, then empty after 0.54 s: gone between 3.06 s and 3.21 s after the text.
  - With no empty line, it stays: it was still up after 10 s.

This corrects `seihai/documentation/chat-log.md`. That page said an empty CURCHAT starts a fresh 60 frames. In fact it only waits out whatever is left of the 60 frames that began when the text arrived.

Length

When the line is sent, a chat longer than 256 characters is cut to its first 253 characters plus `...`. The receiving client draws whatever arrives.

Method

- **Static.** Every client's font globals were read: the `Tempus Sans ITC`, `Helvetica` or `Dialog` string followed by the height dword. The 2.05 rev 2 text routine and `TPlayer` draw were disassembled. The shadow rule's byte sequence is identical in 1.31, 1.38, 1.39, 1.41, NewMain 1.42, 2.00 to 2.05, 2.17, 2.22 and 2.31.
- **Live.** The 2.05 rev 2 client ran under Wine in the lab (`graal-decompile/live-trace/SETUP.md`), with chat injected from the stub server as `PLO_OTHERPLPROPS` prop 12. The screenshot comparison above puts Seihai's renderer next to it, using the `tempsitc.ttf` shipped with the 2.22 install. Wine anti-aliases the text. A Windows 98 machine without font smoothing would draw the same glyphs with hard edges.

`client-internals/client-architecture-205.md`

Client 2.05 rev 2 architecture

Date: 2026-10-21

Build: `GNW03081`, the "2002ver9" leak build in GServer's `versions.txt`. Delphi (Borland), DelphiX (DirectDraw/DirectSound/DirectInput), zlib 1.1.2. Derived from the class list and behavior of the 2.05 binary. Names only, no code. Tag: **[2.05]** unless marked.

Networking classes

Class	Parent	Role
`TGraalClient`	TObject	socket, out buffer, zlib flush, u16be frame
`TServerListClient`	TGraalClient	listserver link: base send (no cipher), receive handles ids 0 and 1

Class	Parent	Role
`TAdventureClient`	TGraalClient	game link: gen-3 insertion on send, a receive jump table that handles 59 ids, login builder
`TWebClient`	TObject	HTTP (web levels / `openurl`) [inf]
`TAdventureServer`	TObject	local or offline server stand-in [inf]

The game socket keeps the gen-3 iterator, the key, and a "first message sent" flag in the client object. Disconnecting resets the server name to `Offline`. Messages are buffered and flushed at 0x1000 bytes. The client drops the link if 256 KB is queued. A UDP socket handles peer-to-peer traffic (`protocol/udp-p2p.md`).

Game model

Class	Parent	Notes
`TAdventure`	TDXForm	main window and game loop
`TAdventureLevel`, `TBoard`, `TBoardModify`	TObject	level, 64×64 board, pending board changes
`TLevelObject`	TDrawableObject	base for drawable level objects
`TAdventureExtra`, `TAdventureLeap`, `TAdventureBomb`, `TAdventureExplosion`, `TAdventureBall`, `TAdventureFlying`, `TAdventureCarry`	TLevelObject	items, leaps, bombs, explosions, fireballs, arrows, carried objects [inf from names]
`TServerPlayer`	TLevelObject	networked character. `TPlayer` (self) and `TEditorNPC` derive from it

Class	Parent	Notes
`TComputer`	TSurfaceOwner	baddy ("compu")
`TAdventureHorse`	TSurfaceOwner	horse or boat
`TDisplayedNPCImg`, `TTempWeaponImg`	—	`showimg` overlays, weapon icons
`TAdventureWeapon`	TObject	weapon (name, image, GS1 script)
`TTileDef`, `TTempTile`	TObject	`addtiledef` and temporary tile overrides
`TGraalMap`	TObject	big-map / minimap overlay [inf]
`TGraalAni`, `TGraalAniStep`, `TGraalAniPart`, `TGraalAniSprite`, `TGraalAniSound`	TObject	GANI model (see `formats/gani.txt`)
`TCachedStream`	TObject	file cache [inf]

Script engine

`TAdventureCommand`, `TAdventureOperation`, and `TAdventureVariable` are the parse-tree nodes, with `TArray`, `TInteger`, `TDouble`, and `TStringPosList` as values. Execution is a recursive tree walk. See `npc-library/gs1-reference-205.md`.

UI forms

`TConnectFrame`, `TReconnectFrame`, `TServerListFrame`, `TLoadLevel`, `TServerMessage`, `TShortMessage`, `TPlayerList`, `TPlayerListOptions`, `TProfileEdit`, `TOptions`, `TMoreOptions`, `TLinkList`, `TLinkOptions`, `TShowCodes`, `TPersonAction` (the NPC "Set Npc's Action" script editor, not a player menu), `TChestEdit`, `TDebuggerFrame`. The editor and the client are **one executable** in 2.05: link, chest, and NPC editing forms are built in. They split into separate programs at 2.3 (see `version-tokens.md`).

Other formats seen in 2.05

- **GWEBL001**: a web-level cache file. It starts with the magic `GWEBL001`, then GSTRING fields and a GSHORT version that is checked against the client. The client also computes a running product hash over a key string, seeded with `0x04A80B38`. **[2.05]** The field list is not fully traced.
- A Delphi DES cipher class (`UDesCipher`) is linked in. It is not the game wire codec. It protects the saved passwords, the hidden listserver host (`<list-host-classic>`, port 14921, list `newmain`), and the `GWEBL001` web-level cache. **[2.05]**
- Image decoders: GIF (`TGifAnimation`), MNG (`TMNGAnimation`), JPEG. MNG animation fits with `gif2mng`.

Level magics accepted

`GLEVNW01`, `GR-V1.00`–`GR-V1.03`, `Z3-V1.03`, `Z3-V1.04`. No `GR-V1.04`. A `GRMAP001` reader is also present (with the generated-map and `HEIGHTS` keys), but no static caller was found; see `formats/gmap.md`.

`client-internals/client-assets.md`

Client asset tree

Date: 2026-10-14

Measured from the official client install. No decompile material.

Tile sheets

File	Size	Role
pics1.png	2048×512	Standard tiles, 128×32 of 16 px, 4096 ids
picso.png	1024×512	Offline tileset (type 1)
sprites.png	128×452	Sprite sheet
letters.png	—	Bitmap font

Tile id `t` maps to `col = (t % 16) + (t // 512) * 16`, `row = (t // 16) % 32`, source rect `(col*16, row*16, 16, 16)` on pics1. The sheet is eight side-by-side blocks of 512 ids, each 16 columns by 32 rows, filled row by row. (Corrected 2026-10-25: an earlier version of this note said `col = t % 128`, `row = t // 128`, which is wrong. See `tools/level-editor/222-reference.md` section 5.)

Tiledef

Offline line recovered from the tree:

```
picso.png,offlinetutorial,1,0,0
```

Format is `image,prefix,type[,x,y]`. Type 0 is the old Graal order (pics1). Type 1 is the new order. `addtiledef2` paints a rectangle on top of the base sheet.

levels/ counts (files on disk)

Dir	Files	Contents
ganis/	93	86 .gani plus images
midis/	49	music
heads/	35	33 png, 2 mng
weapons/	33	weapon images, 2 txt
bomys/	29	bomy images and ganis
emoticons/	29	png and mng
webgifs/	26	mostly png
backpals/	23	background palettes
bodies/	20	body images
images/	19	misc images
smilies/	15	
baddies/	10	one sheet per baddy
shields/	7	
swords/	6	
npcs/	5	script text
horses/	4	
hats/	4	
tutorial/	4	
letters/	2	blueletters, graal2002letters
snapshot/	2	ss_arrow, ss_bomb
tiledefs/	1	offline tiledef line

Other top-level

- maps/ holds bigmap.txt, minimap material.
- arrays.dat / arrays2.dat and the corresponding .sig files.
- bass.dll, basswma.dll, hvdi.dll.
- readme.txt documents HawkVoice (LGPL), joystick calibration, and classic controls (F1 options, F4 editor, F5 load, arrows/IJKL walk, S/space strike, A act, M map).

`client-internals/combat-200-204.md`

Combat in 2.00 to 2.04 (victim's client)

How the 2.00, 2.01, 2.02, 2.03 and 2.04 clients judge a sword hit and the hurt packet, compared with the 2.05 rev 2 rules in `decomp-findings-2026-10-24-d/` and `damage-sources.md`. Read in the binaries of 2.00 (rev 0), 2.01, 2.02, 2.03 and 2.04 (MD5s in `build-catalogue.md`), each run through the same Ghidra pipeline as `build-diff-204-205.md` (11.4.2, Delphi compiler spec). The 2.00 revs 1 to 3 and the 2.02 revs 1 and 2 were not read.

Cleaned note: no addresses, no decompiler output, no generated names. Nothing here was seen on a wire or in the lab.

Short answer

All five builds have a sword hit test and a damage source. None of them is without combat.

- **2.04 is 2.05 rev 2.** The sword judge, the reach box, the hurt routine, the can-be-hurt test, the heart change, the hurt packet handler (ids 40 and 46), the sword-power getter and the no-kill flag test are the same code as 2.05 rev 2. Only the field offsets move (the class layout changes listed in `build-diff-204-205.md`). The 2.05 rules apply to 2.04 as they stand.
- **2.03 differs from 2.04 in three places**: the no-kill flag sits on the player, not the level; the hurt packet has no per-player field test; and neither does the props path that leads to the sword judge.
- **2.01 and 2.02 differ from 2.03 in one more place**: the sword power is stored without the offset and limit, so it is 0 to 63 and never negative.
- **2.00 is the oldest shape.** No kick test, no shared can-be-hurt test in the sword judge, no `playerhurt` event, no id 46, and a hurt packet (40) with only the hurtable, guild and range gates.

The same in every build (2.00 to 2.05 rev 2)

- **Who decides.** The victim's client judges another player's swing from that player's props, as in 2.05.
- **Swing.** The attacker's animation step is 9 to 13 (sprite S 6 to 10).
- **Reach box.** The victim must be 0 to 4 tiles ahead of the attacker along its facing (both ends count) and strictly under 2 tiles to the side, measured between the two players' X/Y props with the same (1.5, 2) centre offset. The box test is skipped while the victim is in hurt mode (mode 3). The code is the same in all six builds.
- **Damage.** The attacker's sword power, as the victim last received it in SWORDPOWER (prop 8), in half hearts.
- **Wire form of prop 8.** A byte below 10 is the power (1 to 4 are the default swords, with their images). From 10 the power is the byte minus 30, followed by the image name.
- **Victim gates in the sword judge.** Victim AP not exactly 100; the attacker's `(tag)` not in the victim's nickname (the same bracket test as 2.05); two attacker flags clear, the same two flags 2.05 tests, which `combat.md` maps to STATUS 0x02 and 0x20 (that the 2.00 to 2.04 flags carry the same STATUS bits is inferred from their place in the test).
- **Hurt mode.** Mode 3 for 40 ticks (2 s), started for every accepted hit, with the knockback vector normalised.
- **Sword knockback direction.** Where the decompile keeps the box test's call arguments (2.00 and 2.02), the knockback handed to the hurt routine is the victim's centre minus the attacker's point, halved; the hurt routine then normalises it to length 1 (straight down when it is zero). In 2.01, 2.03, 2.04 and 2.05 rev 2 the decompile drops those arguments, so they were not compared.
- **Heart change.** A heal (negative change) needs AP at or above the same threshold expression as 2.05 (40). The paths that block it (swimming, a paused or editor state outside a sparring level) read the same.
- **Hurt packet (40).** Knockback `(byte - 0x60) / 16` tiles, power `byte - 32` half hearts, capped at one heart, and the attacker within 10 tiles (squared distance at most 100).
- **`noplayerkilling`** is in the GS1 command table of every build, and each build has a no-kill test in the sword judge (where it lives differs, below).

What changes, by build

Rule	2.00	2.01	2.02	2.03	2.04 = 2.05 rev 2
Kick test (no hit while the attacker plays `kick`)	absent	present (by animation name `kick`)	present	present	present
Victim test in the sword judge	its own: not hurt, not swimming, hearts above 0, play state exactly 0	the shared can-be-hurt test: as 2.00, plus a paused/editor test, and a non-zero play state is allowed in a sparring level	shared	shared	shared
Sword power store	the received power as is, in 6 bits (0 to 63)	as 2.00	as 2.00	power + 20, limited to -20..20	as 2.03
Negative sword power	cannot occur: it wraps in the 6-bit store (inferred from the store's mask)	as 2.00	as 2.00	heals at AP 40 or more	as 2.03
No-kill flag set by `noplayerkilling`	on the player, cleared when a level loads	on the player	on the player	on the player	on the level, with a tamper check
`playerhurt` script event on a hit	absent	present	present	present	present
Hurt packet id 46 (`PLO_HITOBJECTS`)	not handled	handled	handled	handled	handled
Hurt packet: AP 100 immune	no	yes	yes	yes	yes
Hurt packet: no-kill flag	no	yes	yes	yes	yes
Hurt packet: once per attacker per second	no	yes	yes	yes	yes
Hurt packet: guild test	yes	yes	yes	yes	yes
Hurt packet: per-player field equal (meaning not traced)	no	no	no	no	yes
Props path to the sword judge: same per-player field	no	no	no	no	yes

Notes on the table:

- **2.00's victim test.** 2.00's sword judge does not call the routine the other paths use. It requires the play state to be exactly 0, so a player in a sparring level with another state is not hit by a sword, and there is no separate paused/editor test. The hurt routine it calls does have the sparring exception.
- **The power store.** The per-player value is kept in a 6-bit slot in every build. In 2.00 to 2.02 the received power goes into it unchanged, so a power below 0 or above 63 does not survive (a wire byte of 10 to 29 gives a negative power). In 2.03 and later the power is limited to -20..20 and stored plus 20. This is the store; how a real 2.00 to 2.02 attacker's negative power looks to a victim was not seen.
- **No-kill.** In 2.00 to 2.03 the command sets a flag on the local player that is cleared on level load, and the sword judge and the hurt packet test it. In 2.04 and 2.05 the flag is on the level (and is read through a tamper-checked byte). Both end when a new level loads, so they behave the same for one player in one level.
- **Once per second.** In the builds read here the per-attacker second-of-day stamp is in the hurt packet path. It was not found in the sword judge of any build (2.00 to 2.05 rev 2). `combat.md` puts it on the sword; that placement was not confirmed by this read.

Confirmed, inferred, not tested

- Confirmed from the program: everything in the table and in "The same in every build", except the two items marked inferred.
- Inferred: that the two attacker flags are STATUS 0x02 and 0x20 in 2.00 to 2.04; that a negative power wraps in 2.00 to 2.02 (from the store's bit mask).
- Not read: 2.00 revs 1 to 3, 2.02 revs 1 and 2, 1.42 and earlier.
- Not tested in the lab or on a wire.

`client-internals/config-and-input.md`

Config and input

Date: 2026-10-14

Derived from observed client behavior, the public config key list, and format notes. No decompile material.

graalconfig.txt

> **Superseded** by `graalconfig-reference.md`, which lists every key 2.05 reads and writes. `serveraddr`, `loadcolorsfromserver` and `gamemode` below are not read by 2.05. The "password mask literal" is the fixed DES key for saved passwords (`saved-passwords.md`).

The client reads approximately 27 keys from `graalconfig.txt`. Known keys fall into four groups:

Connection - `serveraddr` — host override - `fixedport` — TCP port override - `fixedudpport` — UDP port override - `dontconnectlevels` — skip automatic level connect - `dontloadlistheads` — skip list header images - `loadcolorsfromserver` — pull color table from server

UI - `sortplayersbyname` - `limitnicknames` / `nicknamelimit` - `gamemode` - `language` - `font` / `fontsize`

Audio and input - `musicon` - `joystickon` - `nodirectinput` - `noudp` - `reversestereo`

Compatibility - `nodecompressionerror` - `nohtmlimages` / `nohtmlinpms` - `nomassmessages` / `notoalls` - `saveimages` - `compabilitydraw` — toggles a DirectDraw compatibility path

A password mask literal exists in the binary and is not decrypted. It is used only for display.

Device path

Joystick support is present. The device is activated when a specific internal bit is set. The WinMM joystick identifier is stored separately from the main window object. An offline session that does not need joystick input can ignore this path.

Drop files

Dragging a file onto the client window is handled only when an internal flag is set (the same flag that enables the editor drop target).

- A single `.txt` extension spawns a script NPC.
- Three specific image extensions (`.gif`, `.png`, `.mng`) spawn an image NPC.

- Anything else is treated as a level load.

This matches the offline observation that local level files, GANIs, and NPCs can be loaded from a `levels/` tree. An offline session that already accepts a dropped `.nw` or `.graal` file is exercising the same path.

Offline implication

None of the config keys are required for the offline slice. The drop-files behavior is the only part that intersects with current offline work: a dropped level file should load, and a dropped image or script should spawn the corresponding NPC. That is already the expected next step after collision and CmdCall.

Update 2026-10-21: 2.05 vs 2.22 graalconfig

Compared the 2.22 `graalconfig.txt` (user's Drive, 86 keys) with the 2.05 string table. **[2.05][doc]**

- **Present in 2.05:** `compabilitydraw`, `dontconnectlevels`, `dontloadlistheads`, `dontsavepasswords`, `dontsavepms`, `dontusedirectx7`, `fixedport`, `fixedudpport`, `font`, `fontsize`, `fullscreenmode`, `head`, `joyselected`, `joystickon`, `language`, `limitnicknames`, `nicknamelimit`, `nodecompressionerror`, `nodirectinput`, `nohtmlimages`, `nohtmlinpms`, `nomassmessages`, `notoalls`, `noudp`, `reversestereo`, `saveimages`, `sortplayersbyname`, `waitvertical`, `musicon`. That is about 29 keys, which fits the "~27 keys" estimate above.
- **Key bindings:** the only related 2.05 string is the bare prefix `keyset_`, so 2.05 probably builds `keyset_<action>_key0..2` and `keyset_<action>_joystickbutton` names at runtime [inf]. The actions in 2.22 are up, left, down, right, item, attack, action, map, switchtochat, itemlist, and pause, with defaults arrows/IJKL, D, S/space, A, M, Tab, Q, and P.
- **2.22-only** (absent in 2.05): `automapping`, the `micro*`/`voice*` voice-chat keys, `midivolume`/`mp3volume`/`radiovolume`/`sfxvolume`, `startinfullscreen`, `quickplayerworlds`.
- `fixedudpport` defaults to 14899 and `noudp=true` in 2.22. In 2.05 the UDP channel is a peer-to-peer channel between players; see `protocol/udp-p2p.md`.

`client-internals/cross-version-timeline.md`

Client cross-version timeline, 1.33 → 2.1 tests

Date: 2026-10-21

This note follows the Delphi client across twenty builds: 1.33 to 1.39, 1.41 rev1, NewMain 1.42, 2.0, 2.01, 2.02 rev2, 2.03, 2.04, 2.05 rev1, 2.05 rev2, the two 2.1 test builds and the two NewWorld betas. Builds are identified by MD5 in `build-catalogue.md`. Function data come from the same static analysis as `build-diff-204-205.md` (#11). All facts here come from RTTI, the GS1 command table and the packet switch tables, read the same way in every build. It is cleaned: no addresses, no decompiler output.

How it was read

- **Classes.** Class names, parents, instance sizes and virtual-slot counts come from the Delphi VMTs.
- **GS1 commands.** The script executor matches commands against one table of fixed-size records, each holding a pointer to the command text. The table was found by its first entry (`say`) and read to the end. Records are 16 bytes in 1.33 and 12 bytes from 1.34 on. Trailing spaces in a command (`say`, `setshape`) are part of the match text and mean the command takes arguments.

- **Packet ids.** Each packet and prop parser is a Delphi `case` on the first byte minus 32, compiled as a jump table (two-level in most parsers: a byte index table, then a pointer table). The ids that reach a real handler, not the default branch, are the ids the build handles. Ids are listed after the minus-32 step, so they are protocol ids.

Function maps onto 2.05 rev2

Every build was matched function by function onto 2.05 rev2 (6529 functions) with the matcher described in `build-diff-204-205.md` (#11), applying the 2.05 virtual-slot shift for the level-object classes. The per-build maps (old function, rev2 function, name, identical or changed) are kept outside the repo with the lab files, since they carry addresses.

Build	Functions	Identical to 2.05 rev2	Changed	Only in this build	Only in 2.05 rev2
1.33	5228	4312	345	571	1872
1.34	5240	4299	388	553	1842
1.35	5435	4479	383	573	1667
1.36	5437	4477	385	575	1667
1.37	5459	4494	395	570	1640
1.38	5546	4534	439	573	1556
1.39	5593	4550	470	573	1509
1.41 rev1	5675	4783	419	473	1327
NewMain 1.42	6468	5592	507	369	430
2.0	6396	5637	515	244	377
2.01	6412	5663	528	221	338
GraalX (2.01 with `.nw`)	6429	5662	531	236	336
2.02 rev2	6350	5678	484	188	367
2.03	6428	5754	479	195	296
2.04	6471	5799	558	114	172
2.05 rev1	6522	6345	94	83	90
NewWorld beta 1	6381	5810	405	166	314
2.1 test (first)	6598	5774	611	213	144
2.1 test (second)	6595	5762	619	214	148
NewWorld beta 2	6484	5668	489	327	372

Most of the identical count for the 1.3x builds is shared Delphi library code. The largest single step is 1.41 rev1 to NewMain 1.42, about 800 more functions, part of which is the new DirectDraw 7 and grid library code. GraalX (see `build-catalogue.md`) matches 2.01 in its classes, GS1 commands and packet ids, so it is left out of the tables below. Against 2.01 it differs only by the `.nw` reader and writer. The 2.1 tests have more functions than 2.05 but fewer that are unique to rev2, which fits them being a later branch of the 2.05 code.

Classes

Build	Added	Removed
1.34	`TServerPlayer`, `TEditorBaddy`	`TOtherClient`, `TEditorOpponent`, the OLE classes (the new names look like renames of the removed ones)
1.35	`TWebBrowser` and the OLE control classes	—
1.36 to 1.38	—	—

Build	Added	Removed
1.39	`TPlayerListOptions`, `TProfileEdit`	—
1.41 rev1	`TGraalClient`, `TServerListClient`, `TServerListFrame`, `TServerMessage`, `TShortMessage`, `TWinamp`	`TAdventureBoolOperation`, `TRTFMessage`
NewMain 1.42	`TGraalAni` (+ `Part`, `Sound`, `Sprite`, `Step`), `TGraalImage`, the DelphiX DirectDraw 7 classes, string and draw grids, `TClipboard`	`TAdventureSign`
2.0	—	the DDE client/server classes
2.01	—	—
2.02 rev2	—	the zlib error classes (back in 2.03)
2.03	`TNPCMove`	—
2.04	`TLevelObject`, `TObjectSet`, `TTempTile`, `TTileDef`	—
2.05 rev1	`TGraalMap`, `TADP` (anti-debug)	`TEditorBaddy`, `TEditorLevel`
2.05 rev2	—	—

Against 2.05 rev2:

Build	Added	Removed
NewWorld beta 1	`TLevelHeights`, `TAdventureBacktileArea`, `TDrawText`	horse, ball and carry objects, `TComputer`, `TEditorChest`, `TChestEdit`, `TEditBaddy`, `TPos`, `TWebClient`
2.1 test (both)	`TEditNPC`, `TEditVariable`, `TEditorDefNPC`, `TPageControl` / `TTabSheet`, `TMemo`; the second test adds `TSmilie`	`TPersonAction`, `TEditPerson`, `TShowCodes`, `TObjectSet`
NewWorld beta 2	both lists above, including `TSmilie`	both lists above

So **NewWorld beta 1 was cut from the 2.05 code base and beta 2 from the 2.1 test code base.** Beta 2 carries every 2.1-test class change on top of the NewWorld removals.

Sizes of the main game classes (bytes, virtual slots)

Class	1.33	1.34	1.38	1.39	1.41	NM 1.42	2.0	2.01	2.02r2	2.03	2.04	2.05	NW b1	2.1 test	NW b2
`TAdventure` (main form)	3268	3240	3244	3256	3268	1864	1864	1860	1860	1860	1084	1084	1084	1108	1108
`TAdventureLevel`	56	84	84	84	84	84	88	88	88	88	96	184	33968	184	33968
`TAdventureCommand`	64	68	76	76	68	68	68	68	68	68	68	76	76	76	76
`TDrawableObject`	20	20	20	20 / 2	28 / 2	28 / 2	28 / 2	28 / 2	28 / 2	28 / 2	28 / 2	32 / 2	32 / 3	32 / 3	44 / 3	44 / 3
`TServerPlayer`	—	220	224	240 / 2	272 / 2	272 / 2	272 / 2	280 / 2	288 / 2	288 / 2	340 / 7	336 / 8	320 / 8	364 / 9	348 / 11
`TPlayer`	1516	1512	1524	1544	1564	1496	1500	1516	1528	1528	804	800	760	832 / 836	792
`TEditorNPC`	208	220	228	224	244	476	476	488	496	508	568	564	548	592	572
`TGraalMap`	—	—	—	—	—	—	—	—	—	—	—	76	96	256	34060

Two points for anyone carrying names between builds by virtual-slot number:

- **2.04 puts `TLevelObject` under the player and NPC classes**, which is why `TServerPlayer` jumps from 2 to 7 slots. Slot numbers from 2.03 and earlier do not line up with 2.04 for these classes.
- **2.05 adds a slot to `TDrawableObject`**, shifting every descendant by one (see `build-diff-204-205.md`, #11). The 2.1 tests and NewWorld beta 2 add more slots to the player classes again.

The NewWorld `TAdventureLevel` (33968 bytes) and beta 2 `TGraalMap` (34060 bytes) are large enough to hold tile arrays inline, which fits the NewWorld level heights and back-tile layers.

GS1 commands

Count of commands in the table, and what each build adds over the previous one. The table is followed directly by a short direction table (`up`, `right`), and in some builds by other keyword lists (`*=`, `:=`, the 1.33 soldier names, and `nwweek` / `nwmonth` / `nwmin` in the NewWorld betas). Those entries are not counted. In 2.05 this gives 161 commands, matching the earlier count on main.

Build	Count	Added
1.33	88	—
1.34	93	`dontblocklocal`, `blockagainlocal`, `showlocal`, `hidelocal`, `carryobject`, `throwcarry`, `setbacktile`; `seteffect` and `unseteffect` leave the ta in NewMain 1.42)

Build	Count	Added
1.35	93	same table as 1.34
1.36	104	`lay2`, `take2`, `takehorse`, `takeplayerhorse`, `removebomb`, `removearrow`, `removeitem`, `removeexplo`, `removehorse`, `explodebomb`, `reflectarrow` arguments
1.37	106	`playlooped`, `stopsound`
1.38	109	`setlevel2`, `changeimgpart`, `hitnpc`; `setlevel` now takes arguments
1.39	111	`setani`, `setcharani`
1.41rev1	126	`canwarp`, `callnpc`, `say2`, `break`, `continue`, `callweapon`, `spyfire`, `function`, `return`, `changeimgvis`, `setimg`, `setimgpart`, `drawovertrees`, `put` `showstats`; `putexplosion` now takes arguments
NewMain1.42	134	`canwarp2`, `setshape`, `seteffect`, `setcoloreffect`, `setzoomeffect`, `drawaslight`, `changeimgcolors`, `changeimgzoom`
2.0	142	`setgender`, `setchargender`, `join`, `freezeplayer2`, `unfreezeplayer`, `enabledefmovement`, `disabledefmovement`, `setmusicvolume`
2.01	144	`openurl2`, `hitobjects`; `openurl` now takes arguments
2.02rev2	146	`tokenize`, `tokenize2`
2.03	148	`triggeraction`, `move`
2.04	157	`addtiledef`, `addtiledef2`, `removetiledefs`, `replaceani`, `attachplayertoobj`, `detachplayer`, `setshape2`, `enableselectweapons`, `disablese` `setshape` now takes arguments

Build	Count	Added
2.05rev1/rev2	161	`setbacktile2`, `setz`, `updateterrain`, `loadmap`; `setbacktile` now takes arguments
2.1tests	161	same table as 2.05
NewWorldbeta1	161	same table as 2.05
NewWorldbeta2	165	`addstring`, `insertstring`, `removestring`, `deletestring`

A command being in the table does not mean it does anything. #4 lists the 2.05 commands that have no executor case, so they parse and are ignored. Examples include `setz`, `updateterrain` and `setbacktile2` (2.05), `setbacktile` (1.34), `drawovertrees` and `canwarp` (1.41), `canwarp2` (1.42), `join`, `freezeplayer2` and `unfreezeplayer` (2.0), and `seturllevel`, `setfocus` and `resetfocus`. See #4 for the full list. This note has not checked executor cases in the other builds.

New commands are inserted in the middle of the table, not appended. So the internal index of a command changes between builds, and an executor decompiled in one build cannot reuse another build's case numbers.

Packet ids handled by the client

Server → client packets

Build	Change
1.33 to 1.37	43 ids: 0–41, 100
1.38	+ 43
1.39	+ 75 (45 ids: 0–41, 43, 75, 100)

Build	Change
1.41 rev1	same set
NewMain 1.42	+ 42, 44, 45, 150, 151, 153
2.0	+ 154, 155
2.01	+ 46, 156
2.02 rev2	+ 47
2.03	+ 48, 165
2.04	+ 166; − 26
2.05 rev1 / rev2	+ 168
NewWorld betas	2.05 minus 2, 4, 17, 18, 154, 155
2.1 tests	2.05 plus 49, 169

Id 42 is not handled until NewMain 1.42, although 43 is handled from 1.38 on. NewWorld drops six ids at once, alongside the removed baddy, horse and chest classes.

Prop parsers

Each prop switch reads a run of `(prop id, value)` pairs. Ids handled:

Parser	1.33	Added later
Player props	0–33 *[inf]*	34 (1.39); 35, 36 (1.41); 37–41 (2.02 rev2); 42 (2.04); 43–45 (2.1 tests)
NPC props	0–34 *[inf]*	35 (1.41); 36–40 (2.02 rev2); 41–43 (2.1 tests)
Baddy props	0–10	none; the switch is gone in both NewWorld betas

The player switch was told apart from the NPC switch by its callers: the player prop parser is the one that calls the sparse switch described below. That holds in every build from 1.34 on. In 1.33 neither switch calls it within its own body, so the 1.33 row assumes the same split as 1.34 (one switch is 0–33, the other 0–34). So NPC prop 34 is handled from 1.34 at the latest (from 1.33 if the assumed split holds), well before player prop 34 in 1.39.

The five ids added together in 2.02 rev2 (player 37–41, NPC 36–40) sit at the same place in both lists, so they are probably one feature added to players and NPCs at once. This is consistent with the public prop numbering used by open-source server work, where both lists gain five attribute props at these positions.

The player prop parser also calls a second, sparse switch with ids up to 51 (21 ids in 1.33, 23 in 1.39 and 31 in 2.05). It handles a subset of player props and grows in step with the player prop list. Its role (likely props of other players) has not been confirmed.

UDP packets

The UDP parser handles ids 0 and 10–23 in 1.33 to 1.37. 1.38 adds 24, 2.01 adds 25, 2.03 adds 26 and the 2.1 tests add 27. The NewWorld betas drop 16 and 17.

`client-internals/damage-sources.md`

Damage from bombs, arrows, thrown objects and baddies (2.05)

How many hearts each non-sword source takes from the local player, and when it hits. Read from the 2.05 program. The sword rule is in `decomp-findings-2026-10-24-d/01-sword-damage.md`.

Each source below calls the same hurt routine with a power in hearts and a knockback direction. That routine takes the hearts off, starts hurt mode (40 ticks), and runs `playerhurt`. It does nothing while the player cannot be hurt: already in hurt mode, dead, swimming, or in the editor or paused, as in the sword notes. So overlapping pieces of one blast take hearts only once.

Powers

source	hearts taken
explosion piece of type 1 (`putbomb 1`, `putexplosion`, the blast of a fireblast or nukeshot)	0.5
explosion piece of type 2 (`putbomb 2`, the super bomb)	0.5
explosion piece of type 3 (`putbomb 3`, the jolt bomb)	1
arrow (`shootarrow`)	0.5
fireball (`shootfireball`)	1
fireblast (`shootfireblast`)	0.5, then a blast
nuke (`shootnuke`)	1, then a blast
thrown object (`waspelt`)	1, or 0.5 when the player has exactly 1 heart left
baddy touching the player	0.5, or 1 for the golden warrior (type 7)

Bombs and explosions

- `putbomb power,x,y` makes a bomb with that power. Powers 1 to 3 are the bomb, super bomb and jolt bomb of the built-in weapons.
- When it goes off:
  - Power 3 plays `thunder.wav`, and any other power plays `bomb.wav`.
  - Power 2 makes five blasts: four of reach 2 at the four diagonal corners, 2 tiles out, and one of reach 4 in the middle. All five are type 2.
  - Any other power makes one blast of reach 2, with the bomb's power as its type.
- A blast of reach `n` (at most 15) is a cross: a centre piece, then `n` pieces in each of the four directions. The last piece of each arm uses the end picture.
- `putexplosion radius,x,y` makes one blast of type 1 with the given reach.
- A piece hurts the player when its centre, offset by one tile, is less than one tile from the player's centre (x + 1.5, y + 2) on both axes. The knockback is the piece's direction.

Projectiles

- The four shoot commands make a flying object of type 0 (arrow), 1 (fireball), 2 (fireblast) or 3 (nuke), at speed 1, and play `arrow.wav`.
- Types 1 and 3 take 1 heart and the others take 0.5.
- After hitting, types 2 and 3 make a blast of reach 2 and type 1 at the hit.

Thrown objects

- A thrown object hurts the player when it lands within 2 tiles of the player's point (x + 0.5, y + 1) on both axes.
- It takes 1 heart, except when the player has exactly 1 heart left, when it takes half a heart.

Baddies

- A baddy that touches the player takes half a heart. The golden warrior takes 1.

- Touching means the player's centre is within one tile of the baddy's point on both axes.
- On a hit, the baddy may say its "when it hurts the player" verse.

Who can hurt whom

- A bomb, blast, arrow or thrown object can belong to another player. If it does, and the level has run `noplayerkilling`, it does not hurt.
- A source that belongs to no player, such as a baddy's or the level's own, always hurts.
- When a player-owned source leaves the player at 0 hearts or fewer, the client reports the owner as the killer.

Other players' bombs, blasts and shots

How the 2.05 client takes the server's copies of another player's bomb (`PLO_BOMBADD`, 11), explosion (`PLO_EXPLOSION`, 36) and shot (`PLO_ARROWADD`, 19), and when the result hurts the local player. Read from the 2.05 rev 2 program. The bytes follow the sender's player id; every value below is `byte - 32` unless it says otherwise.

Bomb (11)

- Needs at least 4 bytes: `x×2`, `y×2`, power, fuse.
- Power is `v & 7`; bit 8 marks the bomb as the sender's. The fuse is the byte itself, in ticks.
- The bomb goes through the same routine as a local `putbomb`, so water under `(x + 1, y + 1)` makes a splash and no bomb. Nothing is sent back.
- It is put in the sender's level (the local player's level when the sender is unknown).
- It burns and goes off exactly like a local bomb (60-tick fuse for a local `putbomb`, the sender's copy 5 ticks shorter), with the sender as the owner of every piece.

Explosion (36)

- Needs at least 4 bytes: reach, `x×2`, `y×2`, power.
- Power is `v & 7`, 0 read as 1 and anything above 3 read as 3; bit 8 marks the explosion as the sender's.
- It makes one cross of that reach (at most 15) and that type at `(x, y)`. The five-cross split of the super bomb belongs to a bomb's bang only, not to this packet.
- It plays one of two sounds, chosen by whether the power is 3. The two names are not visible in this routine; that they are `thunder.wav` and `bomb.wav`, as for a bomb, is an inference.

Shot (19)

- Needs at least 5 bytes: `x×2`, `y×2`, flags, sprite, type. The type byte is `byte - 33`.
- Flags: `& 3` is the direction, 4 first removes a shot this client already has, 8 marks the shot as the sender's.
- The byte before the type is the sprite id, not the type. Arrows use sprites 107 to 110 (107 + direction).
- The shot flies 1 tile per tick along its direction.
- With flag 4, the client first deletes every shot it holds that flies the opposite way with the same sprite (any arrow sprite matches any other), on the same line. That this is the copy of a shot a shield sent back is an inference.
- In a level that has run `noplayerkilling`, the receiver marks the shot as a player's shot, and such a shot never hits the local player.

When a piece hurts the local player

Each tick of its 11-tick life, a piece hurts when all of these hold:

1. the piece is in the player's level;
2. the player can be hurt (see the sword notes);
3. the centre test of "Bombs and explosions" above passes.

Then the ownership test: the piece counts as a player's when the client is online, the piece has an owner, and that owner is still in the client's player list. A player's piece in a `noplayerkilling` level does not hurt. A piece whose owner has left counts as nobody's, and it hurts.

A hit takes half a heart, or a whole heart for a piece of type 3, and knocks the player along the piece's arm. When a player's piece leaves the player at 0 hearts or fewer, the client reports the owner as the killer.

When a shot hurts the local player

Each tick the shot moves first, then:

- **Level edge.** Past -2 or at 64 and beyond in its direction, it moves to the neighbouring level of the loaded gmap block, or it is gone.
- **Hit test.** It is skipped while the shot is bouncing back, when it is a player's shot (the local player's own, or one received in a `noplayerkilling` level), or when the player cannot be hurt.
  - The shot's size comes from the sprite table entry `v` (array 24): width `((v >> 6) & 63) / 16` and height `(v & 63) / 16` tiles.
  - It hits when `x + w > px + 0.5`, `x < px + 2.5`, `y + h > py + 1` and `y < py + 3`, where `(px, py)` is the player's position.
- **Shield.** Before hurting, a shield can stop the shot. This needs all of:
  - the player faces the shot;
  - the player's shield power is above 0;
  - the shot is of type 0 or 1;
  - the player's animation is one of two names from the built-in animation list. That these are `idle` and `walk` is an inference.

  The result depends on the shield power:
  - Power 1: an arrow sprite turns back for 3 ticks with a sound. Any other sprite is gone.
  - Power 2 or more: the shot is sent back the other way, with its sprite turned, and the client sends it again as a new shot.
- **Hurt.** Otherwise the shot hurts. It uses the same ownership and `noplayerkilling` test as a piece, and the same killer report. It takes half a heart, or a whole heart for type 1 or 3. The knockback the shot passes is not visible in this decompile.
- **After a hit.** The shot is gone. A shot of type 2 or 3 leaves a blast of reach 2 and type 1, owned by the shooter, at the hit.

Open

- **Walls.** In the decompiled flow, types 0 and 1 skip the blocking test that stops types 2 and 3, so arrows and fireballs would fly through walls. The blocking test lost its arguments in the decompile, so this is not settled. Check it in the lab.
- **Baddies.** The baddy touch above was not re-read here.
- **The two shield animation names.** See "Shield" above.

Confirmed, inferred, not tested

- Confirmed from the program: the powers in the table, the bomb split, the blast shape, the projectile types, the hit boxes and the ownership rule.
- Inferred:
  - that bomb powers 1 to 3 are the bomb, super bomb and jolt bomb (from the built-in weapon scripts);
  - that the noplayerkilling check is the level flag the sword gate uses.
- Also confirmed from the program: the receive side of ids 11, 36 and 19, the owner and `noplayerkilling` test, the killer report, and the shot's hit box. The inferences there are marked in place.
- Not tested in the lab or on a wire.

`client-internals/decomp-findings-2026-10-24-b/01-tamper-check-id39.md`

Tamper-check (client PLI 39): how the 10 bytes are built

Derived facts only. All six lab values (two captures each of 2.17.1, 2.22 and 2.31) were reproduced offline from the client executables, so the description below is verified, not guessed. Lab captures and re-send timings: `version-profiles/tampercheck-values.tsv`.

What the 10 bytes are

- Two 32-bit words, A then B, each written as 5 bytes. The 5-byte form is NOT the normal GINT5 and NOT masked:
  - byte 1 = ((v >> 28) & 0x0F) + 0x20
  - byte 2 = ((v >> 21) + 0x20) & 0xFF
  - byte 3 = ((v >> 14) + 0x20) & 0xFF
  - byte 4 = ((v >> 7) + 0x20) & 0xFF
  - byte 5 = (v & 0x7F) + 0x20 Only bytes 1 and 5 are range-limited, so bytes 2-4 can be below 0x20 or above 0x7F. That is why the lab saw bytes under 0x20 and why "two GINT5" is wrong. (Layout is the same in 2.17.1, 2.22, 2.31.)
- Both words are standard reflected CRC-32 values (polynomial 0xEDB88320, initial value 0xFFFFFFFF) computed over the client's OWN memory image (the running exe, in memory), chained: the first call finishes with the usual final inversion, its result is used as the starting value of the next call, and the last call of each word skips the final inversion.
- Word A covers 8-9 contiguous ranges of the code section (a sliding selection of code, not the whole section).
- Word B covers one large code range plus one block of static data (a table of string pointers in the data section).
- Nothing in the input depends on login key, account, level or time. So per build the result is a constant, exactly as the lab saw. The value only changes if the exe bytes change (a patched or re-packed exe changes it; an unpatched original does not).

Per-build values (words in hex, lab bytes in hex)

See `version-profiles/tampercheck-words.tsv`. Short form: - 2.17.1: A ab637cbe, B 3e48dc0c, bytes 2a7bad195e231243d82c - 2.22: A 5dcb77f6, B bc736311, bytes 250e4d0f962b03ede631 - 2.31: A 9ecd4bc0, B ecea65e2, bytes 291655b7602e87c9eb82

What a Seihai client must do

- Replaying the constant per emulated build is enough (it is what the real client sends). Computing it needs the original exe bytes plus the range lists, which are inside each exe; not needed unless Seihai wants to accept patched exes.
- Position/timing per build: see `03-periodic-ids-2.31.md` and the table at the end of this file.

When it is sent

build	login	later
2.17.1	yes, once	never in ~90 s of lab
2.22	yes	re-sent by the level-change routine on every level change (so each gmap edge crossing in the lab); no timer
2.31	yes	every 3.2 s; a 50 ms main-form timer drives it (see 03)

Does the server verify it?

Lab: GServer-v2 accepts all three and logs nothing. The decompiles only show the sender; no check on the receive side of the client. Treat as sent-and-ignored for a compatible server.

`client-internals/decomp-findings-2026-10-24-b/02-server-to-client-layouts.md`

Server -> client payloads above id 48 (and 25, 39, 41, 44, 47, 156, 168-189): 2.17.1, 2.22, 2.31

Source: the receive dispatchers of the 2.17.1, 2.22 and 2.31 clients, read from the disassembly. Full table: `version-profiles/plo-layouts-217-231.tsv`. Handled-id set per build: `version-profiles/plo-handled-ids-by-build.tsv`.

Dispatcher shape (all builds)

- id = first byte - 0x20; bound check; byte-index table; dword jump table; every unhandled id shares one default arm that does nothing.
- Dispatch bounds: 2.17.1 up to id 185 (bound 0xb9); 2.22 up to 186 (0xba); 2.31 up to 189 (0xbd).
- Number encodings read by the handlers: GCHAR = byte-0x20; GSHORT = 7-bit shifts (b0<<7 + b1); GINT3 shifts 14,7,0; GINT4 21,14,7,0; GINT5 28,21,14,7,0. Text payloads are the rest of the line. Several "comma text" ids go through the TStrings comma-text parser (double quotes allowed).

Handled ids above 48

- 2.17.1: 49, 68, 69, 75, 100, 150, 151, 153-156, 165, 166, 168-185. (79 handled ids in total.)
- 2.22: same + 186. (80 total.) Id 7 is a three-instruction stub forwarding to a helper (in 2.17.1 the body is inline); wire format of 7 unchanged as far as the diff shows.
- 2.31: the 2.22 set + 84 + 189, minus 44. (81 total.)
- Ids 50-67, 70-74, 76-83, 85-99, 101-149, 152, 157-164, 167 are unhandled in all three (default arm). 152 and 167 are NOT handled in any of them.

Layouts, short

(see `version-profiles/plo-layouts-217-231.tsv` for the full text and version gates; "dec" = decoded from the binary) - 25 SIGNATURE: GCHAR v. v=1 quits; v=0 quits unless host is the loopback address; other values ignored. - 39 LEVELMODTIME: raw rest passed to active gmap/level object (needs an active level). - 41 STARTMESSAGE: raw RTF/HTML text, cached as servermessage.rtf/.html, shown in viewer. - 44 HASNPCSERVER: no payload, 2.17/2.22 only, gone in 2.31. - 47 STAFFGUILDS: comma list. - 49 PLAYERWARP2: GCHAR x2, GCHAR y2, GCHAR z (unused here), GCHAR gmapX, GCHAR gmapY, level name. x,y halve (value/2.0). gmapX/Y only used for .gmap levels, clamped to the grid. - 68/69 LARGEFILESTART/END: raw name; 84 LARGEFILESIZE (2.31 only): GINT5. - 75 PROFILE: repeated GCHAR n + n chars; first field = account, rest = lines. - 100 RAWDATA: GINT3 length. - 150 NPCDEL2: GCHAR n, n-char level name, GINT3 npc id. 151 HIDENPCS: none (removes NPCs 1..999 of active level). - 153 SAY2: text. 154/155 FREEZE/UNFREEZE: none. 156 SETACTIVELEVEL: name, .gmap handled specially; 2.31 adds a follow-up step. - 165 MOVE: GINT3 id + rest. 166 SHOWIMGNPC: GINT3 id + props. 189 MOVE2 (2.31): same shape as 165, extra flag. - 168: payload ignored (local reset). 169: URL text. 170 GCHAR flag. 171/172: comma text txt,png,x,y (needs >=4 fields). 173 text. 174 GCHAR flag. - 175 SHOOT: GSHORT player id + rest, only when in the active level. - 176/177: mode switches, no payload. 178 SERVERWARP: comma text with >=4 fields, field 0 unused, 1 = name, 2 = host, 3 = port (lab-confirmed on 2.31, `08-lab-findings-levelchange-24-178.md`). - 179 RPGWINDOW: text. 180 STATUSLIST: text. 181: GSHORT id + GCHAR toggle. - 182 LISTPROCESSES: no payload; client replies PLI 44 with its process list. 183 HASPROCESSRUNNING: names; client replies PLI 45 = GCHAR 0/1 (whether the server's string occurs in its own process list). (These are PLI ids sent BY the client; the server-side PLO 44 is something else.) - 184 TAKESCREENSHOT: file name, only ".png" accepted, saved under a screenshots folder. - 185 BOARDHEIGHTS: GCHAR mapX, mapY, blockX, blockY, w-1, h-1 then w*h cells of two raw bytes, value = (b0-0x20-50) + (b1-0x20)/128.0. - 186 BOARDMODIFY2 (2.22+): GCHAR mapX, mapY, x, y, w, h then w*h GSHORT tiles row-major (bytes after the tiles are ignored padding: lab-tested with injected frames, see `07-lab-findings-46-186-200.md`).

Differences 2.17.1 -> 2.22 -> 2.31 that change behaviour or ids

- 2.17.1 -> 2.22: new id 186; id 7 refactored into a helper; every other handler's wire format identical after diffing 37, 42, 49, 75, 150, 151 (only local-variable/offset noise). Tamper check re-sent on level change (new). Login token changes GNW22122 -> GNW03014.
- 2.22 -> 2.31: id 44 no longer handled; new ids 84 and 189; id 6 (level name) now also compares with the current level name; 68 clears the stored file size; 156 gets an extra step; object offsets shift by 0x40 in the player/level classes (irrelevant on the wire). Timer-driven PLI 39/24 and PLI 46 added (see 03).

`client-internals/decomp-findings-2026-10-24-b/03-periodic-ids-2.31.md`

2.31 periodic traffic: PLI 24, PLI 39 re-send, PLI 46

Mapping of the lab observations to the client's behaviour (all values are durations).

PLI 39 re-send every ~3.2 s

- The main form of 2.31 has a 50 ms timer. Its handler keeps a counter modulo 64 (64 = length of the 64-character alphabet string used there), so one full cycle = 64 x 50 ms = 3.2 s.

- Phases of the cycle (counter value -> action):
  - 1: anti-debug probe (a privileged port-I/O instruction guarded by an exception handler, then a breakpoint-trap check), and a 5000 ms timer is armed.
  - 10: recompute CRC word A of the tamper check.
  - 20: scan all top-level windows (title and class) against an obfuscated blacklist (cheat-tool detection). The list was not decoded and is not needed.
  - 30: recompute CRC word B.
  - 31: send PLI 39 with the freshly computed A+B.
- Because A and B recompute to the same constant each time (see 01), the 10 bytes are identical on every re-send (lab confirmed). I did not find the blacklist scan or the probe result feeding into the words; what the client does on a hit was not traced.
- 2.17.1 and 2.22 have no such timer. 2.22 only re-sends from the level-change routine.

PLI 24 about every 5.1 s (1 byte, lab value 9)

- 2.31 only. The 5.1 s cadence fits the 5000 ms timer armed in phase 1 (5.0 s plus timer granularity). That link is an inference, not traced to the end.
- **Lab batch c (`08-lab-findings-levelchange-24-178.md`):** 2.31 sends it at login and every 5.11 to 5.17 s (median 5.13 s) whatever it is doing, in every level; level changes neither trigger nor reset it. **2.22 also sends 24 = 9** every 5.1 s, but only while another player is in the same level (0 in 3 solo sessions), so "2.31 only" is wrong for two-player 2.22. The stock server relays any payload unchanged as `PLO_SHOWIMGPLAYER` (32) to the others in the level; injected payloads caused no reaction.
- The payload byte (value 9) is a constant in the sender; I could not tie it to anything else. Meaning NOT resolved (heartbeat-like by behaviour only). Treat as: send GCHAR 9 as PLI 24 every ~5 s on a 2.31 emulation; unknown whether servers care.

PLI 46 on every level entry (GCHAR 1)

- Constant 1 byte (`GCHAR 1`). **Updated by the lab run of batch b2:** it is sent on every level entry, not only at login (7 sends in one gmap run: login plus six edge crossings), always just before the props message that carries the cell prop (43 or 44) and the wrapped coordinate. Details: `07-lab-findings-46-186-200.md`.
- 2.31 does not re-send PLI 39 at a crossing. Not sent by 2.17/2.22.
- **Lab batch c:** also sent on doors, edge links and server-side warps (always just before the first props of the new level; in the same frame as a cached-level link request, otherwise in the first frame after the server's level answer). 2.31 does not re-send 39 there either. 2.22 sent no 46 in 0 of 8 transitions, but sent 39 after each client-initiated link request. Details: `08-lab-findings-levelchange-24-178.md`.

What to implement

- 2.31: send PLI 46 = GCHAR 1 at login and on each level change; PLI 39 (constant) every 3.2 s; PLI 24 = GCHAR 9 every ~5.1 s.
- 2.22: PLI 39 at login and after each client-initiated level-link request (not after a server-side warp); no 46; PLI 24 = 9 every ~5.1 s while another player is in the level. 2.17: login only.

`client-internals/decomp-findings-2026-10-24-b/04-v2.00-client.md`

Graal 2.00 (and rev1-3, 2.01-2.05): login, tables, lab setup

> **Batch c (`../decomp-findings-2026-10-24-c/`) supersedes the 2.00 and 2.05 parts of this note where they differ.** It decompiled all four 2.00 builds and 2.01 to 2.1 test with a rebuilt dispatcher. The tokens, the id sets and the dropped id 26 here match it. Corrections: id 6 is a real sender (below), and the four 2.00 builds have identical handler shapes.

Sources: the 2.00 decompile and the four 2.00 builds from the Drive clients set (`../drive-clients-inventory.md`). All four 2.00 builds give the same token and the same tables. Derived facts only.

Login string (gserver)

- Built from: GCHAR(random key & 0x7F) as the key byte, then the 8-char token `GNW24011` (assembled character by character from immediates, so a plain text search finds nothing), then GCHAR(len account) + account, GCHAR(len password) + password. It is sent through the client's send method with packet type 0 (same "type 0, key byte, token, account, password" shape as 2.05 and 2.17). Account and password are NOT hidden or hashed in 2.00.
- The key byte seeds the gen-3 stream cipher framing used afterwards (zlib + one junk byte per message after the first, as documented for 2.05 and 2.17). No new framing in 2.00.
- Tokens for the whole 2.00-2.1 family: `version-profiles/version-tokens-by-build.tsv` (all confirmed from the binaries) and `../version-tokens.md`.

Server -> client ids handled (2.00)

- Dispatch bound 0x9b; 53 ids: 0-45 (including both 25 and 26), 75, 100, 150, 151, 153, 154, 155. No 46-49, 156, 165-169, 68-69.
- Growth: 2.01 +46, +156 (55 ids); 2.02 +47 (56); 2.03 +48, +165 (58); 2.04 drops 26 (25 stays in every build; 2.04 does not swap one for the other) and adds 166; 2.05 +168 (59); the 2.1 test builds +49, +169 (61). Full list: `version-profiles/plo-handled-ids-by-build.tsv`.
- `PLO_SIGNATURE` (25) is handled in 2.00 already, so the kill-switch exists from the first 2.x client.
- `PLO_NPCACTION` (26) is handled by 2.00-2.03 and dropped in 2.04. Later clients ignore it. Its payload is not decoded here.

Client -> server ids the 2.00 binary can send

0-24, 26-30, 32-34, 80, 81. Compared with 2.05 rev2 (0-24, 26-30, 32-38, 80, 81): 2.00 lacks 35-38; 35 and 36 arrive in 2.01, 37 in 2.02 rev2, 38 in 2.03. Table: `version-profiles/pli-sent-ids-by-build-20x.tsv`. Method: call sites of the client's send method with a constant id. The 2.05 rev2 result matches the id set already in `protocol/packet-ids.md`, so the method is sound. Id 6 (`PLI_TOALL`, chat) shows up in every build. **Correction (batch c):** it is a real sender in all 20 builds, with chat text clipped to 200 characters, not a scan artefact (`../decomp-findings-2026-10-24-c/05-pli-client-to-server.md`).

Token allow-list

- A GServer-v2 gserver has a text allow-list of tokens. For a lab, a **config entry** for `GNW24011` in that list is enough. No binary patch is needed on the server, and none is needed in the client for the login itself.

(Earlier notes, `version-profiles/version-coverage-2026-10-24.md`, said an allow-list patch; that meant this config entry.)
- Still needed by the 2.00 client: a list server it can reach. The list host is DES-hidden inside the exe (scheme in `../des-and-hidden-strings.md`, port 14921 per `protocol/listserver.md`), and 2.00 does not read `arrays2.dat`. For a lab, resolve that name to the lab listserver and run the lab listserver on 14921, or patch the hidden string.
- **Partly observed (batch b2, `07-lab-findings-46-186-200.md`):** the 2.00 client sends one plain zlib frame, `GCHAR 0` + `newmain` + LF, with no account or password, then makes no second listserver connection and goes straight to the gserver. 2.00 accepted a 2.17-style two-frame list reply, but that reply was **synthetic** (a lab-only forwarder translated the request). What a real 2.00-era listserver sends is still unobserved. Table: `version-profiles/lab-200-listserver-handshake.tsv`.

Does it run?

Yes under Wine 10 (separate prefix, loopback-only lab network, private display), with a data-file caveat. The Drive 2.00 exe alone stops with "Cannot open file commands.rtf", because the Drive folder holds only the exe. Placing the data files of a NewMain 26 folder (commands.rtf, images, languages) beside it makes it reach its start screen. A copy of the 2.22 data folder also works for the start screen and login, except that `objects.dat` is missing there and 2.00 needs it (nickname box, Training / Options / Start). See `../drive-clients-inventory.md`.

`client-internals/decomp-findings-2026-10-24-b/05-open-items.md`

Open items after the 2.00 / 2.17.1 / 2.22 / 2.31 decompile pass

Settled by this pass (for reference)

- **`PLO_HIDENPCS` (151)** removes the NPCs with ids 1 to 999 of the active level. Id 0 is excluded. (Earlier note: "probably ids below 1000".)
- **`PLO_SIGNATURE` (25)** value 1 makes the client quit. Value 0 makes it quit unless the connected host string is the loopback address. Any other value is ignored. Identical in 2.17.1, 2.22 and 2.31, and the id is handled from 2.00 on. The value 73 that public servers send at login ("more than eight players") is in the ignored group, so it never triggers the quit rule.
- **Tamper check:** how the 10 bytes are built (`01-tamper-check-id39.md`), and why 2.31 repeats it every 3.2 s (`03-periodic-ids-2.31.md`).
- **2.00 login** and its allow-list requirement (`04-v2.00-client.md`).
- **`PLI_ENTERLEVEL` (46)** (lab, batch b2): `GCHAR 1`, sent on every level entry, just before the props message with the cell prop (43/44) and wrapped coordinate. 2.31 does not re-send the tamper check at a crossing. **Batch c:** also sent on doors, edge links and server warps (2.31); 2.22 never sends it. Only a script-initiated warp is untested (`08-lab-findings-levelchange-24-178.md`).
- **`PLO_BOARDMODIFY2` (186) trailer** (lab, batch b2, synthetic frames): bytes after the tiles are ignored; a second record there is not applied; one byte short of `6 + 2*w*h` is rejected silently; a block past column 63 is clipped. See `07-lab-findings-46-186-200.md`.

Still open

Item	State	What would settle it
Client `PLI_SHOWIMGPLAYER` (24), 1-byte payload value 9 (2.31 always; 2.22 only while another player is in the level; about every 5.1 s)	**Partly settled by lab batch c:** the stock server relays any payload unchanged as 32 and nothing reacts (injected payloads, synthetic); 2.22 sends it too in a two-player level. The 5.1 s cadence fits the 5000 ms timer armed in the 2.31 main-form timer (inference, not traced to the end). The 9 is a constant in the sender and is not tied to anything else. Meaning unknown.	Trace the sender's caller chain; a real client's reaction to a 1-byte or emote-form 32 with real image files; the exact 2.22 send condition
`PLO_SERVERWARP` (178) field meanings	**Settled by lab batch c (2.31, synthetic 178 frames plus one real chain):** field 0 unused, field 1 = server name (not needed to connect), field 2 = host, field 3 = port; the client closes the connection and reconnects with a fresh login, no listserver step; fewer than 4 fields is ignored. Still open: whether field 1 is used elsewhere; 2.22; host names, port range, extra fields; the text of a real service (the lab text came from the open-source listserver reimplementation).	A capture of a real serverwarp answer
`PLO_SETACTIVELEVEL` (156), extra step in 2.31	2.31 adds a follow-up step (a call with all-zero arguments, then another call) that 2.17.1 and 2.22 lack. Purpose not resolved. No wire change.	A longer read of the 2.31 handler
Client to server ids 36, 42 and 45 in 2.22	Not matched to a call site. This pass scanned 2.0x only, not 2.17/2.22/2.31, for client senders. In 2.0x, 36 is sent from 2.01 on; 42 and 45 never appear in any 2.0x or 2.1 test build. 45 is the reply to `PLO_HASPROCESSRUNNING` (183) per the 2.22 and 2.31 receive side.	The same send-site scan over the 2.17.1, 2.22 and 2.31 decompiles
2.00 listserver login frames	**Partly observed.** Client side seen: one plain zlib frame `GCHAR 0` + `newmain`, no account or password, no second listserver connection; the list host is DES-hidden and was redirected by name. The reply 2.00 accepted was **synthetic** (forwarder-injected 2.17-style request, lab listserver reply). A real 2.00-era listserver reply is unobserved (`07-lab-findings-46-186-200.md`).	A capture of a real 2.00-era listserver reply, or a decompile read of what 2.00 sends in later list frames
2.31 window-title blacklist (cheat-tool scan inside the 50 ms timer)	Deliberately not decoded. What the client does on a hit is not traced, and the scan result was not found feeding the tamper words.	Not needed unless a server is seen to enforce the tamper check
Per-build handled-id sets for 2.01 to 2.04 and the 2.1 test builds	**Settled by batch c** (decompiled; dispatcher rebuilt; 20 builds): see `../decomp-findings-2026-10-24-c/04-plo-server-to-client.md`. Field layouts of the long handlers are still not extracted.	Per-handler manual reading
`PLO_NPCACTION` (26) in 2.00 to 2.03	Batch c read the start of it: `GINT3` npc id first, acts only when the player's level is the active level. The rest of the payload is not decoded (`../decomp-findings-2026-10-24-c/04-plo-server-to-client.md`).	A longer read of the 2.00 to 2.03 handler
2.1 to 2.16 and 2.3	No binaries anywhere (`../drive-clients-inventory.md`).	New binaries

`client-internals/decomp-findings-2026-10-24-b/06-crosscheck.md`

Cross-check against the repo (after PRs #30 and #31)

New facts (not in the repo before this pass, or only marked unverified)

- Tamper-check derivation: two chained CRC-32 words over the client's own memory, 5-byte non-GINT5 encoding with unmasked middle bytes. All six lab values reproduced. Words per build: `version-profiles/tampercheck-words.tsv`.
- 2.31 timer: a counter modulo 64 on a 50 ms timer gives 3.2 s; phases 1, 10, 20, 30, 31 (probe, CRC A, window scan, CRC B, send). The 3.2 s is derived, not only observed.
- Payload layouts for every server id above 48 plus 25, 39, 41, 47, 156, 165, 166, 168 to 189 that the repo had as names or blank cells: `version-profiles/plo-layouts-217-231.tsv`. Notable: 49 (x and y halved, z unused, gmap cell, then level), 75 (length-prefixed fields), 150, 171 and 172 (comma text, at least 4 fields), 178 (at least 4 comma fields, 1 to 3 used), 181, 182 and 183 reply ids (client 44 and 45), 184 (only `.png`), 185 (height cell formula), 186 (GSHORT tiles).
- `PLO_SIGNATURE` (25) semantics: value 1 quits; value 0 quits unless the host is the loopback address.
- Login tokens for the whole 2.0x family are confirmed from the binaries and all match the repo (`version-profiles/version-tokens-by-build.tsv`).
- Client to server id sets per 2.0x build (`version-profiles/pli-sent-ids-by-build-20x.tsv`): 2.00 lacks 35 to 38; 35 and 36 arrive in 2.01, 37 in 2.02 rev2, 38 in 2.03.
- 2.31 `PLO_LEVELNAME` (6) now also compares against the current level; 68 clears the stored file size; 156 has an extra step.
- 2.00: login string shape with plain account and password, no `arrays2.dat`, and runs under Wine once the data files are supplied.
- `PLO_NPCACTION` (26) is handled by 2.00 to 2.03 only.

Confirmations

- Handled-id set sizes (80 for 2.22, 81 for 2.31; 2.31 drops 44, adds 84 and 189) match `version-profiles/2.22-2.31.md` and `protocol/client-versions.md`.
- The 2.1 test builds send the 2.05 token, as `client-internals/version-id-map.md` says.
- The `protocol/packet-ids.md` names for 168 to 189 match the 2.22 and 2.31 behaviour (ghost, minimap, kingdoms, new world mode, serverwarp, RPG window, status list, process list, screenshot, board heights, board modify 2).
- 2.31 dropping 44 is consistent with the `protocol/packet-ids.md` note "client 5.07+ ignores it".
- The 10-raw-bytes description of PLI 39 (PR #30) is explained by the unmasked encoding.

Corrections made in this change

- `PLO_HIDENPCS` (151): was "probably removes NPCs with id < 1000"; decoded as ids 1 to 999 of the active level.
- `version-profiles/version-coverage-2026-10-24.md` and `client-internals/version-profiles/141-217.toml` say a lab needs an allow-list "patch" for 2.00. A plain allow-list config entry is enough (`04-v2.00-client.md`). The report now carries a status update; the TOML note is unchanged (it only says stock GServer-v2 treats the token as unknown, which is still true).
- `version-profiles/2.22-2.31.md` lists client to server ids 36, 42 and 45 as unmatched; that is still true (this pass did not scan 2.17/2.22/2.31 send sites). Listed in `05-open-items.md`.

Corrected by batch c

- Id 6 (`PLI_TOALL`) is a real sender in every 2.0x build, not a possible scan artefact (`04-v2.00-client.md` corrected).
- 2.04 only drops id 26; id 25 is handled in every build. Nothing is swapped.
- The 2.00 builds (r0 to r3) share handler shapes and one token. Batch c supersedes the 2.00 and 2.05 statements here where they differ: `../decomp-findings-2026-10-24-c/10-open-questions-and-corrections.md`.

`client-internals/decomp-findings-2026-10-24-b/07-lab-findings-46-186-200.md`

Lab findings: PLI 46, PLO 186 trailer, 2.00 listserver handshake (batch b2, 2026-10-24)

Derived facts only. Lab-only runs (loopback network, private display, placeholder accounts). No public server was contacted. Tables are under `version-profiles/`:

Table	Content
`lab-enterlevel-231.tsv`	the seven PLI 46 sends of one 2.31 run, with the props that follow
`lab-boardmodify2-186-trailer.tsv`	the six `PLO_BOARDMODIFY2` test cases on 2.22 and 2.31
`lab-200-listserver-handshake.tsv`	the 2.00 listserver and gserver handshake, step by step, with provenance

How to read the provenance

- **Observed** means a real client talking to the stock lab servers, with nothing in between that changed the traffic.
- **Synthetic** means the lab needed a lab-only helper in the path. Two results rest on one:
  - The **2.00 listserver reply** was not sent by a real listserver. The lab listserver only answers a 2.17-style request, so a forwarder added such a request upstream after the client's frame. What 2.00 accepts is confirmed; what a real 2.00-era listserver sends is **unobserved**.
  - The **id 186 frames** were injected by a re-encoding proxy. Stock GServer-v2 defines the id only as an unknown packet and never sends it, and its scripting has no raw-packet send. So the client reactions below are real, but no server of any kind has been seen sending 186.

PLI_ENTERLEVEL (46), real 2.31 client [observed]

- Payload is always one byte, `GCHAR 1`. It is **not** a login one-shot: it is sent on every level entry. One run produced 7 sends: login plus six gmap edge crossings (east, west, east, west, south, north). Table: `version-profiles/lab-enterlevel-231.tsv`.
- Order inside the client's frame: 46 first, then the props message that carries the cell prop (43 for a horizontal crossing, 44 for a vertical one) and the wrapped local coordinate. At login it is the first message of the second client frame, before PLI 35 (adjacent levels) and PLI 44.
- It is client-initiated and does not wait for the server. The server's answer to a crossing is the same as for 2.22 (level name, mod time, board, gmap name, 174, 10, 42, 156).
- **2.31 does not re-send the tamper check (id 39) at a crossing.** Its 39 stays on the 3.2 s timer, and 24 stays about every 5.1 s. 2.22 is the build that re-sends 39 at a crossing.
- Not tested: a level change that is not a gmap edge crossing (door, warp). The "every level entry" wording rests on the gmap crossings plus login.
- Not sent by 2.17 or 2.22 (client id sets in `version-profiles/2.22-2.31.md`).

- Emulation rule for a 2.31 profile: send PLI 46 = `GCHAR 1` at login and on each level change, just before the props message that changes the cell prop.

PLO_BOARDMODIFY2 (186), 2.22 and 2.31 [client reaction observed, frames synthetic]

Layout: `GCHAR mapX, mapY, x, y, w, h`, then `w*h` GSHORT tiles. Results, identical on 2.22 and 2.31 (`version-profiles/lab-boardmodify2-186-trailer.tsv`): - The handler's length check is "at least `6 + 2*w*h`". Bytes after the tiles are **unread padding**. - A trailer of arbitrary bytes has no effect and does not disconnect. - A complete second record placed in the trailer is **not applied**. - One byte short of `6 + 2*w*h` is rejected silently: nothing is drawn and the client stays connected. A payload that is much shorter behaves the same. - A block that runs past board column 63 is **clipped**, not rejected: only the columns inside the board are drawn. - Not tested: `mapX`/`mapY` other than the current cell, or out of range.

2.00 listserver handshake [client side observed, list reply synthetic]

Setup: the 2.00 exe run beside a copy of the 2.22 data folder, plus `objects.dat`, which 2.00 needs and the 2.22 folder does not contain (it came from a NewMain 26 folder). The hidden list host resolved through a per-namespace hosts file, so the exe was not patched. Table: `version-profiles/lab-200-listserver-handshake.tsv`. - **Observed:** the client sends **one** plain zlib frame: `GCHAR 0` + `newmain` + LF, then half-closes. No account, password, token or key. This differs from 2.17, whose first frame also carries `GCHAR 1 account password`. - **Synthetic:** the list reply (two plain zlib frames: id 0 list; ids 2, 3, 5) came from the lab listserver after a forwarder injected a 2.17-style request. 2.00 parsed and drew it, so the 2.17-style reply format is **accepted** by 2.00. What a real 2.00-era listserver replies to the bare `newmain` request is **still unobserved**. - **Observed:** pressing Connect opens no second listserver connection. The client goes straight to the gserver with type 0, key byte, `GNW24011`, `GCHAR len + account`, `GCHAR len + password`, then gen-3 framing (junk-byte rule, 0 mismatches). The lab server's allowed-versions entry for `GNW24011` was enough; no patch. - **Observed:** first client ids after login: 34, 23, 18 (twice, the gr.x and gr.y flags), 2. None of 37, 39, 46, 24 or 44 appear. The server's 168, 190, 194, 47 and 182 are ignored without a disconnect. - **Data-file caveat:** the 2.22 data folder alone is not enough for 2.00; `objects.dat` is missing there. Without it 2.00 opens a blank grey window after Connect and never dials (`../v2.00-connect-path.md`).

`client-internals/decomp-findings-2026-10-24-b/08-lab-findings-levelchange-24-178.md`

Lab findings: PLI 46 on level changes, PLI 24, PLO 178 (batch c, 2026-10-24)

Derived facts only. Lab-only runs: real 2.22 and 2.31 clients against the stock open-source game server and listserver, on a loopback-only lab network, with placeholder accounts. No public server was contacted. Follows `07-lab-findings-46-186-200.md` (batch b2). Tables are under `version-profiles/`:

Table	Content
`lab-levelchange-46-222-231.tsv`	18 level changes (door or edge link, server-side warp) on 2.31 and 2.22: whether PLI 46 and PLI 39 went out, and where
`lab-showimg24-relay.tsv`	PLI 24 payloads (the real 9 and eight injected ones) and how the server relayed them
`lab-serverwarp178.tsv`	PLO 178 tests: the real chain and five injected field-order cases

How to read the provenance

- **Observed**: a real client and a real lab server, nothing altered.
- **Synthetic**: a lab-only re-encoding proxy rewrote or injected a message. The client's or server's reaction is what was observed. It says nothing about what a real public service sends.
- **Unresolved**: not answered by this run.
- In the 178 table, `<name>`, `<lab host>` and `<port>` are placeholders for the lab server's name, a loopback address and its port.

A. PLI_ENTERLEVEL (46) on doors, edge links and server warps

Table: `version-profiles/lab-levelchange-46-222-231.tsv`. 2.31: 2 sessions, 10 transitions. 2.22: 1 session, 8 transitions. Both mix first visits and cached visits.

2.31 [observed] - Sends `46 = GCHAR 1` on **every** level entry, not only on a gmap crossing: a level link (a door or an edge strip) and a server-side warp (the chat command `warpto`). The payload never changes. - The position depends on who starts the change and on whether the client has the level cached: - **Level link, level not cached.** The client sends `PLI_LEVELWARP` (0) with x, y and the level name. The server answers (ids 14, 6, 100, 101, 39, 1, 0, 174, 10, 42, 156). Then the client sends one frame: `46`, then the props. So 46 comes **after** the server's answer. - **Level link, level cached.** The client sends `PLI_LEVELWARPMOD` (30), which adds the level's mod time before x, y and the name. The 46 is in the **same frame** as the 30, right after it and before the props, so before the server's answer. That answer has no id 14, and no 100/101 when the board is cached too. - **Server-side warp.** The server sends 14, 6 and the level data. The client then replies with a frame whose first message is 46. The props follow in the same frame only if the position changed; a warp to the same x, y gives `46` alone. - **Gmap crossing** (batch b2): 46, then the props, in one frame before the server's answer. - One rule covers all four: 46 goes out when the client has the level and is about to send the first props of it. At login it is in the login frame after the `PLI 35` messages and before `PLI 44` and the props. - **2.31 does not re-send the tamper check (39) on a door, a link or a warp.** Its 39 stays on the 3.2 s timer.

2.22 [observed] - **Never sends 46**: 0 of 8 transitions, and not at login. - It does send the tamper check (39, the same 10 bytes) in the same frame as every **client-initiated** level-link request (`0` or `30`, then `39`), as it does at a gmap crossing. It does not send 39 after a server-side warp.

Not done / unresolved - A level change started by a level NPC script (`setlevel2`). The lab's level scripts never ran, so there is no capture of a script warp. Whether it looks like a link on the wire is unresolved. - Separate lab observation, not investigated: an NPC whose image was a large sprite sheet made the 2.31 client stop sending anything (no heartbeat) for over a minute.

B. PLI_SHOWIMGPLAYER (24)

Table: `version-profiles/lab-showimg24-relay.tsv`.

What the real clients send [observed] - **2.31** sends `24 = GCHAR 9` in its first frame after login (after 46 and 44, with the props) and again every 5.11 to 5.17 s (median 5.13 s; one gap of 5.5 s), idle or walking, in every level used. 48 messages in two solo sessions. The value never changed. A level change neither triggers nor resets it. - **2.22 also sends `24 = 9`**, at login and then every 5.1 s, **but only while another player is in the same level**: 0 messages in 3 solo sessions of 70 to 100 s each. Earlier notes calling 24 "2.31 only" are wrong for 2.22 with two players. The exact condition (another player present, their props received, or the same level) was not isolated. **Unresolved.**

Server reaction [observed in the source and on the wire] - The stock server relays 24 unchanged as `PLO_SHOWIMGPLAYER` (32) = `GSHORT sender id` + the client's payload, to every other player in the level area (not back to the sender). Both real clients were seen receiving the other's 9 every 5 s.

Injected payloads [synthetic] - A lab-only proxy replaced the payload of each client 24 with `GCHAR 0`, `1`, `5`, `14`, `58`, `95`, the six bytes `xx.png`, and an empty payload. The server relayed each one verbatim as 32. No disconnect, no invalid-packet entry in the server log, no error. Neither 2.31 nor 2.22 showed any visible reaction: screenshots matched the baseline apart from an animated status bar. The lab had no player sprites or emoticon files, so a drawn image could not have shown anyway.

Conclusion - The stock server treats 24 as an opaque payload to relay. The 2.31 form is a single byte 9, unlike the four-field emote form (index, x, y, name length and name) that the 2.05 notes describe. What the 9 means, and what a receiving client does with a one-byte payload, is **unresolved**. An emote-form payload with a real image file in the receiver's folder was not tried. (Later read: 2.05 does not use a four-field form; it sends the same tagged image list, see `../id24-205.md`.)

C. PLO_SERVERWARP (178)

Table: `version-profiles/lab-serverwarp178.tsv`. Real 2.31 client only.

The chain [request synthetic, rest real] - A client `PLI_SERVERWARP` (41) carrying the target server's name was injected on the 2.31 connection. The real game server passed the name to the real lab listserver, which answered. The game server wrapped the answer as 178 for the client. The wire text had four comma fields: `<name>,<name>,<host>,<port>`. - That answer comes from the **open-source listserver reimplementation**. What a real service sends is **unobserved**, although the 2.31 client accepted this one.

What the client does on 178 [observed] - It closes the connection to the current game server (FIN). A few milliseconds later (2 to 3 ms) it opens a TCP connection to the host in field 2 and the port in field 3, and logs in again by itself: login frame type 5, a new random key, its normal version token, the same account and the same stored password. - There is **no new listserver connection** and no server-list screen. The second game server accepted the login and sent its usual login sequence.

Field order [synthetic 178 frames] | Fields sent (field 0 first) | Client reaction | |---|---| | decoy host, name, host, port | connected to the host in field 2 and the port in field 3; no attempt to the decoy. **Field 0 is unused.** | | any, host, port, name | no TCP connection; old connection closed; "The connection couldn't be established!" with Reconnect and Open serverlist | | any, name, port, host (port and host swapped) | same failure | | any, a bogus name, host, port | connected normally. **Field 1 is a name and is not needed to connect.** | | any, name, host (three fields) | ignored: connection stays, nothing happens. Consistent with "at least 4 fields". |

- So on the wire: **field 0 unused, field 1 = server name, field 2 = host, field 3 = port.** This matches the decompile reading in `02-server-to-client-layouts.md`.
- **Unresolved:** whether field 1 is used for anything else (a title text, say). Not tried: 2.22, a host name instead of an IP in field 2, a port above 65535, extra trailing fields. A script-initiated `serverwarp` did not run in the lab (see A), which is why the request was injected.

What to implement (Seihai, 2.31 and 2.22 profiles)

1. **2.31:** send `46 = GCHAR 1` once per level entry, immediately before the first props frame of the new level. In the same frame as the warp request when the level is cached; in the first client frame after the server's level answer (6 or 14) otherwise; at login, after the `PLI 35` messages and before `PLI 44`. Drop the "login one-shot" idea.

2. **2.22:** do not send 46. If the tamper check is enabled, send 39 in the same frame right after each client-initiated `PLI_LEVELWARP` or `PLI_LEVELWARPMOD`, not after a server-side warp.
3. **178:** parse four comma fields, use fields 2 and 3, close the connection, reconnect to that host and port with a fresh login (new key, same credentials). Ignore the message with fewer than four fields. No listserver step.
4. **24 and 32:** treat 32 as `GSHORT id` + opaque bytes. Do not crash on a one-byte or empty payload. 2.31 sends 24 = 9 always; 2.22 was seen sending it only while another player was in the level (condition not isolated).

`client-internals/decomp-findings-2026-10-24-b/README.md`

Decompile findings, batch b (2.00, 2.17.1, 2.22, 2.31 + Drive clients), 2026-10-24

Derived facts only. **See also batch c (`../decomp-findings-2026-10-24-c/`): 20 builds from 2.00 to 2.1 test, NewWorld and NewMain; it supersedes the 2.00 and 2.05 parts of this batch where they differ.** No code, addresses or decompiler names. Lab-only runs (loopback network, private display). Built from decompiles of 2.00, 2.17.1, 2.22 and 2.31, plus the Drive clients set and lab captures.

File	Content
01-tamper-check-id39.md	how the 10 bytes of client PLI 39 are built; per-build words; when sent
02-server-to-client-layouts.md	payloads for server ids above 48 and for 25, 39, 41, 44, 47, 156, 168-189; 2.17.1 vs 2.22 vs 2.31
03-periodic-ids-2.31.md	2.31 timer: PLI 39 every 3.2 s, PLI 24, PLI 46
04-v2.00-client.md	2.00 login, id tables, token allow-list, lab setup
05-open-items.md	what is settled and what is still open
06-crosscheck.md	new facts, confirmations and corrections against the repo
07-lab-findings-46-186-200.md	lab: PLI 46 on every level entry, 186 trailer behaviour, 2.00 listserver handshake (observed vs synthetic)
08-lab-findings-levelchange-24-178.md	lab batch c: PLI 46 on doors, links and server warps (2.31 yes, 2.22 never), PLI 24 relay, PLO 178 field order and reconnect (observed vs synthetic)
../drive-clients-inventory.md	what the Drive clients set contains; Wine status

Tables (all under `version-profiles/`):

File	Content
plo-layouts-217-231.tsv	one row per server id: fields, version gates, confidence
plo-handled-ids-by-build.tsv	handled server-to-client ids per build, 2.00 to 2.31
pli-sent-ids-by-build-20x.tsv	client-to-server ids per 2.0x build
tampercheck-words.tsv	CRC words and lab bytes per build (companion to `version-profiles/tampercheck-values.tsv`)
version-tokens-by-build.tsv	login tokens per build
lab-enterlevel-231.tsv	PLI 46 sends of one 2.31 gmap run
lab-boardmodify2-186-trailer.tsv	`PLO_BOARDMODIFY2` trailer and length test cases (synthetic frames)
lab-200-listserver-handshake.tsv	2.00 listserver and gserver handshake with provenance
lab-levelchange-46-222-231.tsv	PLI 46 and PLI 39 on 18 level changes, 2.31 and 2.22
lab-showimg24-relay.tsv	PLI 24 payloads and the server's relay as PLO 32
lab-serverwarp178.tsv	PLO 178 real chain and field-order tests
drive-clients-exes.tsv	every exe in the Drive clients set (folder, size, PE file version)

client-internals/decomp-findings-2026-10-24-c/01-toolchain-and-coverage.md

01 Toolchain and coverage

Derived facts only. No raw decompile text, addresses or generated function names are in this folder. Raw exports are kept private and are not in any repo. Run date: 2026-10-24 (Asia/Tokyo).

Result

- All 20 requested builds were decompiled with Ghidra 12.2 headless (JDK 25), at most 2 jobs at a time. No job hit the 7200 s limit and no job failed. Per-build table: `coverage_by_build.tsv`.
- Builds: 2.00 (main) and revs 1-3; 2.01 and rev1; 2.02, rev1, rev2; 2.03; 2.04; 2.05, rev1, rev2; 2.1 test2 and test3; NewWorld beta1/beta2; NewMain 24/26.
- 2.05 rev2 and 2.00 main already had decompiles. They were re-run only as a diff baseline (same pipeline, so differences are not tool artefacts).
- Each export holds about 5,480-5,720 functions. Most are VCL/RTL, JPEG library and DirectX wrappers. About 100 are game classes.

Why plain Ghidra output was not enough (Delphi finding)

- Every build is a Delphi 3/4/5-era PE (CODE/DATA/BSS sections, VCL RTTI). Plain auto-analysis missed about 520 functions per build, including the whole server-to-client packet switch and some real senders.
- Fix, in two Ghidra scripts run per build before export:
    1. Create functions at standard Delphi prologue sites that analysis skipped.
    2. Rebuild the packet dispatcher as one function from its jump table plus the byte index table in front of it. The table is indexed by `id` and has 0x9b..0xa9 entries depending on the build.
- Without step 2 the handled-id set per build cannot be read from the decompile at all. With it, the sets in `plo_handled_ids_by_build.tsv` match the 2.05 set already documented in the repo (59 ids), which is the check that the method is sound.
- Lesson for any later pass on another Delphi build: always check that the dispatcher exists as a function and that the jump-table size matches the "last index" in `plo_handled_ids_by_build.tsv`.

IDR (Interactive Delphi Reconstructor) status

- IDR 2.x runs under Wine 10 on a private virtual display, driven by xdotool. It works for every build. The first attempt on 2.1 test2 failed (a stray click opened a form preview) and the retry succeeded.
- 20 of 20 IDC name tables were produced (about 6,590-6,730 names each: RTTI classes, RTL, VCL and the application's own methods). They were substituted into the Ghidra C export. About 27-29k substitutions per build.
- Value: the dispatcher cases, the login function and the senders become identifiable. Method names of the game's own classes (TAdventureClient, TPlayer, TAdventureLevel, ...) are the only game-specific names IDR can recover. Packet handlers themselves are anonymous bodies in one switch, so their names come from gs2lib (`PLO_*` by id) and not from IDR.

Method for the tables

- PLO ids handled: dispatcher jump table minus the shared default arm.
- PLI ids sent: the union of (a) constant-id send calls found in the decompile and (b) an assembly scan for the same call pattern. The union is needed because one 2.03 sender (id 80) is missing from the decompile but

present in the assembly. Id 6 (chat/TOALL) is a real sender in every build. The earlier "may be a false positive" remark in batch b was wrong.
- Handler layouts: heuristic scan of how a handler reads the payload (GCHAR/GINT2/GINT3 reads, rest-of-string copies, length guards). It is exact for simple handlers and incomplete for long ones. `plo_decode_signatures.tsv` records the result and is not authoritative for long handlers.
- Handler change detection: normalized statement shapes (names, big numbers and casts removed), compared with a sequence-similarity ratio. See `handler_shape_changes.tsv`. Small bodies (4-12 statements) change shape from compiler noise, so only large drops in similarity count.

Limits

- The 2.1 test decompiles drop some arguments and calls (the Ghidra decompiler loses some register arguments in that compiler version). Structural facts (ids, field counts, which handler changes) are reliable. Exact argument values are not.
- The raw exports are private. Re-running needs the lab's private tool folder (Ghidra, IDR; not in a repo).

`client-internals/decomp-findings-2026-10-24-c/02-login-token-and-version.md`

02 Login token and version string

All tokens come from the binaries (the 8 characters are built one byte at a time from immediates, so a text search misses them). Table: `version_tokens.tsv`.

Token per build

build	token
2.00, rev1, rev2, rev3	GNW24011
2.01, 2.01 rev1	GNW23021
2.02, 2.02 rev1	GNW15041
2.02 rev2	GNW20041
2.03	GNW06051
2.04	GNW24061
2.05, rev1, rev2	GNW03081
2.1 test2, test3	GNW03081 (same as 2.05)
NewWorld beta1, beta2	G2N15041 (different prefix, same digits as 2.02)
NewMain 24, 26	GNW08011

Reading the digits [inferred]

- The shape is prefix + dd + mm + y (day, month, last digit of the year). Read that way, 2.00 is 24 Jan 2001, 2.01 is 23 Feb, 2.02 is 15 Apr, 2.02 rev2 is 20 Apr, 2.03 is 6 May, 2.04 is 24 Jun, 2.05 is 3 Aug. All sit inside the year-2001 changelog (`newfeatures2001.txt`) and rise in order, so this is very likely a build date.
- NewMain (08011 = 8 Jan) is older than 2.00 by this reading, which fits a branch that was frozen before the 2.00 release.
- NewWorld beta carries "G2N" and the 2.02 date, so it branched from the 2.02 code line. Its class list agrees (see 09).

- Revisions within one version do not change the token (2.00 rev1-3, 2.01 rev1, 2.02 rev1, 2.05 rev1/rev2) except 2.02 rev2, the only rev that changes both token and login behaviour. A server allow-list therefore needs one entry per version, plus an extra entry for 2.02 rev2.

Login string shape (unchanged across all 20 builds)

- Type 0 packet: one random key byte (GCHAR of key & 0x7F), the 8-character token, GCHAR(len) + account, GCHAR(len) + password. Account and password are plain.
- Straight after it, 2.02 rev2 and every later 2.x build (also NewWorld) send PLI 37 with the language string. 2.00 to 2.02 rev1 do not. This matches the vendor changelog for 2.02 rev1 ("the server knows the language of the client", planned) and the actual send shows up in rev2.

What changed in the login function between revisions

- 2.02 rev1 to rev2: only the token and the extra id-37 send. No other login changes.
- 2.00 rev3: a change in a player-creation routine and the tab-control handler, not in login.
- 2.05 rev1/rev2: changes only in main-form creation and mouse handling.

Server consequence

A lab or Seihai server that wants all of these clients must list all token strings above. Tokens never contain a version number as such, only the date code, so the server cannot derive a build's capabilities from the token alone: see `04` and `05` for what each build can send/handle.

`client-internals/decomp-findings-2026-10-24-c/03-framing-and-encryption.md`

03 Framing and encryption (2.00 - 2.1, NewWorld, NewMain)

What is the same everywhere

- Message = GCHAR(id) + payload + newline; id = byte - 0x20. GCHAR = byte - 0x20. GINT2 = (b0-0x20)*0x80 + (b1-0x20). GINT3 adds a third byte with multipliers 0x4000, 0x80, 1. These read patterns are used by handlers in all 20 builds with the same constants.
- Frames on the wire are a 2-byte big-endian length followed by a zlib body; the client's outgoing messages are collected and flushed as one zlib block. The shared Borland LCG constant occurs in every build (it is also used by the Delphi random-number routine, so by itself it does not prove the junk-byte step; the junk-byte description in batch b for 2.00 still applies). No build changes the flow in a way that needs a new decoder.
- PLO 100 (RAWDATA, GINT3 length) is handled in every build from 2.00 on, including NewWorld and NewMain. Ids 101/102 never appear as handled ids in any build (they travel behind 100).

Where zlib comes from (the one real difference)

builds	zlib source
2.00 rev1-3, 2.01, 2.01 rev1	compiled-in Pascal zlib unit (z_stream record visible in names)
2.02, 2.02 rev1, 2.02 rev2	external `zlib.dll` (import, with deflate/uncompress/deflateInit thunks)

builds	zlib source
2.03 and later (2.04, 2.05, 2.1 test, NewWorld, NewMain)	compiled-in Pascal unit again

- The 2.02 family therefore needs zlib.dll beside the exe, or a Wine override, to run. All other builds run without it.
- The change is link-level only; the wire format is the same.
- Possible reason [inferred]: a short-lived switch to the DLL during 2.02, then back. No changelog line mentions it.

Encryption generations

- All 20 builds send type 0 login with the 8-character token in the first 8 bytes. A server that treats "first 8 bytes is a known token" as generation 2 (zlib, no cipher) serves them. The 2.05 behaviour (insert one junk byte per message after the first) is described in batch b for 2.00 and 2.05 and was not seen to change in 2.01-2.04.
- No build before 2.1 has bz2 or a per-frame compression-type byte. The 2.1 test builds also do not (they behave like 2.05 here).
- NewWorld beta1/2 and NewMain 24/26 use the same transport and the same login shape as 2.00-2.05.

Not verified here

- Whether the gen-3 junk-byte step is byte-for-byte identical in 2.01-2.04 (the call structure matches 2.00 and 2.05, but I did not diff the exact insertion index formula per build).

`client-internals/decomp-findings-2026-10-24-c/04-plo-server-to-client.md`

04 Server-to-client ids (PLO) per build

Tables: `plo_handled_ids_by_build.tsv` (sets), `id_changes_between_builds.tsv` (adds/removes), `plo_decode_signatures.tsv` (field layouts, heuristic). Names are gs2lib's.

Set growth

step	added	removed
2.00	53 ids: 0-45, 75, 100, 150, 151, 153, 154, 155	
2.00 rev1-3	none	none
2.01	46 HITOBJECTS, 156 SETACTIVELEVEL	
2.01 rev1	none	
2.02	47 STAFFGUILDS	
2.02 rev1, rev2	none	
2.03	48 TRIGGERACTION, 165 MOVE	
2.04	166 SHOWIMGNPC	26 NPCACTION
2.05, rev1, rev2	168 (UNKNOWN168)	
2.1 test2	49 PLAYERWARP2, 169 UPGRADEURL	
2.1 test3	none	

- Dispatcher bound (last index): 0x9b (2.00-2.00r3), 0x9c (2.01-2.02), 0xa5 (2.03), 0xa6 (2.04), 0xa8 (2.05), 0xa9 (2.1 test). It always grows with the highest handled id.
- Correction to batch b note 04: 2.04 drops 26 only. 25 (SIGNATURE) stays in every build. The earlier wording "swaps 26 for 25" was wrong.

- Id 26 (NPCACTION, absent from the repo's 2.05 table) exists in 2.00-2.03. It reads GINT3 npc id and the rest, and only acts when the player's level is the active one. It has no handler in 2.04 and later, which agrees with the repo's "no jump-table entry for 26 in 2.05".
- Id 168 has an almost empty body (client-side reset only). In the 2.00-2.05 family it is first handled in 2.05 (all three 2.05 builds), consistent with batch b's 2.05 note that the login server sends it.
- 156 SETACTIVELEVEL is handled from 2.01 on. It stores the active-level reference by looking up the raw level name. The handler was reworked (much shorter) in 2.03.

Active-level gating

- Only id 26 is gated on "player's level == active level" in the 2.0x builds, by my scan. For later ids (46-48, 165, 166) the batch-b 2.17+ analysis already lists the gating; I did not re-derive it per 2.0x build.
- Because 156 only appeared in 2.01, a 2.00 client has no active-level concept. Level-addressed packets in 2.00 act on the current level only.

Layouts that are stable and verified by structure

(GINT/GCHAR counts; field meanings come from gs2lib. "len>N" means a minimum payload length guard.) - 3 NPCPROPS, 24 NPCMOVED, 29 NPCDEL, 150 NPCDEL2, 165 MOVE, 166 SHOWIMGNPC: GINT3 npc id first. - 8 OTHERPLPROPS, 11 BOMBADD, 19 ARROWADD, 20 FIRESPY, 21 THROWCARRIED, 32 SHOWIMGPLAYER, 36 EXPLOSION, 37 PRIVATEMESSAGE, 40 HURTPLAYER, 46 HITOBJECTS, 48 TRIGGERACTION: GINT2 player id first, then the rest. - 42 NEWWORLDTIME: GCHAR + GINT3 (length guard >3); present since 2.00 and in every build, including NewMain. - 13 TOALL: GINT2 sender + GCHAR + string. The 2.00 and 2.01 decodes looked different only because the scan was incomplete for the long handler.

Handler behaviour changes

See `06-handler-changes-by-build.md` and `handler_shape_changes.tsv`.

Missing in NewWorld and NewMain

See `08`.

`client-internals/decomp-findings-2026-10-24-c/05-pli-client-to-server.md`

05 Client-to-server ids (PLI) per build

`pli_sent_ids_by_build.tsv`; adds/removes in `id_changes_between_builds.tsv`. Names are from the repo's `packet-ids.md`.

build	ids with a direct send	count
2.00 (and rev1-3)	0-24, 26-30, 32-34, 80-81	35
2.01, 2.01 rev1, 2.02, 2.02 rev1	+ 35, 36	37
2.02 rev2	+ 37	38
2.03 to 2.1 test3	+ 38	39

- 35 = ADJACENTLEVEL (matches the 2.01 changelog "levels are preloaded when you walk on the map"). The payload is a string taken from the neighbouring level object, completed with a second computed string [layout not decoded].
- 36 = HITOBJECTS (2.01; script variable `hitobjects` appears in the same build). [inferred] It is the TCP twin of UDP id 25: the same function first sends id 25 through the peer-channel wrapper, then repeats the

message over TCP under 36 when a flag says the server needs it. That agrees with the routing rule in `udp-p2p.md`.
- 37 = LANGUAGE (2.02 rev2; also sent once right after login).
- 38 = TRIGGERACTION (2.03; twin of UDP id 26 in the same way as 36/25 [inferred]). Matches the 2.03 changelog.
- 6 = TOALL (chat) is a real sender in every build. Chat text is clipped to 200 characters before sending. The batch-b remark that 6 could be a scan artefact is withdrawn.
- 80/81 (profile get/set) are present from 2.00 on.
- No direct constant send exists for 25, 31, or any id from 39 up in any 2.0x or 2.1-test build. So the tamper check (39) and the later periodic ids are not part of this family. 42 and 45 never appear.
- No PLI id was removed anywhere in the 2.00 to 2.1 line.

NewWorld / NewMain

- NewWorld beta1/2: 34 ids. They have 35-38 but lack 7 (HORSEADD), 11 (THROWCARRIED), 15 (BADDYPROPS), 16 (BADDYHURT) and 17 (BADDYADD): horses and baddies are removed from the game in NewWorld.
- NewMain 24/26: 35 ids, same as 2.00 (0-24, 26-30, 32-34, 80-81). They lack 35-38, in line with their older token.

`client-internals/decomp-findings-2026-10-24-c/06-handler-changes-by-build.md`

06 Handler behaviour changes, 2.00 -> 2.1

Basis: normalized-shape comparison of every handled id between consecutive builds (`handler_shape_changes.tsv`, similarity below 0.93 only), checked against the vendor changelog. A shape change means different statements, not necessarily different wire behaviour. Small handlers (4-12 statements) shift shape from compiler noise; only large drops are listed below.

Revisions of the same version

- 2.00 rev1/rev2/rev3, 2.01 rev1, 2.02 rev1/rev2, 2.05 rev1/rev2: identical handler shapes for every id except one: 2.05 rev1 has one extra constant assignment in OTHERPLPROPS (id 8) that 2.05 and 2.05 rev2 do not have.
- The revisions differ in the VCL/RTL layer, not in game logic. 2.00 rev2, 2.02 rev2 and 2.05 rev1 each carry about 60 changed VCL/RTL functions (the multi-monitor wrapper set appears in rev2 builds; rev3 rolls part of it back). The only game-class changes are:
  - 2.00 rev1: one added options-form key handler.
  - 2.00 rev3: player creation and the tab-control handler.
  - 2.02 rev2: login (token + language) and the player-list history click.
  - 2.05 rev1/rev2: main-form creation and mouse-move handling; rev2 also a level-creation routine and server-list form creation.
- So for protocol work any rev of a version can stand in for the others, except 2.02 rev2 (different token, extra id 37).

2.00 -> 2.01

Biggest handler rewrites in the family (gs2lib names): - LEVELBOARD (0) and PLAYERWARP (14), WARPFAILED (15): the client no longer does the "wait for level, then request level" steps inside the handler. This is the code side of "levels are preloaded when you walk on the map" in the changelog. - LEVELNAME (6): shortened. - BADDYHURT (27): a distance/direction-based calculation before the effect is removed. - FREEZEPLAYER2 (154) / UNFREEZEPLAYER (155): go through a shared helper with a flag, instead of setting the field directly. 155 shrinks to one short step. - HASNPCSERVER (44): stores a different constant than in 2.00 (value 11 instead of 1). - New handlers: HITOBJECTS (46), SETACTIVELEVEL (156).

2.01 -> 2.02

- Only SAY2 (153) and PLAYERWARP (14) change shape. 47 STAFFGUILDS is new (list stored as comma text).
- The big 2.02 features (hats via gani attributes, signs with images, string parsing, free-server restrictions) are client-side script/UI and do not add any packet id other than 47.

2.02 -> 2.03

- NPCMOVED (24) grows (31 to 48 statements) and TOALL (13) changes slightly. That is the NPC `move` command and the pushed/pulled actions.
- SETACTIVELEVEL (156) shrinks to a single step.
- New: TRIGGERACTION (48), MOVE (165).

2.03 -> 2.04

- NPCMOVED grows again (48 to 55). MOVE (165) shrinks (17 to 9). SHOWIMGNPC (166) is new: matches "showimg works server-side, layer 4 works too".
- 26 NPCACTION is removed.
- Many other ids show only compiler-level churn.

2.04 -> 2.05 (all revs)

- SHOWIMGNPC (166) loses about 8 statements. UDP receive wrapper loses a bit-test step (purpose not resolved).
- New: id 168 (empty-ish body).
- Everything else is churn-level.

2.05 rev2 -> 2.1 test2

- Large churn in almost every handler. Treat as weak evidence: the 2.1 compiler output is lossy, and the new imaging engine (see `09`) touches many calls. Reliable facts are the id sets only (new 49, 169).
- PLAYERWARP2 (49): x2/y2 half-tile coordinates, an ignored z, two gmap cell bytes and the level name, same structure as the 2.17-2.31 layout in batch b. Tile positions are halved on receive.
- UPGRADEURL (169): raw text stored for the reconnect/upgrade button.

2.1 test2 -> test3

- LEVELBOARD (0), LEVELNAME (6) and PLAYERWARP (14) gain "is the incoming level the one already loaded" checks, so the client avoids reloading the same level. Same direction as in 2.31 (batch b noted that LEVELNAME compares with the current level).
- SAY2 (153) gains one extra call. The change to 31 is a constant-type difference only.
- No id added or removed.

`client-internals/decomp-findings-2026-10-24-c/07-udp-channel.md`

07 UDP peer channel in 2.0x

Compare with `Graal_analysis_draft/protocol/udp-p2p.md` (2.05 rev2).

- Every one of the 20 builds imports the same socket calls (socket, bind, sendto, recvfrom, ioctlsocket, getsockname, inet_addr, gethostbyname). The UDP channel therefore exists from 2.00 on, in NewWorld and NewMain too.

- Per build there is one send wrapper and one receive wrapper. The send wrapper has the same shape (about 40 normalized statements) in every build. The receive wrapper has the same shape (about 82 statements) from 2.00 to 2.04 and NewMain, shrinks to 74 in 2.05 and NewWorld beta1, and to 72 in 2.1 test2/test3 and NewWorld beta2.

- The 2.05 change is a removed bit-test on a length value (alignment or sanity step, purpose unresolved) and a field offset; nothing about the datagram layout changes. Datagram shape in 2.00-2.04 is still seq byte + XORed payload [structure confirmed on 2.00-2.04 by the same call structure, not by per-id decode]. #### UDP ids sent per build (constant-id call sites of the peer-send wrapper) | build | ids found | |---|---| | 2.00 (and revs) | 10, 11, 12, 13, 14, 16, 17, 18, 19, 20 | | 2.01 | + 25 (hit objects) | | 2.03 | + 26 (trigger action) | | 2.04 | 11, 16, 17, 19, 20, 25, 26 found (other call sites go through a changed wrapper form; not recovered) | | 2.05 | the 2.05 rev2 set from `udp-p2p.md`; the scan found only the movement sender (id 0) |

- These were partial in the first scan: ids 0 (movement), 15 (showimg) and 22 (chat) use a broadcast helper (prepend GCHAR id, send to every peer with a UDP port) rather than a naked constant at sendto. Manual read recovered them for 2.00 and 2.05; see `version-profiles/udp-ids-200-204.tsv`. Receive switch cases are wire bytes 0x20 / 0x2f / 0x36.

- The additions match the TCP ids: UDP 25/26 appear in the same builds as PLI 36 (2.01) and PLI 38 (2.03), which supports the "UDP + TCP twin" reading in `05`.

- So the 2.00 client has movement, showimg, chat, bomb, arrow, firespy, explosion, horse, throw and item UDP messages (ids 0, 10-20, 22); hit objects arrive in 2.01 and trigger action in 2.03.

- A 2.00 client has no hit-objects message on either channel.

client-internals/decomp-findings-2026-10-24-c/08-test-builds-newworld-newmain.md

08 The 2.1 test builds, NewWorld betas, NewMain

2.1 test2 and test3

- Same login token as 2.05 (GNW03081), same login shape, same PLI set as 2.05 (39 ids, 0-24, 26-30, 32-38, 80-81). No new client-to-server id.
- Two new server-to-client ids: 49 PLAYERWARP2 and 169 UPGRADEURL (61 ids handled, bound 0xa9). Same two ids appear in NewWorld beta2 and are in the 2.17-2.31 layouts documented in batch b, so these builds are where they started.
- test2 to test3: same id sets. Differences are the "skip reloading the already-loaded level" checks in LEVELBOARD, LEVELNAME and PLAYERWARP, plus one extra call in SAY2. Vendor changelog adds snapshot-level lighting fixes and smilies in PMs (`levels/smilies/smiliesdef.txt`, lines `code,filename`). Those are client-only.
- The exe is about 140 KB smaller than 2.05 rev2 despite more functions, because of the rewritten image engine (the changelog says the image handling no longer uses Windows GDI).
- Editor classes changed: the person/NPC/show-code windows were merged into a tabbed editor (class list in `classes_by_build.tsv`).
- Script additions (strings): `ascii`, `defnpc` (predefined npcs from `defnpc*.txt`), `playeronline`, `startinfullscreen`, `tiles` (test2); `groundheights`, `waterheight`, `playerhp`, `playermaxhp`, `smiliesdef.txt` (test3). Also see `script_vocab_deltas.tsv`.

NewWorld beta1 / beta2

- Token G2N15041. Branch point: the 2.02 code line (token digits, class list with the old person-editor windows, same PLO bound as 2.05 for beta1).
- Beta1 server-to-client set: 53 ids. Compared with 2.05 rev2 it lacks 2 (BADDYPROPS), 4 (LEVELCHEST), 17/18 (HORSEADD/DEL), 154/155 (freeze/unfreeze), 49 and 169, and (like 2.04+) 26; it still has 156, 165, 166, 168.
- Beta2 adds 49 and 169 (55 ids) and switches to the 2.1 editor classes. In other words beta2 follows the 2.1 test branch; beta1 follows 2.05.
- Client-to-server: 34 ids. Same as 2.1 test but without 7 (HORSEADD), 11 (THROWCARRIED), 15 (BADDYPROPS), 16 (BADDYHURT), 17 (BADDYADD).
- Script vocabulary: new `nw*` time variables (nwyear, nwmonth, nwweek, nwweekday, nwday, nwhour, nwmin, nwtime), `layer`, `arraysnw.dat`, `bodycolors.txt`, `bodynw.png`, `controlnpc.txt`, `selectedsword`, `playerhearts`, `playerfullhearts`, plus string-list helpers (addstring, insertstring, deletestring, removestring, sarraylen, lindexof) in beta2. Dropped: the Graal classic baddy/animal names (blue soldier, gray soldier, dragon, frog, ...) and `arrays.dat`.
- Confirms that the NewWorld "world" is a different content stack (new body images, own array file) on top of the unchanged 2.0x protocol, with the NEWWORLDTIME packet (42) already present in 2.00.

NewMain 24 / 26

- Token GNW08011. PLO set: 51 ids, 0-45, 75, 100, 150, 151, 153 only. That is the 2.00 set without 154, 155. PLI set: 35 ids, the 2.00 set.
- NewMain 24 and 26 have identical handler shapes and an identical login. They differ only in resources and a few VCL calls.

- NewMain pulls in the Delphi DDE classes (TDdeMgr etc.) and keeps the person/NPC/chest/baddy editor windows, consistent with an older, pre-2.00 code line.
- Handler shapes differ from 2.00 more than any 2.0x revision does (many ids below 0.7 similarity), so NewMain is a separate branch, not a rev of 2.00.

`client-internals/decomp-findings-2026-10-24-c/09-features-classes-vocabulary.md`

09 Features, classes, vocabulary

Tables: `classes_by_build.tsv`, `script_vocab_deltas.tsv`. Vendor source: `newfeatures2001.txt` in each version folder (2.01-2.05 and the 2.1 tests). The 2.00 builds and NewWorld/NewMain have no changelog.

What each step adds (vendor text + binary evidence)

step	feature	binary evidence
2.01	levels preloaded when walking on the map; map-reading animation; showimg artefact fixes; chat text from `setplayerprop #c` stays 3 s	PLI 35 and PLO 156 appear; warp/board handlers simplified; script strings bomy_walk/bomy_idle, maps1-3, hitobjects, playerreading, playerdies/hurt
2.01	`hitobjects`	PLI 36, PLO 46, UDP 25
2.02	hats via gani attributes (5 attributes read from the player account), `#i()` images in signs, string parsing (strlen, startswith, indexof, `#e`, `#T`, tokenize/tokenize2/tokenscount/`#t`), `local.` flags are not sent to the server, flags issparringzone/nopkzone, free-server restrictions, HTML window navigation buttons, staff-guild list editable by server owners	PLO 47; new strings for heads/bodies/shields/hats directories, `local.`, webgifs, weblevels, profiles; zlib.dll import (see 03)
2.02 rev1	speed-cheat check fix, history without web bar, better hacking protection, server learns the client's language	server language is sent starting rev2 (PLI 37)
2.03	`triggeraction`, `actionpushed/actionpulled`, `move dx,dy,time,options`	PLO 48 and 165, PLI 38, UDP 26, NPCMOVED handler grows; strings pushed/pulled/vecx/vecy/triggeraction/paramscount/movementfinished/onmapx/onmapy
2.04	`replacegani`, `addtiledef`, `addtiledef2`, `removetiledefs`, `setshape`, `setshape2`, `attachplayertoobj`/`detachplayer` with playerattachid/type and levelorgx/levelorgy, showimg text and server-side showimg, weaponscount/`#w`/`#W`, enable/disableselectweapons, bomy_kick	PLO 166, 26 removed; strings tiledefs, tileobjects, shapetiletypes, playerattached

step	feature	binary evidence
2.05	maps via `setmap` applied at once, window title with server name, script-editor format button, `.nw` text level format (GLEVNW01), array tiletypes for setshape2	PLO 168; strings updateterrain, loadmap, getz/playerz, setbacktile2
2.1 test	new tile art, new start/server-list window (F8), new image engine, `ascii()`, `tiles[x,y]`, predefined-npc editor with `#COMMENTS`/`#EDIT`, smilies in PMs, tabbed editor	PLO 49 and 169; TPageControl/TTabSheet/TMemo added, person/show-code windows removed

Classes

- 186 classes in 2.00 through 2.03. +1 in 2.04 (an object-set class; matches "support for object sets"). +1 in 2.05 (class TADP; purpose not identified). 2.1 test swaps 4 editor classes for 4 tabbed ones (188 total).
- NewWorld beta1 = 186 classes, the same set as the 2.0x line (without the 2.04 and 2.05 additions); beta2 changes the editor windows like 2.1 test. NewMain has 193 (adds DDE classes, chest/baddy editors).

Vocabulary (script/identifier strings, resources excluded)

See `script_vocab_deltas.tsv` for exact lists. Takeaways: - Everything the changelog calls a new script command or variable appears in the binary's string table in the same build, with no extra undocumented identifier in 2.01-2.05 other than `actionplayer`, `lighteffectsenabled` (2.03) and `bomy_*` animation names. - 2.01 drops `openurl` from the string list and 2.02 drops `error`/`z_stream8` (then 2.03 brings them back and drops the zlib.dll strings). These are string-table moves, not feature removals: the 2.02 changelog still describes `openurl`/`openurl2`.

`client-internals/decomp-findings-2026-10-24-c/10-open-questions-and-corrections.md`

10 Open questions and corrections

Corrections to batch b / earlier notes

- Batch b note 04: "2.04 swaps 26 for 25" is wrong. 2.04 only drops 26; 25 is present in all builds.
- Batch b note 04: "id 6 may be a false positive of the scan". Wrong: 6 (TOALL/chat, clipped at 200 characters) is a real sender in all 20 builds.
- Batch b treated 2.00 as one build; the 4 builds (2.00, rev1, rev2, rev3) have identical handler shapes and one token. The only differences are VCL/RTL layer changes and small UI-class edits.
- Repo `packet-ids.md` says PLO 26 NPCACTION has no handler in 2.05: confirmed, and the earlier history is now known: handled and active-level gated in 2.00-2.03, removed in 2.04.

Open

1. ~~Exact field layouts of the long handlers (PLAYERPROPS 9, OTHERPLPROPS 8, NPCPROPS 3, LEVELLINK 1) per build: the layout scan is too crude. Needs per-handler manual reading or a type-aware recompile of the decompile.~~ Done: `version-profiles/plo-playerprops-layouts.tsv`, `plo-otherplprops-layouts.tsv`, `plo-npcprops-layouts.tsv`, `plo-levellink-layouts.tsv` (builds

2.00, 2.04, 2.05 rev 2, 2.17.1, 2.22, 2.31). 2.04 rows are bracketed from the 2.00/2.05 endpoints plus batch-c shape/signature tables (2.04 `.c` not in the shared Drive Graal Decompiles folder).
2. 2.04 to 2.05 UDP receive wrapper: removal of one bit-test step. Purpose not resolved.
3. ~~Per-build UDP id sets for ids 0, 15 and 22 (movement, showimg, chat) for 2.00-2.04: the call forms were not recovered. Receiver-side sets for 2.00-2.04 are also not extracted (a second jump table the dispatcher rebuild did not cover).~~ Done: `version-profiles/udp-ids-200-204.tsv`. Ids 0/15/22 go through a broadcast helper (GCHAR id prepend + send to every peer with a UDP port), so the old constant-id scan missed them. Receive switch cases are wire bytes 0x20 / 0x2f / 0x36. Mid builds without a Drive `.c` are inferred from the 2.00 and 2.05 endpoints plus batch-c wrapper-shape notes.
4. ~~Whether gen-3 junk-byte insertion is byte-for-byte unchanged in 2.01-2.04. The call structure matches; the index formula was not diffed.~~ Done: `version-profiles/gen3-insertion-200-205.tsv`. 2.00 matches the known 2.05 formula (`it * 0x08088405 + k`, insert `((it>>16)&0x7F)+0x20` at `(it&0xFFFF) mod len`). With identical endpoints and batch-c's matching call structure, 2.01-2.04 are unchanged (those `.c` files are not on Drive).
5. The 2.1 test decompiles are lossy; argument-level facts about 49 and the changes to 0, 6, 14 there are structural only.
6. What PLI 36/38's TCP "second copy" is conditioned on (flag identity). Reading is [inferred] from the shared wrapper.
7. Purpose of TADP (new in 2.05) and of PLO 168 (empty client-side reset).
8. Whether the token digits really are a build date. The dates fit the changelog order for 2.01-2.05 but nothing in the binary names the format.
9. Not touched: tamper-check (PLI 39) is absent from this whole family; nothing here helps derive it.

For Seihai

- To support 2.00-2.1: allow-list the tokens in `02`; handle PLO ids per build set in `04`; PLI sets in `05`. A server that serves 2.1 test must be ready for 49 and 169 as server-to-client only; no new client-to-server id exists in 2.1 test.
- Do not send ids 46+ to a 2.00 client (it has no handler) and avoid 47+ for a 2.01 client, 48/165 for 2.02, 166 for 2.03, 168 for 2.04. Unhandled ids go to a silent default in every build, so the practical effect is lost information, not a crash.
- LEVELLINK, NPCPROPS, OTHERPLPROPS and PLAYERPROPS wire envelopes do not change across 2.00-2.31; only the accepted prop-id sets inside the streams grow. See the four `plo-*-layouts.tsv` tables.

`client-internals/decomp-findings-2026-10-24-c/README.md`

Decompile mining, batch c: Graal 2.00 - 2.1 test, NewWorld, NewMain (Ghidra + IDR), 2026-10-24 (Asia/Tokyo)

Derived facts only. No raw decompile text, no addresses, no generated function names, no code. Raw exports are private (outside every repo). Nothing was pushed, no public server was contacted, the lab was not touched.

Scope: 20 Drive builds decompiled with Ghidra 12.2 and named with IDR under Wine. Supersedes the 2.00/2.05 parts of batch b where they conflict (see 10).

Files

file	content
01-toolchain-and-coverage.md	what was decompiled, Delphi/dispatcher fix, IDR, method, limits
02-login-token-and-version.md	tokens per build, date reading, login shape, language packet
03-framing-and-encryption.md	framing, zlib.dll in the 2.02 family, junk-byte status
04-plo-server-to-client.md	handled ids per build, adds/removes, 26 history, layouts
05-pli-client-to-server.md	sent ids per build, meaning of 35-38, id 6
06-handler-changes-by-build.md	behaviour changes 2.00 -> 2.1 and revision differences
07-udp-channel.md	UDP channel in 2.0x, ids per build
08-test-builds-newworld-newmain.md	2.1 test2/test3, NewWorld, NewMain
09-features-classes-vocabulary.md	vendor changelog vs binary evidence, classes, script strings
10-open-questions-and-corrections.md	corrections to earlier notes, open items, Seihai implications (items 1, 3, 4 closed by the version-profiles layout TSVs)
coverage_by_build.tsv	per build: size, function count, IDR, id counts
plo_handled_ids_by_build.tsv	server-to-client ids handled
pli_sent_ids_by_build.tsv	client-to-server ids with a send
id_changes_between_builds.tsv	adds/removes per step
version_tokens.tsv	login token and post-login send
plo_decode_signatures.tsv	heuristic field layouts per id (not authoritative for long handlers)
handler_shape_changes.tsv	per id, per step: similarity of normalized handler body
classes_by_build.tsv	class counts and class changes
script_vocab_deltas.tsv	script/identifier string changes

Top facts

1. Tokens: 2.00 GNW24011, 2.01 GNW23021, 2.02 GNW15041, 2.02 rev2 GNW20041, 2.03 GNW06051, 2.04 GNW24061, 2.05 and 2.1 test GNW03081, NewWorld G2N15041, NewMain GNW08011.
2. PLO sets: 53 (2.00) -> 55 (2.01: +46, +156) -> 56 (2.02: +47) -> 58 (2.03: +48, +165) -> 58 (2.04: +166, -26) -> 59 (2.05: +168) -> 61 (2.1 test: +49, +169).
3. PLI sets: 35 (2.00) -> 37 (2.01: +35, +36) -> 38 (2.02 rev2: +37 language) -> 39 (2.03: +38). No id removed. Id 6 is a real sender.
4. Revisions of one version are identical in game logic. 2.02 rev2 is the exception (new token + language packet).
5. 2.02, rev1 and rev2 import zlib.dll; all other builds carry zlib inside the exe.
6. UDP ids: 2.00 already sends 10-14, 16-20; 25 from 2.01, 26 from 2.03; PLI 36/38 are their TCP twins.
7. 2.1 test2/3: same PLI as 2.05, new PLO 49 PLAYERWARP2 and 169 UPGRADEURL. NewWorld beta1 follows the 2.05 line, beta2 the 2.1 line; NewMain is an older separate branch (2.00 minus freeze).
8. The 2.1 test2/test3 decompiles are lossy (the decompiler drops some arguments and calls). Treat every 2.1 test fact here as structure only (ids, field counts, which handler changed), not argument-level.

`client-internals/decomp-findings-2026-10-24-d/01-sword-damage.md`

Sword damage (victim's client)

Builds read: 2.05 rev 2, 2.17.1, 2.22, 2.31. The rule is the same in all four.

Who computes it

The victim's client. The attacker's swing reaches others only as props (position, direction, sprite); the hit's size comes from the attacker's SWORDPOWER prop as the victim last received it.

The sword-power byte (prop 8)

- Sender: the power is limited to -20..20. Power 0 sends byte 0 (no image). A power of 1..4 with the matching default image (sword1..sword4) sends the power itself, 1..4. Anything else sends power + 30 (10..50) followed by the image name.
- Receiver: bytes below 10 are the power; from 10 the power is the byte minus 30.
- The victim stores the power for each other player (a signed value) and uses it when a swing is judged.

The formula

Half hearts taken = the attacker's sword power, limited to -20..20. In hearts: power / 2. So a power-3 sword takes 1.5 hearts.

Not read: the victim's shield power, item levels, the attacker's AP, the hurt-power field of the hurt packet (that packet carries its own power; see below).

What happens to the hearts

The shared hurt routine takes a power in hearts and a knockback vector. It asks the heart-change routine for minus the power, then puts the player into hurt mode (mode 3, 40 ticks of 50 ms). - The heart-change routine ignores a positive change (heal) when the player's AP is below 40. So a negative sword power heals only at AP 40 or more, and hearts stop at the maximum. - It is also where the swim, editor/pause and death checks live: hearts reaching 0 start the death. - Hurt mode starts regardless of the sign or size of the power, including 0 and a heal that was ignored.

The hurt packet (id 40) and its power

`PLO_HURTPLAYER` carries the attacker, a knockback and a power byte (half hearts). The client limits the power to one heart (2 half hearts) before calling the same hurt routine; the attacker must be within 10 tiles. The 2.05 to 2.31 handlers differ only in small decode churn (the earlier batch c tables). The client's own id 26 send is the mirror image; its power is the attacker's choice, limited the same way.

Open

See 03.

`client-internals/decomp-findings-2026-10-24-d/02-respawn-entry-save.md`

Respawn destination: the level-entry save

The respawn moves the player to the *saved entry point* (level, x, y, direction). This note is about when the client saves it.

The decision, run after every level entry

Inputs: the name of the level just left (empty at login), the new level's name, and the current gmap's level list (a string list; -1 when a name is not in it or there is no gmap). 1. Level left is empty: save. 2. Compute index(new) * index(old). If it is negative: save. So exactly one of the two is in the list and at an index of 1 or more; an index of 0 gives a product of 0, so the first list entry behaves like "not in the list" (the client's quirk). 3. 2.00 to 2.05: otherwise, compare the name groups. Different groups: save. Same group: do not save. 4. 2.17.1, 2.22, 2.31: otherwise do not save (no group test).

Saving stores the new level's name and the player's current x and y, direction as it is.

The name group (2.00 to 2.05)

Take the level name, drop everything up to the last path delimiter, lower-case it, then take the text up to and including the last `§` (byte 0xA7) or `$`. If there is neither, the group is empty, and two empty groups are equal. The extension stays in the name.

Effect: two plain names such as `a.nw` and `b.nw` are always the same group. The test only separates names that use the delimiters.

Which builds have the group test

Read in the code, step by step, for all eleven 2.00 to 2.04 builds: 2.00 and its revisions 1 to 3, 2.01 and rev1, 2.02, rev1 and rev2, 2.03, 2.04. Each has the same decision as 2.05 rev2, in the same order: the empty-name test, the product of the two list indices, then the name-group test, then the save. The name-group routine is the same instruction for instruction in all eleven builds and in 2.05 rev2. (Confirmed, from the machine code.)

One difference: 2.05 returns -1 for both indices when there is no current gmap. In 2.00 to 2.04 the list is taken straight from the client's state with no such test. Whether that list is simply empty when there is no gmap, which would give the same -1, was not traced. (Inferred, not tested.)

2.17.1, 2.22 and 2.31 have no group test. The 2.1 test builds 2 and 3 do not contain the delimiter-set constant, so they are expected to have none; their code was not read. A scan of the build files for that constant finds it in 1.37 to 1.41, 2.00 to 2.05, NewMain 24 and 26 and NewWorld beta 1. Builds before 2.00 were not read. See name-group-constant-by-build.tsv.

After the respawn

The respawn routine moves the player (same-level shortcut or a level change), then calls the same save routine once more, unconditionally. If the warp completed in the same call, this stores the respawn point itself. If an online level change finishes later, the stored point could be the spot where the player died. Not resolved.

Open

See 03.

client-internals/decomp-findings-2026-10-24-d/03-hit-gates-and-open-points.md

Extra gates on a hit, and open points

Gates

Beyond the swing sprite (S 6..10), the reach box, the victim's state (not dead, not in hurt mode, not swimming) and AP not 100, the client's shared hit-judging routine also requires: - the current level has not run the script command `noplayerkilling` (a per-level flag, cleared when a level is created; the client prints the text `nopkzone` when its own attack is refused by it); - the attacker has not already been judged in the current second of the day: the routine keeps a per-attacker last-second stamp and drops a second judgement in the same second. The stamp is set before the guild and box tests; - the attacker's `(tag)` is not in the victim's nickname: the attacker's nickname is searched for its first `(`, then the first `)` after it with at least one character between them, and that bracketed text is searched in the victim's nickname (case-sensitive, anywhere). Nicknames are kept obfuscated in memory; - (swing path) the attacker's STATUS has neither bit 0x02 (hidden) nor bit 0x20, and the attacker is not in the kick animation (2.05 by animation name; 2.17.1 and later by an object id of 3). The hurt packet (40) handler checks the no-kill level flag, a per-player field that must be equal for attacker and victim, the 10-tile range, and caps the power at one heart.

Level flags seen

- `noplayerkilling` (script command) sets the no-kill flag.
- `sparringzone` (NPC script command) sets a second level flag that lets a player in the editor or paused state be hurt; the "can be hurt" check is: not in hurt mode, not dead, not swimming, hearts above 0, and (normal play or that flag).

Open

See open-questions.tsv. Main ones: the per-player field in the hurt-packet path; bit 0x20's meaning; knockback of the sword path; which other callers (bombs, arrows, baddies) use the hurt routine with which power; whether the guild test also applies to the hurt packet; the gmap list's order and the timing of its update against the entry save; the end-of-respawn save; the entry rule in the 2.1 test builds; no lab confirmation.

---

client-internals/decomp-findings-2026-10-24-d/README.md

Decompile mining, batch d: sword damage and the respawn entry save, 2026-10-25 (Asia/Tokyo)

Derived facts only. No raw decompile text, no addresses, no generated function names, no code, no paths. The raw exports stay private. Nothing was pushed to Graal_analysis, no public server was contacted, the lab was not touched. Nothing here was seen on a wire.

Scope: the two rules that Seihai's combat and health/death work had left as placeholders. Read in the 2.05 rev 2, 2.17.1, 2.22 and 2.31 clients, with the 2.1 test builds and all other builds checked only for one constant (see 02).

Files

file	content
01-sword-damage.md	how the victim's client turns a hit into hearts, the sword-power byte, heal rule, hurt mode
02-respawn-entry-save.md	which level entry is saved as the respawn point, the gmap-list test, the name group (2.00 to 2.05)
03-hit-gates-and-open-points.md	extra gates on a hit (no-kill level, guild tag, attacker status, per-second limit), and what is still open
damage-rules.tsv	the damage rules, one row each, with the builds they were read in
hit-gates.tsv	every gate a hit must pass, in the order the client checks, per path
entry-save-rule.tsv	the save-entry decision table
name-group-constant-by-build.tsv	which builds contain the name-group delimiter constant
open-questions.tsv	what was not resolved

Top facts

1. A sword hit takes the attacker's sword power in half hearts, limited to -20..20. Shield, item levels and the attacker's AP are not read. Same in 2.05 rev 2, 2.17.1, 2.22, 2.31.
2. A negative power heals only a victim with AP 40 or more; below that it changes no hearts. Hurt mode (mode 3, 40 ticks) starts for every accepted hit, including power 0.
3. The sword-power byte on the wire is 0..4 for the default swords 1..4, and power + 30 (10..50) plus an image name otherwise.
4. A hit is dropped when the level ran the script command `noplayerkilling`, when the attacker's `(tag)` is in the victim's nickname, when the attacker's STATUS has bit 0x02 or 0x20, and when the same attacker was judged earlier in the same second of the day. The hurt packet (40) is stopped by the no-kill level too.
5. The respawn point is the last *saved* level entry. An entry is saved when there is no previous level, or when exactly one of {old level, new level} is in the current gmap's level list (the client multiplies the two list indices with -1 for "absent", so index 0 behaves like "absent": a quirk).
6. In 2.00 to 2.05, an entry that fails that test is still saved when the old and new levels are in different name groups. The name group is the file name without its folder, lower-cased, up to and including the last `§` or `$`; with neither, the group is empty. 2.17.1, 2.22 and 2.31 have no group test; the 2.1 test builds do not contain the delimiter constant.
7. With plain level names, the respawn point is therefore the login spot or the last gmap boundary crossing.

`client-internals/decomp-findings-2026-10-24/01-death-respawn.md`

2.05 rev2: death and respawn (derived)

Source: static analysis of the 2.05 rev2 client, 2026-10-24. Behaviour only; no code is reproduced. Tick is 50 ms. See `player-modes.tsv`, `life-states.tsv` and `constants-205.tsv`.

Trigger

- Any change that leaves hearts at 0 or below starts the death sequence once.
- A per-frame safety net also forces the dead life-state whenever hearts are 0 or below.

Sequence on the dying client

1. A death timer starts at 0 and counts up each tick. It stops at 30 (1.5 s).

2. Life-state becomes "dead", mode goes back to 0 (normal) and the player faces down.
3. The "dead" gani plays for the local player and the script event `playerdies` fires.
4. Controls are ignored until the timer passes 27 (1.35 s).
5. Status prop (18) gets the dead bit (0x08) only when life-state is dead and the timer is at least 28. So other players see the player as dead only after about 1.4 s.
6. No sprite for "dead" goes on the wire. The sprite prop shows plain idle plus direction. The dead state is carried by the status bit only.

Respawn

- After the timer passes 27, any movement or action key triggers the respawn. There is no dedicated respawn packet and no server permission step on this path.
- **Hearts after respawn** (AP = alignment points):

AP	Hearts
below 20	3
20 to 39	min(5, max hearts)
40 or more	max hearts

- **Destination** is the last saved level-entry point (level, x, y, direction). It is updated on every level entry, unless the old and new levels share the same name group (see the gap list), and also at login.
  - If nothing was saved and the client is online: x 30.5, y 30.0, facing down, in the current level.
  - Offline: the default start level `tutorial_b-4.graal`.
  - When the online flag is set, the client re-sends the current level at the current x/y, so the server sees an ordinary level warp (the PLI_LEVELWARP route) rather than a "died" message.
- After respawn the status bit 0x08 clears when life-state returns to playing.

Damage rules that feed this

- Healing only happens at AP 40 or above.
- No damage in swim mode (10) or in the editor/pause state unless forced.
- Hurt state: mode 3 for 40 ticks, "hurt" gani, knockback, script event `playerhurt`. A player already in mode 3, mode 10 or dead cannot be hurt.
- The victim's client decides hits (see `04-sword-hitbox.md`). When hearts reach 0 or below after a sword hit, an explosion hit or a UDP hurt, the victim client sends PLI_CLAIMPKER (14) carrying the attacker's player id.

Anti-cheat note

Hearts, AP and max-power sit in an attribute array that the client stores obfuscated twice and cross-checks. A mismatch raises "memory problems error 1/2". Editing hearts from outside the client is therefore detected by the client itself.

Gaps

- The exact "same name group" rule for not updating the saved entry point was only partly traced.
- The server-side reaction to CLAIMPKER is not in the client binary.

client-internals/decomp-findings-2026-10-24/02-sprites-poses-emotes.md

2.05 rev2: sprite index, poses and emotes (derived)

Tables: gani-name-table.tsv, gani-short-codes.tsv, gani-to-step.tsv, sprite-step-table.tsv, status-bits-prop18.tsv, player-modes.tsv.

How the sprite prop (17) is built

- Wire value = 32 + direction + 4 x S.
- S comes from the current animation step through the step table, only when the active gani is the default "def" gani and the step is 38 or lower.
- Hurt is special: S = 32 (value 0x80 + direction as seen in the byte).
- Other ganis do not change the sprite number. (Inferred: they travel by the gani prop.)
- Each gani name maps to a step by a small rule list (gani-to-step.tsv): walk is frame+1, sword/ridesword/spin are 9 + frame mod 5, push 14+frame, pull 20+frame, grab 19, lift 23, carry still 24, carry 25+frame, shoot 33, ride variants 34 to 36, sit 37, sleep 38, hurt 39, dead 40.
- Caveat: the "only for def" condition was read from the sender and is documented as observed. The same sender sets everything to 0 for unknown names.

Swim, lava and dead on the wire

- There is **no swim sprite on the wire**. Swim, lava and dead are all sent as sprite 0 (idle) plus direction.
- Swim is local mode 10, entered on tile type 11 with a splash. Local gani "swim" is name-table index 28 and "lava" is index 29, but neither has a sprite value.
- Tile type 12 shows the "lava" gani.
- Inferred: other players can tell a swimmer apart only through the gani prop, not the sprite.

Gani name table

31 names, index 0 to 30 (gani-name-table.tsv). The short-code table (gani-short-codes.tsv) maps substrings to one letter and is used to shorten cached gani file names.

Emotes (Ctrl+letter)

- Holding Ctrl and pressing A to Z (while no text field has focus and the player is not dead/locked) calls the emote routine with that letter.
- The routine lowercases the letter and walks the loaded file list. It takes the **first file whose name starts with** emoticon_<letter> and whose extension is png, mng or gif. (A prefix match, so Ctrl+A matches emoticon_afk_stay if that file is first in the list.)
- The image is placed at the player's x+2, y-5 (a little lower if a second flag is set) and shown for 50 ticks (2.5 s).
- The emote is data driven. No letter-to-name table exists in the client, and there are no hard-coded AFK/BRB/Zzz strings.
- **No network send was found on the key path.** The PLI_SHOWIMG (24) sender is reached only from the script command showimg. How other players see an emote (the captured AFK image) is therefore not explained by the key handler. Treat that as open.
- The same routine and file-name scheme exist in 1.39, so emotes are not new in 2.05.

Status prop (18)

See `status-bits-prop18.tsv`. Dead (0x08) is delayed as described in `01-death-respawn.md`.

`client-internals/decomp-findings-2026-10-24/03-player-list-popup.md`

2.05 rev2: F7 player list, right-click popup, person action (derived)

Player list window

- Stay-on-top form with a multi-select list view. A panel caption reads "--- N online ---". Buttons: Profile, Options.
- Popup menu items in order: **Message** (default item), **History**, **Buddy**, **Ignore**, **Profile**.
- Right-click selects the row under the cursor, then sets the Ignore and Buddy check marks to "checked if all selected players are in that list".
- Double-click or Enter: opens the read-PM window if a PM from that player is pending, otherwise the compose window.

Actions

Item	Behaviour
Message	Builds the recipient list from all selected rows and reuses an existing PM window for the same set. Recipient encoding: a count as two bytes, then two bytes per player id, at most 50 (0x32) recipients.
History	Opens the message history window (not traced further).
Ignore	Toggles each selected account in the ignore list and saves `ignoredaccounts.txt` and the buddy list.
Buddy	Toggles the buddy list (`buddyaccounts.txt`). Ignore and Buddy are mutually exclusive.
Profile	Uses only the first selected player. If online, sends PLI_PROFILEGET (80).

- Messages from ignored players get the auto reply string "Your messages are ignored." (the string exists; the send condition was not traced).
- PM window titles: "Send Message" and "Read Message".
- Options window: 5 check boxes. Profile edit window: 10 fields.

"Person action" is not a player menu

`TPersonAction` is the "Set Npc's Action" script editor used by the level editor. It has nothing to do with right-clicking a player. There is no per-player action form (trade, duel and so on) in the 2.05 client.

Not found

- No right-click handler on a player sprite in the game world was found in 2.05.

`client-internals/decomp-findings-2026-10-24/04-sword-hitbox.md`

2.05 rev2: sword swing and hit box (derived)

Starting a swing

Preconditions: weapons allowed by the level or server, sword power not 0, not swimming without a horse, and not already in the charged-spin mode. - Normal swing: mode 6. Charged spin: mode 13 (0xd); needs the has-spin flag. - Plays the sword, ridesword or spin gani and a sound, then checks for a cuttable bush at the sword tip. - The sprite sent to others follows steps 9 to 13 (sprites 6 to 10) of the swing; see `02-sprites-poses-emotes.md`.

Who decides a hit

The **victim's client** evaluates the hit and applies damage. The attacker only broadcasts its position, direction and sprite. A hit counts when all of these hold: - the attacker is on step 9 to 13 (sprites 6 to 10); - the victim is 0 to 4 tiles ahead of the attacker in the facing direction; - the sideways offset is under 2 tiles; - the victim's AP is not exactly 100 (such players cannot be hit); - the victim is currently hurtable (not in mode 3, mode 10, dead or editor).

Damage

Inferred: sword power divided by 2 in hearts. Not confirmed from a capture.

Explosions

A bomb hit needs the victim within 1 tile on both axes of the blast, with knockback of 0.5 or 1.0 tile.

Gaps

- Exact damage formula and the timing of the swing frames in milliseconds are not confirmed; frame durations come from the gani files, not the client.

`client-internals/decomp-findings-2026-10-24/05-205-props-misc.md`

2.05 rev2: other facts (derived)

- **Prop 10** is the bow image slot (pre-2.1 meaning), as in the existing prop table. No new meaning was found.
- **Swim state** is a client mode (10), not a prop. It is visible to others only through the animation.
- **Send helper** (also in 1.39): a message whose payload contains a newline is silently dropped, since the newline is the message terminator. The helper prefixes the id byte (id+32), adds the newline, queues, and flushes when the queue would pass 4095 bytes.
- **Disconnect on overload**: if more than 256 KiB (0x3ffff) of compressed output is pending (1.39), the client closes the link. The 2.05 figure was not checked.
- **Obfuscated attributes**: see `01-death-respawn.md`.
- **Respawn and CLAIMPKER** are explained in `01-death-respawn.md`.

client-internals/decomp-findings-2026-10-24/06-first-graal.md

1st Graal (Sept 1998 Delphi beta): protocol summary (derived)

Checked against `relay-1998.md` and `early-clients.md`.

Connection and login

- Default relay host the loopback address, port **14801**, through the event-driven socket component (matches the existing notes).
- No login token, no account, no password, no compression, no cipher. Lines are text ending in a newline; first character is the command.
- Join line: `N<game>#def`. The client always sends the name `def` and sends the real nickname later in a `D` payload.

Game (room) name: new detail

- The constant `Zelda2` is only used in one branch. In the other branch (selected by an argument given when the network client object is created) the client builds the room name from the current level file's base name: `Z` + base + `2`, for example a level `level1.zelda` would give a room `Zlevel12`. Right after that it runs a follow-up routine (not traced).
- Which branch is used for normal play was not traced. The two-client local mode (two client objects exist) is a likely user of the difference.
- The existing note says every 1st/2nd Graal join uses the constant `Zelda2`. That is true for one branch only.

D payload types received

The receive handler has cases for A B C D E F G H L M N O P R S T W (17 letters). That agrees with the existing note, which lists 16 sent types plus a receivable `D`. No other types.

Other confirmed facts

- Save files start with the marker `Graalsav1`.
- Level files use `.zelda` heavily and `.graal` in a few places; some built-in level names are tested by hard-coded comparisons (cave, house, village, level numbers).
- Web levels: the client has an HTTP check for "http" in a level path.

Not covered

Lobby line handling beyond I / + / - / D (P and C were not re-read), and feature behaviour (combat, items).

client-internals/decomp-findings-2026-10-24/07-graal-139.md

Graal 1.39 (Aug 2000 build): protocol summary (derived)

Checked against `versions-133-139.md`, `version-tokens.md` and the 1.39 entry in `v133-v139.toml`.

Login

- Host `<list-host-classic>`, port `14900` (strings in the binary, port parsed at connect time). No listserver step.
- Socket: TCP_NODELAY-style option set, non-blocking after connect.
- Login message is packet id 0 with payload: token `GNW07080` (built one character at a time), then one byte (account length + 32) and the account name, then one byte (password length + 32) and the password. Default account string is "guest". **Same as the existing note.**
- Level warp style join with id 0 and id 30 packets exist for offline `.code` weblevels in the join routine.

Framing and encryption

- Each message: id byte (id+32), payload, newline. A payload that contains a newline is silently dropped by the send helper (new detail, also true in 2.05).
- Queue is flushed when it would pass 4095 bytes. Each flush is zlib-compressed behind a 2-byte big-endian length. Sent to the socket in slices of 256 bytes.
- If more than 0x3ffff bytes (about 256 KiB) of compressed output are pending, the client closes the connection.
- No cipher and no type byte (generation 2). **Matches** the notes.

Packet ids

- Client to server ids passed as constants: 0 to 5, 7 to 24, 26 to 33, 80, 81. The existing list also has 6 (sent from a register-based site) and has no 25. **Matches.**
- Server to client dispatch handles ids 0 to 41, 43, 75, 100 (wire bytes 32 to 73, 75, 107, 132). **Matches** the existing `plo_known` list. Id 100 (raw data) and 75 are in the list.

Features seen in the binary

- DirectDraw and DirectSound through a Delphi DX component set, joystick and force-feedback classes, MNG and GIF animation, DDE classes (library code).
- Emoticon lookup by `emoticon_<letter>` file name with png/mng/gif extensions already exists (same routine as 2.05). Consistent with the existing "emoticons from 1.38".
- Sound files, shield/sword gif sets 1 to 4, `levels\webgifs\`, `levels\recordpos.txt`, `weblevels\`.

Death, hurt, sword and wall (filled 2026-10-31)

See `../../version-profiles/matrix/rules-139.md`. Death timer 0 → 30, STATUS dead bit and respawn key after counter > 27, hurt mode 40 ticks with five knockback steps, sword box 0–4 / <2, six-probe walk. Tag **[bin]**; hurt timings also **[lab]** in `../player-mannerisms-live.md`.

Not re-verified

The F7 popup items for 1.39 (menu strings are built character by character).

`client-internals/decomp-findings-2026-10-24/08-contradictions.md`

New facts and contradictions against Graal_analysis and the laundry notes

Contradicts or corrects

1. **Emote lookup (laundry 11-emotes-ganis.md step 4)** assumed Ctrl+letter maps to named files and sends PLI_SHOWIMG (24). In 2.05 the key path is local only (no packet). The lookup is a *prefix* match on `emoticon_<letter>` in the file list. SHOWIMG (24) is sent only by the script command `showimg`.
2. **1st Graal room name** is not always the constant `Zelda2`; one branch builds `Z` + level base + `2` (relay-1998.md says 1st and 2nd use `Zelda2`).
3. **TPersonAction** is the NPC "Set Npc's Action" script editor, not a player action menu.

New (not in existing notes)

- Death sequence timings (timer cap 30, input lock until 28, dead status bit from 28), respawn hearts by AP, respawn destination rule, no respawn packet.
- Wire sprite table (step to sprite) and the gani name to step rules; no swim sprite on the wire.
- Status-bit meanings (prop 18) and mode values.
- Sword hit evaluated on the victim; reach 0 to 4 tiles, lateral under 2, AP 100 immune.
- F7 popup item order for 2.05 and PM recipient encoding.
- Send helper drops payloads that contain a newline (1.39 and 2.05).

Confirmed (no change)

- 1.39 token, host and port, login layout, no cipher, framing, send id list, receive id list.
- 1st Graal port 14801, `NZelda2#def`, `Graalsav1`, D type set.

Labels

The upload labels for the three bundles do not match their contents (the 2.05 build was not in the folder labelled for it). File names were used to decide.

`client-internals/decomp-findings-2026-10-24/README.md`

Decompile findings, 2026-10-24 (derived facts only)

Built from three Ghidra decompiles (2.05 rev2, Graal 1.39, 1st Graal). Nothing here quotes code or addresses.

File	Topic
01-death-respawn.md	zero-hearts handler, respawn, damage rules
02-sprites-poses-emotes.md	sprite prop, poses, swim, emotes
03-player-list-popup.md	F7 list, popup, PM recipients, TPersonAction
04-sword-hitbox.md	sword swing and hit evaluation
05-205-props-misc.md	other 2.05 facts
06-first-graal.md	1st Graal protocol summary
07-graal-139.md	1.39 protocol summary
08-contradictions.md	new facts and contradictions

Tables (TSV): sprite-step-table, gani-to-step, gani-name-table, gani-short-codes, status-bits-prop18, player-modes, life-states, constants-205.

Gaps still open

- Wire representation of swim (none found) and how others see emotes.
- Damage formula (inferred sword power / 2).
- Meaning of a few unlabelled mode values. (Status bit 0x20 is now answered: `../status-bit-0x20.md`.)
- Exact "same name group" rule for the saved respawn point.
- 1st Graal room-name branch selection; 1.39 popup, death and sword.

`client-internals/decomp-findings-2026-10-25-e/01-request-lists.md`

Request lists

Lists

Each client keeps three lists of file names: - A (wanted): names asked for with `PLI_WANTFILE` (23, name only). - B (update asked): names asked about with `PLI_UPDATEFILE` (34: GINT5 time, then name). - C (settled): every name a reply came back for.

Asking

- No local copy: if the name is not in A, append it; send the request when A now holds at most four names.
- A local copy: if the name is in neither B nor C, append it to B; send when B holds at most four. A name in C is skipped, so each local file is checked once per connection.
- The time sent is the last-write time of the local file, Unix seconds. Lab: it matched the file's mtime for the sword, shield and body images at login.
- Builds whose packet set has no id 34 (1.33 to 1.41) never check a local copy.

Replies

A reply (file, up to date, send failed, or the end of a large file) removes the name from A and B. If the removed position was among the first four and names are still waiting, the one now in fourth place goes out. Then the name is added to C. The pair A/B releases independently.

Who asks

- Image loaders and animation loaders: local file, use it and check it; no file, ask for it.
- Sound loader: ask only when missing. The lab shows no check for a sound that exists.
- Levels: no file transfer; board packets and the level modtime exchange, plus the persistent web-level cache.
- A gmap file: checked like an image when present (lab, 2.22 gmap runs); asked for when missing (lab).

Failed names

A name answered with "send failed" is not asked for again on the same connection (lab: a missing step sound is asked for once however often the player walks). What the client stores for it beyond that is not known.

Cross-build

The list code is the same in 2.00, 2.05 rev 2, 2.17.1, 2.22 and 2.31. In 2.05 rev 2 and 2.22 the request-side function bodies were hard to read in the export; the request rules for those two come from the matching reply-side code and the lab, and are marked inferred in the tsv.

`client-internals/decomp-findings-2026-10-25-e/02-save-rules.md`

Save rules

Whether a file is written

A received file is first held in memory as a cached stream. It is written to disk only when the `saveimages` option is on; the option lives in the client's config file and defaults to on.

Where

1. If the client already has a file under that name, it is replaced where it is. The lookup tries the name as a path from the game folder, then the client's own file-name index (name to path and time).
2. Otherwise by name, case-insensitive:
    - `.wav`: `sounds`.
    - from 2.17.1 on: `.gmap`, and `.txt`, `.gif`, `.png`, `.mng` with `map` in the name: `maps`.
    - everything else under `levels`, in a folder by name: `bomys` (name starts `bomy` or `b_`), `ganis` (`.gani`), `emoticons`, `hats` (`hat` plus a digit as the fourth character), `heads`, `bodies`, `swords`, `shields` (the name contains `head`, `body`, `sword`, `shield`), else `webgifs`.
3. 2.05 rev 2 has no `maps` rule: its downloads all go under `levels`. 2.1 to 2.16 are inferred to match the later builds (no build in that range was read).

Lab: a downloaded gmap on 2.22 was saved as `maps/<name>`; images and animations went to the `levels` folders above.

Time of the saved file

From 2.1 the file reply carries a GINT5 last-write time; the client sets it on the saved file. Before 2.1 the reply has no time, so the file's time is the time of the download.

Names

Compared without regard to case. The fold is plain A–Z only, and the index then compares the folded bytes exactly; see `04-name-index-case-link.md`. Lookups use the same normalisation as the loaders.

`client-internals/decomp-findings-2026-10-25-e/03-open-points.md`

Open points

- ~~Persistent level cache (`GWEBL001`): format already in the notes; its interaction with `PLI_LEVELMODTIME` was not read here.~~ **Closed.** The write is on **server** packet 39 (`PLO_LEVELMODTIME`, not a client `PLI_`): `../../formats/gwebl001-cache.md` and `../../protocol/packet-layouts.md` **[2.05]**. Client

level entry still uses board packets plus that modtime exchange (see also request-lists: levels use no file-transfer ids).
- Login default checks: the lab shows a check for three body-part images and a gmap; the full set (and whether they share a frame) needs a capture with a full and an empty data folder.
- ~~Scan order of the file-name index when two folders hold the same name.~~ **Closed.** No folder is preferred. 2.22 keeps the newer last-write time (an equal time keeps the first file the walk met); 2.05 rev 2 keeps the first hit of a depth-first search. A copy in the current directory wins over either subfolder (the game folder in 2.22; `levels` in 2.05 rev 2). `04-name-index-case-link.md` **[bin]**.
- ~~Save folders for 1.39 and 2.00 (save routines not read), and for 2.1 to 2.16 (inferred).~~ **Closed for 1.39 and 2.00** in `../decomp-findings-2026-10-25-k/02-save-folders-by-build.md` **[bin]**. **Still open for 2.1 to 2.16.** A 2026-11-01 listing of the client archive still has no build between the 2.1 test builds and 2.22.
- ~~Whether builds before 2.1 re-send the check for a settled name. Their reply has no time, so "up to date" has no form there; inferred only.~~ **Closed** for re-send: settled-list C skips a second check on the same connection in 2.00 and 2.05 — 01-request-lists.md **[bin]**. (Before 2.1, "up to date" is folded into send-failed id 30; that reply form is already noted there.)
- ~~What an empty cached stream after "send failed" does.~~ **Closed.** Save routine returns at once for a missing or zero-length stream: nothing written, nothing deleted — `../decomp-findings-2026-10-25-k/02-save-folders-by-build.md` (The guard) **[bin]**.
- ~~Whether the case folding is plain ASCII.~~ **Closed.** Yes: only `A`–`Z` are folded, then the index compares the bytes exactly. The locale compare is not used for file names. Same note **[bin]**.

`client-internals/decomp-findings-2026-10-25-e/04-name-index-case-link.md`

File-name index, case fold, and the packet 40 link

Static read of the 2.05 rev 2 and 2.22 clients. No capture and no client was run. **[bin]**

Case fold

File names are folded with the plain A–Z fold: each byte from `A` to `Z` is turned into `a` to `z`, and every other byte is left as it is. Accented letters are not folded, so they do not match across case.

The file-name index stores that folded base name (the part after the last `\` or `:`). A lookup folds the same way and then compares the bytes exactly. The runtime also has a locale compare, used by ordinary sorted string lists; the file-name index does not use it.

The same A–Z fold is applied before the folder tests (`head`, `body`, `sword`, `shield`, `hat` plus a digit, the `.code` skip, and the `pms` / `profiles` / `weblevels` skip below). **[bin]**

Which copy is used

Folder names do not decide the winner. The same base name in two folders is decided as follows.

Both builds, before the index: if a file of that name exists in the current directory, that file is used.

- 2.22 sets the current directory to the game folder at startup, so a copy in the game folder itself beats either subfolder.
- 2.05 rev 2 sets the current directory to the game folder's `levels` directory, so a copy placed directly in `levels` beats `levels/bodies` and `levels/webgifs`. Opening a world can point the current directory at that world instead.

2.22. At startup the client walks the game folder and fills a name-to-path index.

- In each directory the files are recorded in the order the directory listing returns them, and only then are subdirectories entered, in that same order.
- A directory whose parent is the game folder and whose folded name is `pms`, `profiles` or `weblevels` is not entered. A file whose folded extension is `.code` is not recorded.
- The key is the folded base name. The stored path is kept with the last-write time from the directory listing. A later time replaces the path. An equal or earlier time leaves the path already stored, which is the one the walk met first.

2.05 rev 2. A full-folder indexer with the same skip rules is in the program (it would keep the first name and ignore a later duplicate), but no call site in this image references it. What runs is the lookup:

1. The name as a file in the current directory, as above.
2. Else a path remembered from an earlier lookup, if that file is still there.
3. Else a depth-first search, first of an extra root recorded when a world file is opened, then of the game folder. In a directory, a file of the wanted name is taken before any subdirectory is entered. Subdirectories are tried in directory-listing order, and the search stops at the first hit. The same three folders and `.code` files are skipped. The hit is remembered for the next lookup.

So for two different bytes under the same name, 2.22 draws the newer file, and 2.05 rev 2 draws the first file the search meets. **[bin]**

Packet 40 link

Packet 40 (and the same test on the longer hit form) is dropped unless one integer on the attacker's record equals the same integer on the victim. A new player is created with that integer 0, so two ordinary players pass.

The horse is a different slot. Setting or clearing the horse image does not write the integer. A player who only has a horse still carries 0 and still matches another ordinary player. **[bin]**

In 2.22 the integer is the layer/parent link:

- It stays 0 until a `layer` property is applied. The value is then the layer number minus one. If that character has a parent, the parent's link is added as well.
- A spawned follower copies the source's link, or one more than the source's link. That is the follower's own link, not the player's. The player's link is unchanged by the spawn.

2.05 rev 2 has the same equality test and the same zero at creation, and its horse image is likewise a different slot. It has no `layer` property string. **[bin]**